

AWS Cloud Practitioner

CLF-C02 Arabic

*Based on the [CloudKode](#) YouTube [Course](#) by
[Mohamed Rizk](#)*



Table Of Content

Video 0 – Introduction	22
Traditional Data Centers	22
Core Infrastructure Components	22
Server Components	23
Network Connectivity	24
Additional Requirements	24
Upfront Investment	25
Public Cloud Benefits	26
Cost Saving	26
Pay As You Go	26
Scalability and Flexibility	26
Scaling Types	27
Scale Up	27
Scale Down	27
Scale Out	27
Scale In	28
Accessibility	28
Disaster Recovery and Backup	28
Automatic Updates and Maintenance	28
High Availability	29
Cloud Providers	30
Cloud Deployment Models	30
Private Cloud	30
Public Cloud	31
Hybrid Cloud	31
Shared Responsibility Model	32
On-Premises Responsibilities	32
Cloud Service Models	33
Infrastructure as a Service	33
Platform as a Service	34
Software as a Service	34

AWS Regions	35
Region Selection Factors	36
Latency	36
Service Availability	36
Cost	36
Compliance	36
Availability Zones	37
Course Services Overview	38
EC2 Instances	38
Storage Services	38
Permissions	38
Database Services	39
Load Balancer	39
Monitoring Services	40
Additional Services	40
AWS Account Setup	41
AWS Free Tier	41
Resource Cleanup	41
AWS Credits	42
Video 1 – EC2	43
EC2 Instances	43
Instance Configuration	43
Instance Categories	44
General Purpose	44
Compute Optimized	45
Memory Optimized	45
Storage Optimized	45
Accelerated Computing	46
EC2 Payment Options	47
On-Demand	47
Reserved Instances	47
Spot Instances	48
Savings Plans	48
Compute Savings Plan	48
EC2 Instance Savings Plan	49
Dedicated Host	49

Dedicated Instances	49
Video 2 – EC2 LAB	50
EC2 Networking Foundations	50
Virtual Private Cloud	50
Subnets	51
Availability Across Zones	52
Private IP Address	52
Internet Connectivity	53
Internet Gateway	53
Route Table	54
Public IP Address	55
Network Security Controls	55
Security Groups	56
Stateful Behavior	56
Instance-Level Protection	56
Allow Rules	56
Inbound and Outbound Rules	57
Network Access Control Lists (NACLs)	58
Stateless Behavior	58
Rule Priority	58
Creating a VPC	59
VPC Configuration	60
Default Network ACL	60
Route Table Configuration	61
Launching an EC2 Instance	62
Instance Name	62
Operating System	62
Instance Type	62
Key Pairs	63
EC2 Network Settings	64
VPC Selection	64
Subnet Selection	64
Public IP Assignment	64
Web Security Group	65
SSH Rule	65
HTTP Rule	65

User Data	66
Launching and Viewing the Instance	67
Security Rules Review	68
Testing the Web Server	69
Connecting to the Instance	70
EC2 Instance Connect	70
SSH From macOS	71
SSH From Windows	72
SSH Source Restrictions	72
Video 3 – EC2 Storage types	73
Amazon Machine Images	73
Benefits	73
AMI Types	73
EBS Volumes	74
Data Persistence	75
Instance Attachment	75
Availability Zone	75
Network USB Stick Analogy	75
Delete on Termination	76
Root Volume Behavior	76
Additional Volume Behavior	76
EBS Snapshots	77
Snapshot and Restore Process	77
EBS Volume Types	79
GP2 and GP3	79
IO1 and IO2	79
ST1	79
SC	79
EBS Multi-Attach	80
Requirements	80
Instance Store	81
Performance	81
Data Loss	81
Use Cases	81
EBS and EFS Storage Models	81
Elastic File System	82

Pricing and Compatibility	82
File Sharing	83
EFS Performance Modes	84
General Purpose	84
Max I/O	84
EFS Throughput Modes	84
Bursting	84
Provisioned	84
Elastic	84
EFS Storage Tiers	85
Standard	85
Infrequent Access	85
EFS Availability	86
Standard Availability	86
One Zone	86
EBS and EFS Comparison	87
EBS Pricing	88
EFS Pricing	88
Video 4 – S3	89
Amazon S3	89
Main Use Cases	89
Buckets and Objects	89
Bucket Naming Rules	89
Global Service and Bucket Region	90
Object Paths	90
Versioning Overview	91
Creating an S3 Bucket	91
Bucket Configuration	91
Uploading an Object	92
Object URL and Access	92
Presigned URL	93
S3 Access Control	93
User-Based Access	93
Resource-Based Access	93
Bucket Policy Structure	94
Resource	94

Effect	94
Action	94
Principal	95
Public Read Policy Example	95
Applying a Bucket Policy	96
Disabling Block Public Access	96
Using the Policy Generator	96
Static Website Hosting	98
Uploading Website Files	98
Enabling Static Website Hosting	98
S3 Versioning	100
Version History	100
Delete Marker	100
Stopping Versioning	101
Enabling Versioning	101
Existing Object Version IDs	102
Uploading New Versions	102
Restoring Deleted Objects	103
S3 Replication	104
Replication Types	104
Cross-Region Replication	104
Same-Region Replication	105
Creating a Replication Rule	105
Storage Classes	107
S3 Standard	107
S3 Standard-Infrequent Access	107
S3 One Zone-Infrequent Access	107
S3 Glacier	108
Glacier Instant Retrieval	108
Glacier Flexible Retrieval	108
Glacier Deep Archive	108
S3 Intelligent-Tiering	108
Changing Storage Classes	109
Lifecycle Rules	109
S3 Encryption	110
Server-Side Encryption	110

Client-Side Encryption	111
Video 5 – Identity & Access Management	112
Identity and Access Management	112
Root Account	112
Users and Policies	112
Groups and Policies	113
Multiple Group Membership	114
IAM Policies	115
Effect	115
Action	115
Resource	115
Multi-Factor Authentication	117
Virtual MFA	117
Hardware MFA	118
IAM Console	119
Creating an Admin Group	119
Creating a User	119
User Password	120
Assigning User Permissions	120
Signing In as an IAM User	120
Account Alias	121
Assigning Users to Groups	121
User Security Settings	122
Password Policy	122
AWS Access Methods	123
AWS Management Console	123
AWS CLI	124
AWS SDK	124
AWS CLI Configuration	125
Installing AWS CLI	125
Creating an Access Key	125
Configuring AWS CLI	126
Testing S3 Access	126
IAM Roles	127
Role Purpose	127
Creating a Role	127

Assigning Role Permissions	128
Video 6 – Databases & Analytics	129
Database Fundamentals	129
Databases and Traditional Storage	129
On-Premises Database Responsibilities	129
Fully Managed Databases	130
SQL and NoSQL Databases	131
SQL Databases	131
NoSQL Databases	132
Amazon RDS	133
RDS Benefits	133
RDS Limitation	133
Application Architecture	133
RDS Deployment Options	134
Read Replica	134
Multi-AZ	135
Multi-Region	135
Creating an RDS Database	136
Creation Method	136
Database Engine	136
Database Template	136
Database Settings	137
Instance Class	137
Storage Configuration	137
Connectivity	137
Reviewing the Database	138
Database Monitoring	138
Database Operations	139
Amazon Aurora	139
Aurora Serverless	139
Amazon ElastiCache	140
Amazon DynamoDB	141
DynamoDB Accelerator	141
Creating a DynamoDB Table	142
Adding Items	143
Flexible Schema	143

Additional Database Services	144
Amazon DocumentDB	144
Amazon Neptune	144
AWS Database Migration Service	145
Amazon MemoryDB for Redis	145
Analytics Services	146
Amazon Redshift	146
Amazon EMR	146
Amazon Athena	146
Amazon QuickSight	146
AWS Glue	147
Amazon Kinesis	147
Video 7 – LoadBalancer	148
Load Balancing Fundamentals	148
The Problem with a Single Server	148
Load Balancer Concept	149
Application Load Balancer	150
Target Groups	150
Path-Based Routing	150
Host-Based Routing	151
Health Checks	151
Client IP Address	151
X-Forwarded-For	151
Target Types	152
Network Load Balancer	153
Performance	153
Static IP Address	153
Target Types	153
Gateway Load Balancer	154
Sticky Sessions	155
Session Cookies	155
SSL Certificates	156
Public and Private Keys	156
AWS Certificate Manager	156
Server Name Indication	157
Auto Scaling Groups	158

CloudWatch Alarms	158
Minimum Capacity	158
Desired Capacity	158
Maximum Capacity	159
Dynamic Scaling Policies	160
Target Tracking	160
Simple or Step Scaling	160
Scheduled Scaling	160
Video 8 – VPC Networking	161
Amazon VPC	161
Default and Custom VPC	161
VPC CIDR Range	162
Subnets	162
Availability Zone Distribution	162
Route Tables	163
Route Table Association	163
Private Subnets	164
Outbound Traffic	164
Public Subnets	165
NAT Gateway	165
Private Subnet Routing	166
Internet Gateway	167
VPC Network Architecture	168
Private Subnets	168
Public Subnets	168
Reserved Subnet IP Addresses	169
Public EC2 Connectivity	169
Outbound Internet Access	170
Inbound Internet Access	170
Video 9 – VPC Lab	171
AWS Network Lab	171
VPC Creation	171
Subnet Creation	172
Private Subnet	172
Public Subnet	172
Internet Gateway	173

NAT Gateway	173
Public Route Table	174
Route Table Creation	174
Subnet Association	174
Internet Route	174
Private Route Table	175
Route Table Creation	175
Subnet Association	175
NAT Route	175
Traffic Flow	176
Public Subnet Traffic	176
Private Subnet Traffic	176
Public Web Server Test	176
Instance Configuration	176
Video 10 – Direct connect vs VPN	178
Hybrid Connectivity	178
Site-to-Site VPN	179
Main Components	179
Traffic Flow	179
Deployment Time	179
Limitations	180
AWS Direct Connect	180
Direct Connect Location	180
Connection Architecture	180
Performance	181
Provisioning Time	181
Direct Connect Connection Types	181
Dedicated Connection	181
Hosted Connection	182
Video 11 – SQS,SNS, CloudWatch, CloudTrail, Eventbridge, Config	183
Amazon SQS	183
Core Concept	183
Application Decoupling	184
Producer and Consumer Access	184
Message Retention	184

Visibility Timeout	184
Creating an SQS Queue	185
Standard and FIFO Queues	185
FIFO Queue	185
Standard Queue	185
Queue Configuration	186
Sending a Message	186
Polling a Message	187
Amazon SNS	188
Core Concept	188
Publisher and Subscribers	189
Subscription Types	189
SNS Architecture	189
S3 Notification Example	189
Lambda Subscriber Example	190
Creating an SNS Topic	190
Creating an Email Subscription	190
Publishing a Message	191
Amazon CloudWatch	192
CloudWatch Monitoring	192
Metrics	193
Resource Metrics Examples	193
Logs	194
Alarms	194
CPU Alarm Example	194
Auto Scaling Action	195
EC2 Reboot Action	195
CloudWatch Metrics Lab	196
Viewing EC2 Metrics	196
CPU Utilization Graph	196
Metrics Explorer	197
CloudWatch Dashboard	198
Creating a Dashboard	198
CloudWatch Logs Lab	198
Sending EC2 Logs to CloudWatch	198
CloudWatch Agent Configuration	199

Log Groups	199
Log Streams	199
Viewing EC2 Logs	200
CloudWatch Alarms Lab	201
Creating a CPU Alarm	201
Alarm Action	201
SNS Notification Flow	201
Amazon EventBridge	203
Core Concept	203
Scheduled Jobs	203
Event-Based Actions	204
EC2 State Change Example	204
S3 Event Example	204
Root User Sign-In Example	205
EventBridge and CloudWatch Difference	205
EventBridge Lab	206
Creating a Rule	206
Scheduled Rule	206
EC2 Stop Event	206
Selecting the Target	207
AWS CloudTrail	208
Core Concept	208
Account Activity Tracking	209
Event History	209
Long-Term Log Storage	209
Audit Example	209
CloudTrail Lab	210
Viewing Event History	210
Creating a Trail	210
Selecting Events	211
AWS Config	212
Core Concept	212
Configuration Monitoring	212
Continuous Monitoring	213
Compliance Rules	213
Pricing	213

AWS Config Lab	214
Initial Configuration	214
Recording Frequency	214
IAM Role	214
Log Storage	214
SNS Notifications	215
S3 Public Access Rule	215
Video 12 – Serverless, LAMBDA & API Gateway	216
AWS Lambda	216
Lambda Overview	216
Serverless Execution	217
Automatic Scaling	217
Lambda Pricing	217
Service Integrations	218
Image Resizing Architecture	219
Lab Components	219
Image Compression Scenario	220
Lab Preparation	221
S3 Buckets	221
DynamoDB Table	221
Lambda IAM Role	221
Creating the Lambda Function	222
Function Configuration	222
Adding the Code	222
Configuring the S3 Trigger	223
Trigger Behavior	223
Trigger Setup	223
Testing the Lambda Workflow	224
Uploading the Image	224
Checking the Resized Bucket	224
Checking DynamoDB	225
API Gateway	226
API Gateway Overview	226
API Gateway Use Cases	228
Reading Data from DynamoDB	228
Uploading Images to S3	228

Products API Example	228
Product Data	228
Creating the API Gateway	229
API Types	229
Creating the API	229
API Resources	230
Resource Paths	230
API Methods	230
GET Method	230
Multiple Resources	231
Deploying the API	232
Deployment Stage	232
Invoke URL	232
Testing the Products API	233
Video 13 – Containers, ECS, ECR & EKS	234
Container Services Overview	234
Docker Basics	234
Traditional Application Setup	234
Docker Image	235
Docker Container	235
Docker Registry	236
Amazon ECS	237
ECS with EC2	237
ECS with Fargate	238
Amazon EKS	239
Managed Nodes	239
Self-Managed Nodes	239
Fargate with EKS	239
Amazon ECR	240
Creating a Docker Image	241
Pushing the Image to ECR	242
Authenticating with ECR	242
Tagging the Image	242
Pushing to ECR	242
Creating an ECS Fargate Cluster	243
Creating a Task Definition	244

Compute Resources	244
Task Execution Role	244
Container Configuration	244
Running a Task on the Cluster	245
Task Configuration	245
Networking	245
CPU Architecture Issue	246
Subnet Configuration	246
Running Container	247
Security Group Issue	247
Video 14 – AI artificial intelligence	248
AWS AI Services	248
Amazon Rekognition	248
Object Detection	249
People and Face Detection	249
Text Detection	249
Face Analysis	249
Sports Game Analysis	249
Unsafe Content Detection	250
Rekognition Practical Example	250
Amazon Transcribe	251
Meeting Transcription	252
Amazon Polly	253
Amazon Comprehend	254
Sentiment Analysis	254
Language Detection	254
PII Detection	255
Amazon Comprehend Medical	256
Amazon Polly Practical Example	256
Amazon Comprehend Practical Example	257
Amazon Lex	258
Amazon Kendra	259
Amazon Textract	260
Driving License Example	260
Invoice Example	261
Amazon SageMaker	261

Video 15 – Security, WAF, Shield, Secret Manager, KMS, GuardDuty, Cognito	262
AWS WAF	262
Web Application Firewall	262
Supported Services	263
Web ACL	263
Managed Rules	263
Custom Rules	263
AWS Shield	264
Shield Standard	264
DoS and DDoS Attacks	265
Shield Advanced	265
WAF Practical Lab	266
Application Load Balancer Setup	266
Creating a Web ACL	266
Using Managed Rules	266
Creating a Custom Rule	266
Testing the Custom Rule	267
Data Encryption	268
Encryption in Transit	268
Encryption at Rest	269
AWS Certificate Manager	269
AWS KMS	270
KMS Key Types	270
Customer Managed Key	270
AWS Managed Key	271
S3 Encryption with KMS	271
Object and Key Access	271
KMS Practical Lab	272
Users	272
Creating the KMS Key	272
Creating the S3 Bucket	273
Uploading an Object	273
Testing the Test User	273
Granting Key Access	273
AWS Secrets Manager	274

Amazon Macie	275
Amazon Cognito	276
AWS Artifact	277
Amazon GuardDuty	278
Amazon Inspector	279
Network Exposure Example	279
Amazon Detective	280

Video 16 – CloudFront, Route53, Outpost, Wavelength, local zone **281**

Content Delivery Network	281
CDN Purpose	281
Regions and Availability Zones	283
Region	283
Availability Zones	283
Edge Locations	284
Edge Location Concept	284
Edge Location Distribution	284
Amazon CloudFront	285
Origin Access Control	285
OAC Concept	285
CloudFront Origins	286
Origin Concept	286
CloudFront Scenario	286
CloudFront Practical Lab	287
S3 Origin	287
Creating a CloudFront Distribution	287
Accessing the Cached Object	287
CDN Performance Test	288
S3 Performance	288
CDN Performance	288
Amazon Route 53	289
DNS Concept	290
DNS Record Types	290
A Record	290
CNAME Record	291
Alias Record	291

Route 53 Routing Policies	292
Simple Routing	292
Weighted Routing	292
Route 53 Hosted Zones	293
Public Hosted Zone	293
Domain Registration	293
Private Hosted Zone	293
Route 53 Practical Lab	294
Web Server	294
Creating an A Record	294
Hosted Zone Structure	294
AWS Outposts	296
Outposts Use Cases	297
Latency	297
Data Residency	297
Supported Services	297
Region Connectivity	297
AWS Wavelength	298
AWS Local Zones	299
Local Zone Concept	299
Supported Services	299
Managing Local Zones	300
Video 17 – Organization, Cloudformation, Snowfamily, Billing&Costs	301
AWS Organizations	301
Organization Purpose	301
Management Account	301
Consolidated Billing	302
Organizational Units	302
Service Control Policies	303
SCP Purpose	303
OU Level Policies	303
Account Level Policies	303
Policy Impact	303
AWS Organizations Structure	305
CloudFormation	306

Infrastructure as Code	306
Resource Definition	307
CloudFormation Template Sources	307
AWS Snow Family	308
Snowball Purpose	308
Snowball Device Types	308
Data Migration Scenario	308
Snow Family Job Types	308
Import Data	308
Export Data	309
Local Compute and Storage	309
EC2 Pricing Review	309
On-Demand	309
Reserved	309
AWS Marketplace	310
AWS Pricing Calculator	310
AWS Budgets	311
AWS Cost Explorer	311
AWS Trusted Advisor	312
Performance Example	312
Security Example	312

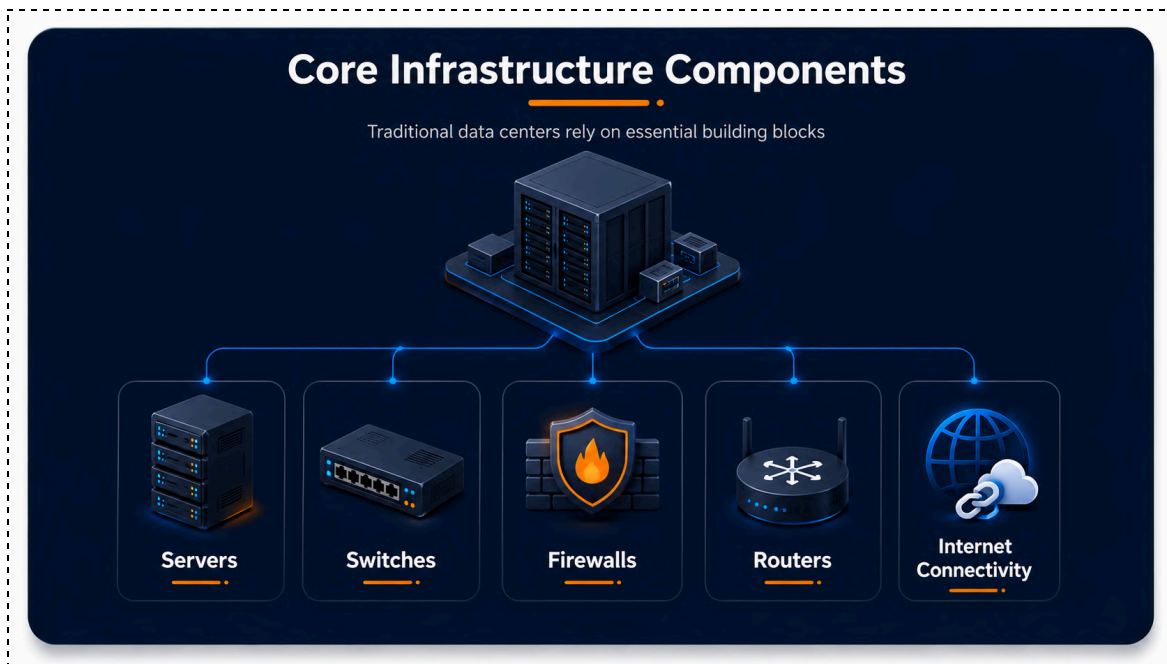
Video 0 – Introduction

Traditional Data Centers

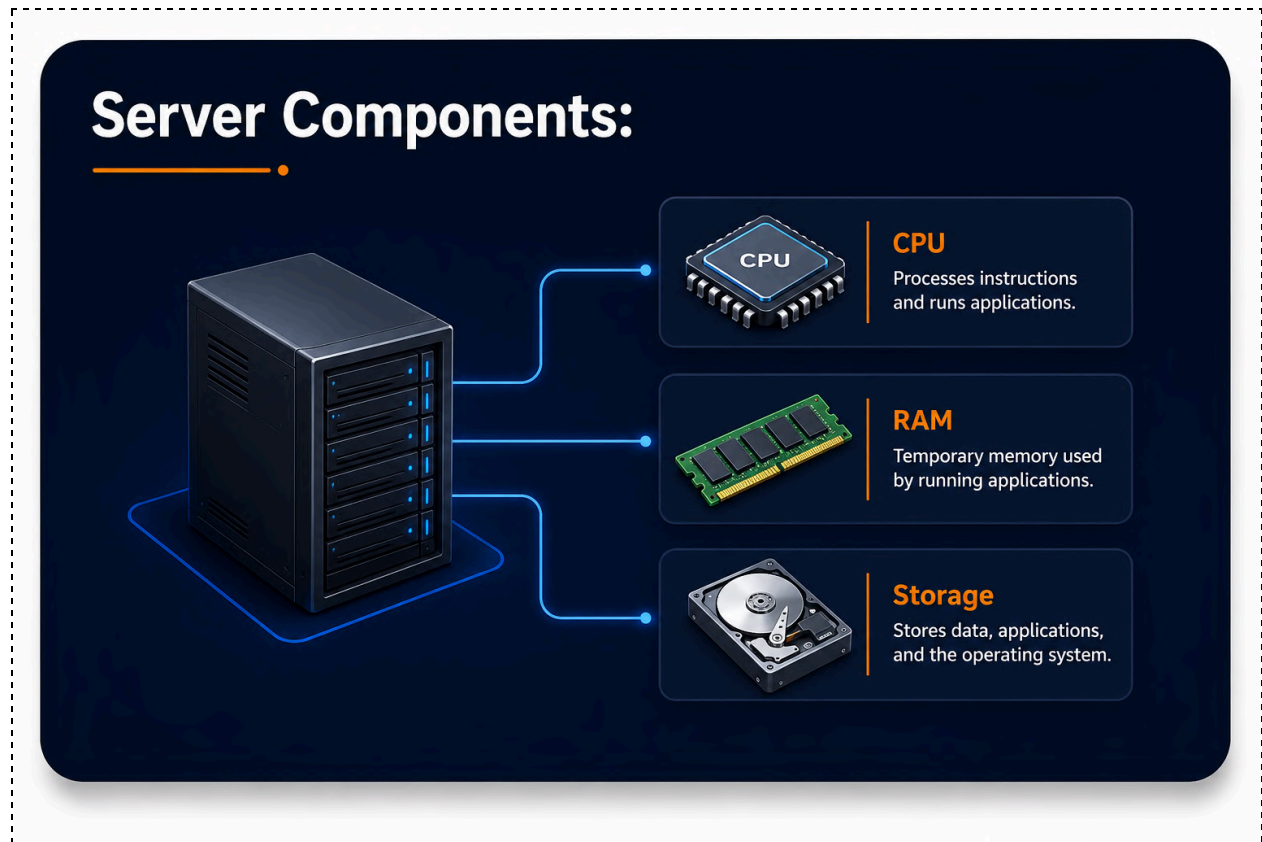
الشركات الكبيرة اللي عندها مجموعة من Servers كانت بتحطها في مكان اسمه Data Center.

Core Infrastructure Components

الـ Data Center كانت بتعتمد على مجموعة من المكونات الأساسية:



Server Components

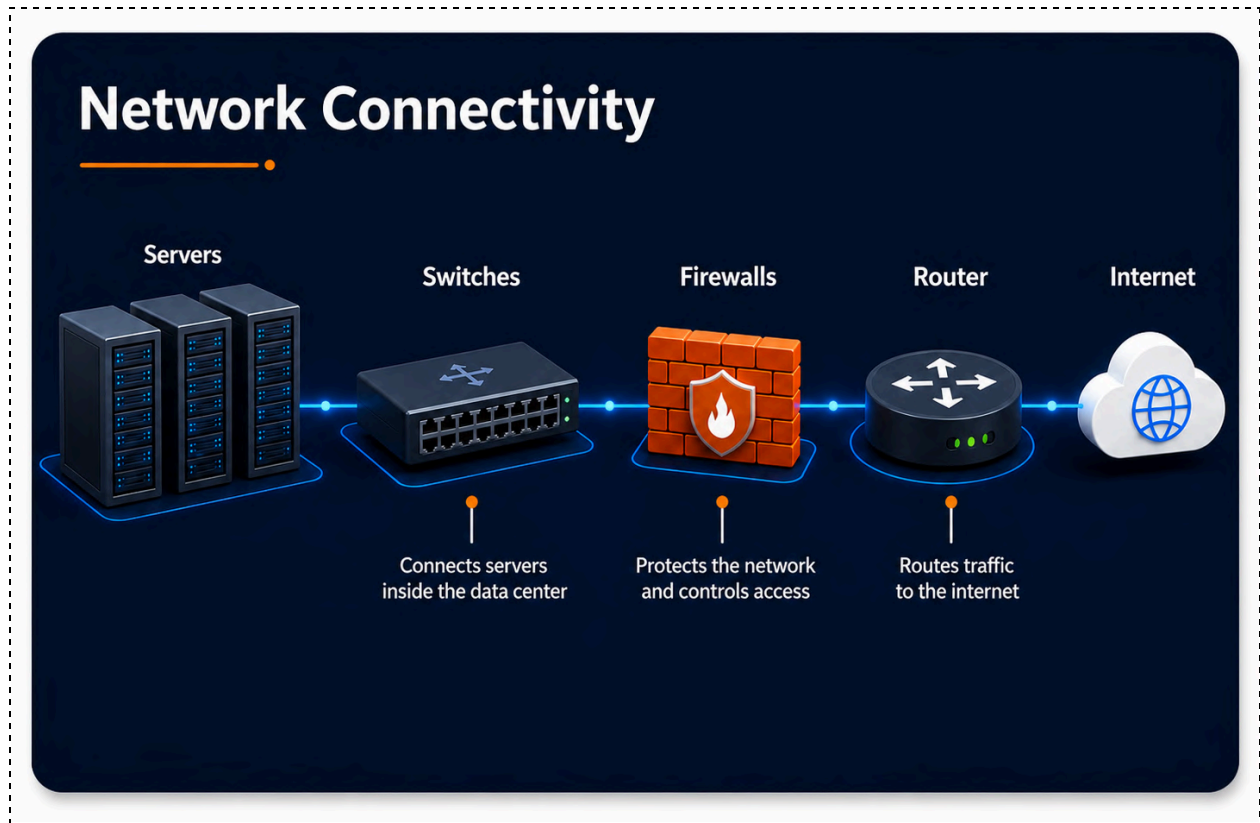


الـ Server نفسه بيتكوّن من:

- CPU.
- RAM.
- Storage.

الـ Servers ممكن تكون Servers عادية أو Databases.

Network Connectivity



الـServers بتكون متوصلة ببعض عن طريق Switches جوه الـData Center.

الـSwitches بتكون محمية على مستوى الـNetwork باستخدام Firewalls، علشان مش أي حد يقدر يعمل Access للـServers.

بعد كده، الاتصال بالإنترنت بيتم من خلال الـRouter.

Additional Requirements

بجانب مكونات الـInfrastructure، كنت محتاج توفر:

- Cooling للمكان.
- Physical Security لتأمين الـData Center.
- Cabling لتوصيل الأجهزة ببعض.
- Software Licenses و Hardware Licenses
- Maintenance و Updates للـInfrastructure

Upfront Investment

إنشاء الـ Data Center كان محتاج استثمار مبلغ كبير جدًا من البداية علشان تشتري الـ Hardware والـ Software وتوفر الـ Licenses وباقي المتطلبات.

Public Cloud Benefits

Cost Saving

أول مشكلة حلها الـ Public Cloud هي تقليل تكلفة الـ Initial Setup. في السيناريو ده، مش محتاج تدفع مبلغ كبير Upfront علشان تبني الـ Infrastructure بالكامل من البداية.

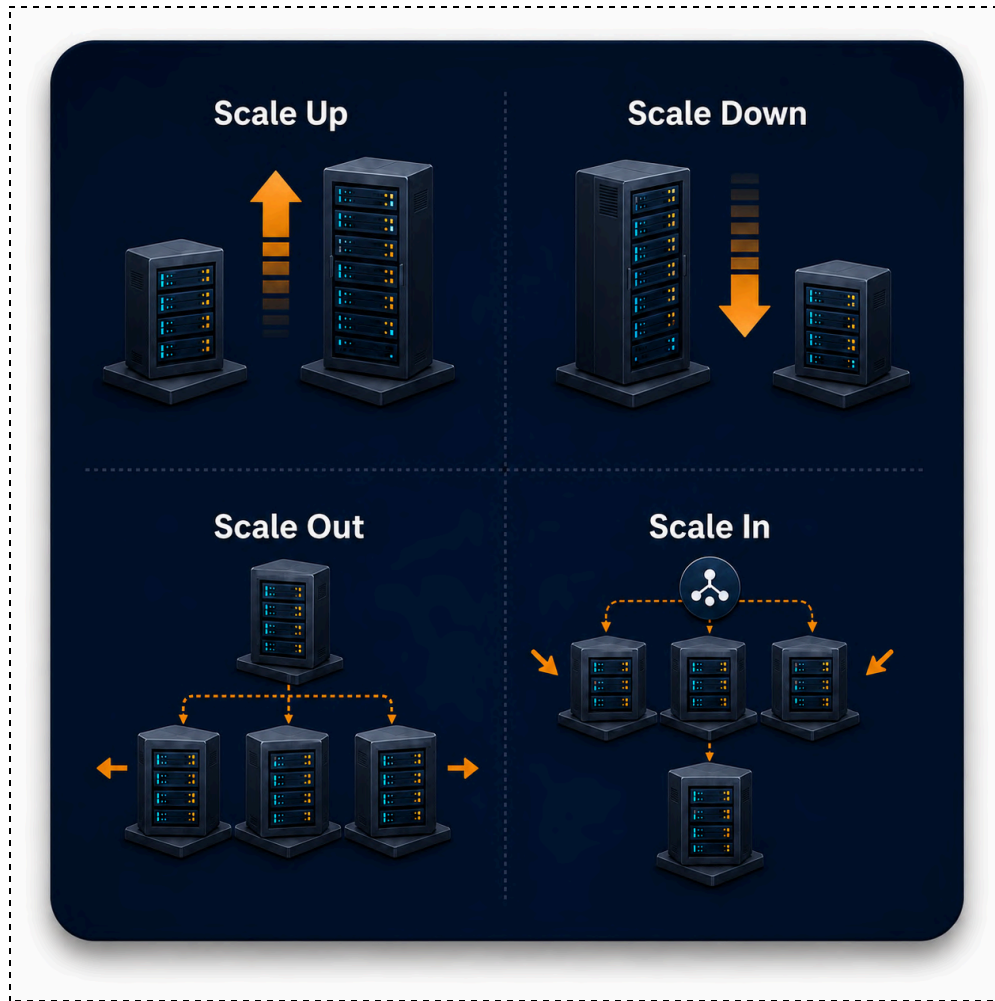
Pay As You Go

واحد من أشهر الـ Payment Models في الـ Public Cloud هو Pay As You Go، رغم إنه مش الـ Model الوحيد. ممكن تطلب Server وتدفع عليه بالساعة. مثلاً، لو تكلفة الـ Server هي 1 دولار في الساعة، هتدفع مقابل عدد الساعات اللي استخدمت فيها الـ Server فقط. لو مبقتش محتاج الـ Server، تقدر تقفله في أي وقت. ده مختلف عن الـ Traditional Data Center، لأنك لو استثمرت مبلغ كبير في المشروع وبعد كده المشروع حصلت فيه مشكلة، ممكن تضطر تعمل Shutdown أو Decommission الـ Data Center وتخسر جزء كبير من الاستثمار. في الـ Public Cloud، ممكن تشغل المشروع لمدة سنة، ولو المشروع مكملش تقدر تعمل Terminate الـ Resources في أي وقت.

Scalability and Flexibility

في الـ Traditional Data Center بتكون مرتبطة بمكان ومساحة محددين. لو الشركة كبرت واحتاجت Servers أكثر، هتضطر تشتري Servers جديدة، وتوصلها، وتضيف الكابلات المطلوبة، وممكن تكون مساحة المكان نفسها محدودة. في الـ Cloud تقدر تعمل Scale الـ Resources بناءً على الـ Demand.

Scaling Types



Scale Up

Scale Up معناه زيادة إمكانيات نفس الـ Server.

مثلاً، لو الـ Server عنده 8 GB RAM، ممكن تستبدله بـ Server عنده 16 GB RAM.

Scale Down

Scale Down معناه تقليل إمكانيات الـ Server.

مثلاً، ممكن تقلل الـ RAM من 8 GB إلى 4 GB.

Scale Out

Scale Out معناه إضافة Servers جديدة بجانب الـ Server الموجود.

بالشكل ده، الـApplication ممكن يشتغل على أكثر من Server، ويكون فيه Load Balancer لتوزيع الـTraffic عليهم.

Scale In

Scale In معناه تقليل عدد الـServers.

مثلاً، لو عندك 3 Servers شايلين الـApplication والـDemand مبقاش عالي، ممكن تشيل Server بن وتخلي الـApplication على Server واحد بدل ما تدفع تكلفة الـServers كلها.

Accessibility

في الـTraditional Data Center، الوصول للـServers وإدارتها بيعتمد على الـRouter والـInternet Connectivity الخاصة بالمكان.

لو حصلت مشكلة في الـRouter أو الإنترنت، ممكن تفقد الاتصال بالـData Center بالكامل.

في الـCloud تقدر تعمل Access للـServices من أي مكان عندك فيه Internet Connectivity.

Disaster Recovery and Backup

في الـTraditional Data Center، كل الـServers ممكن تكون موجودة في مكان واحد.

في الـCloud تقدر توزّع الـServers في أكثر من مكان وأكثر من Region، وهنعرف معنى الـRegion بعد كده.

Automatic Updates and Maintenance

الـServers والـInfrastructure محتاجين Updates وMaintenance باستمرار.

في الـCloud، الـCloud Provider بيتولى إدارة وصيانة المكونات الموجودة على مستوى الـInfrastructure، زي:

- Switches.
- Firewalls.
- Routers.
- باقي مكونات الـInfrastructure.

High Availability

كل Service سيكون ليها SLA يحدد الـGuaranteed Uptime.

مثلاً، ممكن تكون الخدمة مضمونة بنسبة:

• 99.99%.

• 99%.

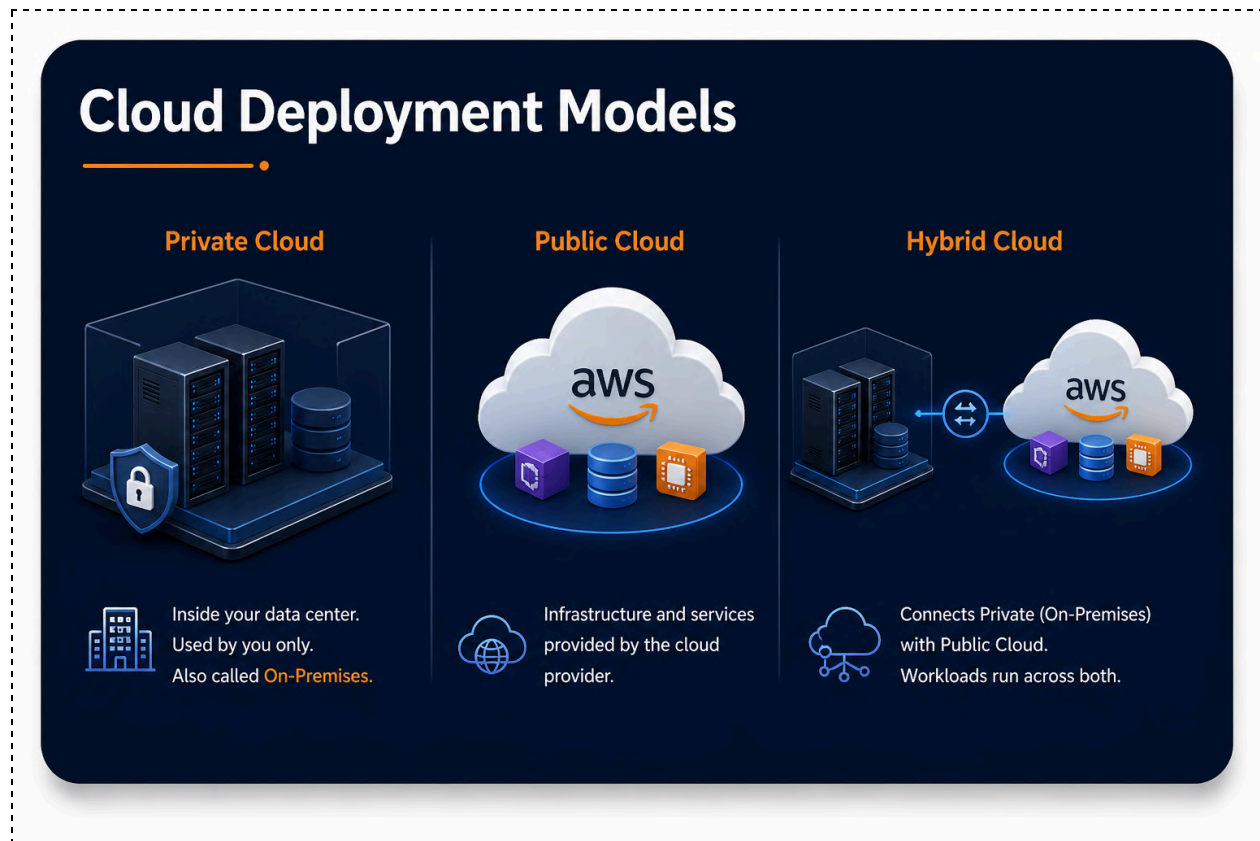
ده ببساعد إن الـService تفضل Highly Available بنسبة كبيرة.

Cloud Providers

من أشهر الـ Cloud Providers:

- AWS من Amazon.
- GCP من Google.
- Azure من Microsoft.

Cloud Deployment Models



Private Cloud

الجزء الموجود داخل الـ Data Center يمكن يتقال عليه Private Cloud.

الـ Private Cloud يتكون موجودة داخل الـ Data Center الخاص ببيك، ومفيش حد غيرك يقدر يستخدمها وبنقول عليها "On-Premises".

Public Cloud

الـPublic Cloud هي الخدمات والـInfrastructure التي يقدمها الـCloud Provider.

Hybrid Cloud

لما توصل الـPrivate Cloud أو الـOn-Premises Data Center بالـPublic Cloud، ده بيتسمى الـHybrid Cloud.

في الـHybrid Model، جزء من الـWorkload بيكون موجود في الـPrivate Cloud، وجزء تاني بيكون موجود في الـPublic Cloud.

Shared Responsibility Model

الـ Shared Responsibility معناها إن المسؤوليات بتتقسم بين:

- الـ Customer.
- الـ Cloud Provider.

جزء من المكونات بيكون مسؤولية الـ Cloud Provider، والجزء التاني بيكون مسؤولية الـ Customer.

On-Premises Responsibilities

في الـ On-Premises أو الـ Private Cloud، كل المكونات بتكون مسؤوليتك، ومنها:

- Storage.
 - Networking.
 - Switches.
 - Routers.
 - Servers.
 - Virtualization.
 - Operating System.
 - Applications.
 - Data.
-

Cloud Service Models

الـPublic Cloud يقدم 3 Cloud Service Models:



Infrastructure as a Service

في Infrastructure as a Service أو IaaS، الـCloud Provider يكون مسؤول عن:

- Storage.
- Networking.
- Compute.
- Virtualization Layer.

الـCloud Provider يوفر لك Virtual Machine، ومن أول ما تستلمها بتكون مسؤول عن:

- Operating System.
- Applications.
- Data.

أشهر مثال على IaaS هو EC2، وهي Virtual Machine أو Server بتأخذه داخل الـCloud.

Platform as a Service

في Platform as a Service أو PaaS، Cloud Provider سيكون مسؤول عن:

- Storage.
- Networking.
- Compute.
- Virtualization.
- Operating System.

مش بتحتاج تحدد Linux أو Windows، ومش بتتعامل مباشرة مع Virtual Machine أو Server.

مسؤوليتك بتكون:

- Application or Code.

• Data.

أشهر مثال على PaaS هو Lambda. في الـ Model ده بتحط الـ Code والـ Data فقط.

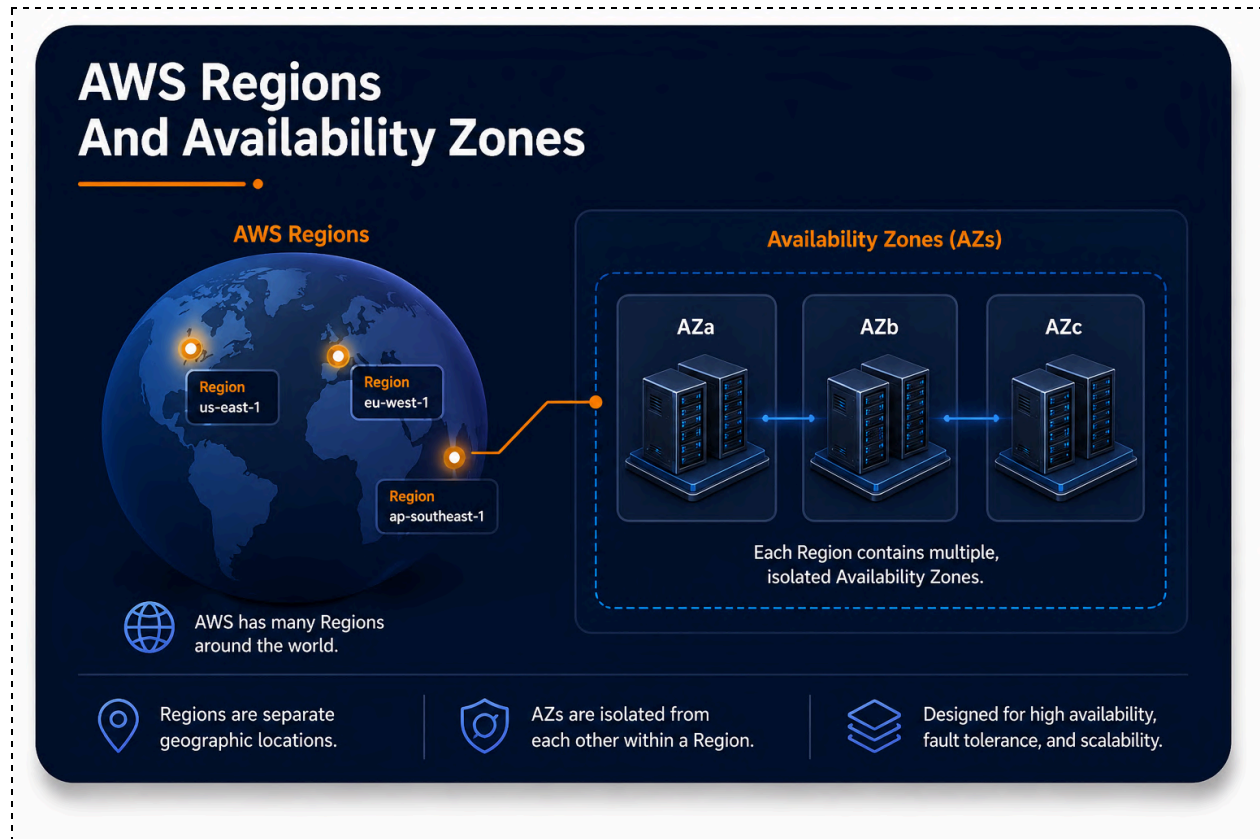
Software as a Service

في Software as a Service أو SaaS، Cloud Provider سيكون مسؤول عن كل المكونات، بما فيها الـ Application نفسه.

مسؤوليتك بتكون إنك تستخدم الـ Application وتحط الـ Data الخاصة بيك.

مثال على كده Office 365. بتفتح Application زي PowerPoint، والـ Application بيكون موجود على الـ Cloud، وإنك بتحط الـ Data الخاصة بيك وبتخزن على الـ Cloud.

AWS Regions



الـRegion هي مكان جغرافي يتواجد فيه AWS Services.

AWS عندها Regions مختلفة موجودة حوالين العالم، زي:

- London.
- Frankfurt.
- Paris.
- Regions inside United States

لما تدخل على AWS Console، أول حاجة بتعملها هي اختيار الـRegion، وبعد كده بتختار الـService اللي هتستخدمها داخل الـRegion.

كل Region منفصلة تمامًا عن باقي الـRegions. لو حصلت مشكلة في Region، ده ملوش علاقة بالـRegions الأتانية.

Region Selection Factors

اختيار الـ Region يعتمد على أكثر من عامل.

Latency

لو عندك Web Server والمستخدمين اللي هيعملوا Access عليه موجودين في أوروبا، مش منطقي تحطه في أمريكا أو آسيا.

الأفضل تحطه في أقرب Region للمستخدمين. مثلاً، لو المستخدمين موجودين في إنجلترا، ممكن تختار London Region.

Service Availability

مش كل AWS Services موجودة في كل الـ Regions.

ممكن تكون محتاج Service معينة، لكن تكون مش متاحة في الـ Region اللي اخترتها، وساعتها هتضطر تختار Region ثانية.

Cost

تكلفة نفس الـ Server ممكن تختلف من Region للثانية.

ممكن سعره في London يكون مختلف عن سعره في أمريكا، علشان كده الـ Cost عامل مهم في الاختيار.

Compliance

ممكن تكون مرتبط بقوانين معينة خاصة بمكان تخزين الـ Data.

مثلاً، لو عندك Sensitive Data وموجود في أوروبا، ممكن القوانين تفرض إن الـ Data متخرجش بره أوروبا، وساعتها لازم تختار Region داخل أوروبا.

Availability Zones

داخل كل Region سيكون فيه أكثر من Availability Zone.

الـ Availability Zone ستتكوّن من One or More Data Centers.

كل Availability Zone ستكون معزولة تمامًا عن باقي الـ Availability Zones.

لو عندك Server داخل Availability Zone وحصلت فيها مشكلة، ممكن Server ثاني موجود في Availability Zone مختلفة يشتغل بدل منه، وبالتالي الـ Traffic والـ Application ميتأثروش.

ممكن تحط الـ Application على أكثر من Server داخل أكثر من Availability Zone، علشان لو حصلت مشكلة في واحدة، يفضل الـ Application شغال في الـ Availability Zones التانية.

كل Availability Zone عندها:

- Independent Power.
- Independent Cooling.
- Independent Network.

كان موجود Region 37 و Availability Zone 117، بالإضافة إلى:

- 700 CloudFront Points of Presence.
 - 13 Regional Edge Caches.
-

Course Services Overview

خلال الكورس هنتعامل مع مجموعة كبيرة من AWS Services.

EC2 Instances

أول Service هي EC2 Instance، وهي Virtual Machine أو Server.

هنتعرف على أنواع الـ EC2 Instances، ومنها Instances بتكون:

- Compute Optimized وتركز أكثر على الـ CPU.
- Instances تركز أكثر على الـ RAM.

هنتعرف كمان على طرق الدفع، زي:

- On-Demand والدفع بالساعة.
- حجز الـ Instance لمدة سنة أو 3 سنين.

Storage Services

ممكن تربط الـ EC2 Instance بـ Volume زي:

- EBS.
- EFS.

هنتعرف كمان على Object Storage Service اسمها S3، وهي Service مهمة في AWS.

Permissions

هنتعرف على الـ Permissions كجزء من الـ Security.

الـ Permissions بتحدد:

- مين يقدر يعمل Access على S3.
- مين يقدر يعمل Access على EC2 Instance.
- إزاي EC2 Instance تقدر تعمل Access على S3.

Database Services

هنتعرف على أنواع مختلفة من Databases، ومنها:

- DynamoDB.
- RDS.

الـ EC2 Instance ممكن تكون مرتبطة بـ Database، وممكن تكون مرتبطة بـ EBS Volume، وممكن تعمل Access على S3 لتخزين Files.

Load Balancer



الـ Load Balancer بيوزع الـ Traffic على أكثر من EC2 Instance.

لو الـ Application موجود على أكثر من Virtual Machine، الـ Load Balancer هو اللي بيوزع الـ Traffic بينهم.

Monitoring Services

هنتعرف على CloudWatch، وهي Monitoring Tool بتتكوّن من:

- Logs.
- Alarms.

هنتعرف كمان على CloudTrail.

Additional Services

الكورس بيغطي Services تانية، منها:

- SNS.
- Lambda.
- WAF.
- KMS.
- ECS.
- ECR.

KMS بتستخدم لإدارة Encryption Keys، وبتساعدك تعمل Encrypt للـ Data داخل S3 أو أي Service تانية.

ECS و ECR ليهم علاقة بالـ Containers، وده جزء مهم في الـ DevOps.

AWS Account Setup

الكورس فيه Labs عملية، وعلشان تطبيقها محتاج تعمل AWS Account من خلال الموقع.

إنشاء Account محتاج Credit Card.

بعد ما الـ Account يبقى Active، تقدر تستخدم AWS Free Tier.

أغلب الـ Services المستخدمة في الكورس مجانية، لكن فيه Services بفلوس، زي WAF في الـ Lab.

AWS Free Tier

محتاج تراجع صفحة AWS Free Tier علشان تعرف:

- إيه الـ Services المجانية.
- إيه الـ Services اللي بفلوس.
- إيه الحدود المجانية لكل Service.

من الأمثلة المذكورة:

- 750 ساعة EC2 Instance من نوع T2 Micro أو T3 Micro.
- 5 GB على S3.
- Up to 20,000 GET Requests.
- RDS عندها Free Tier.
- SNS ممكن تكون Always Free.

بعض الخدمات بتكون مجانية لمدة 12 شهر فقط، وبعد انتهاء الـ 12 شهر بتكون بفلوس.

Resource Cleanup

في أي Lab بتعمل فيه Resources، لازم بعد ما تخلص تسمح الـ Resources على طول.

بعد ما تتأكد إن الـ Lab اشتغل، احذف الـ Resources علشان متدفعش تكلفة عليها.

قبل ما تطبق أي Lab، راجع هل الـ Service المستخدمة مجانية ولا لأ.

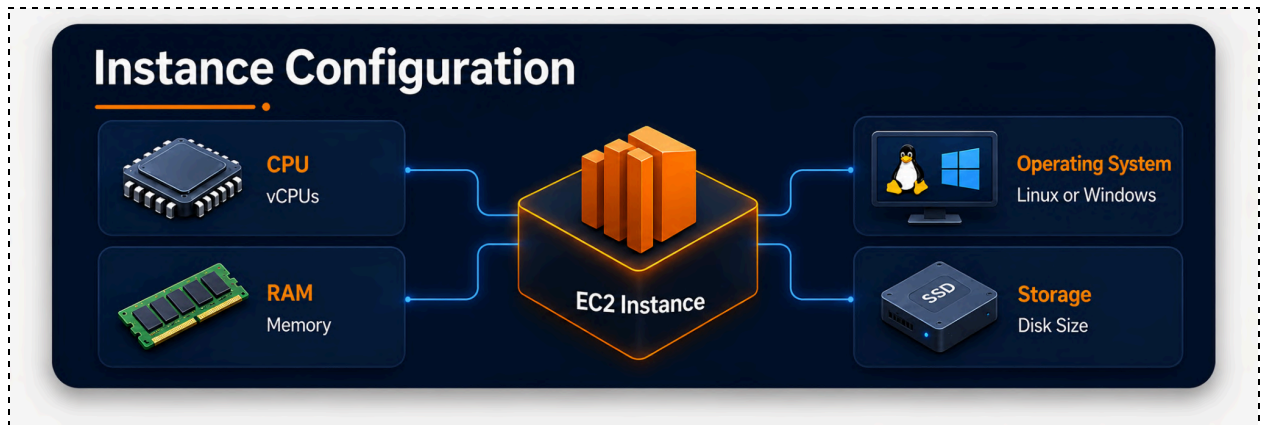
AWS Credits

ممکن تقدم للحصول على 300 دولار AWS Credits من خلال الموقع.
التقديم يحتاج يكون عندك Project معين ناوي تنفذه، وبتقدم طلب وتملأ البيانات المطلوبة.
قدمت على الـ 300 دولار قبل كده وقدرت أحصل عليهم.

Video 1 – EC2

EC2 Instances

Instance Configuration

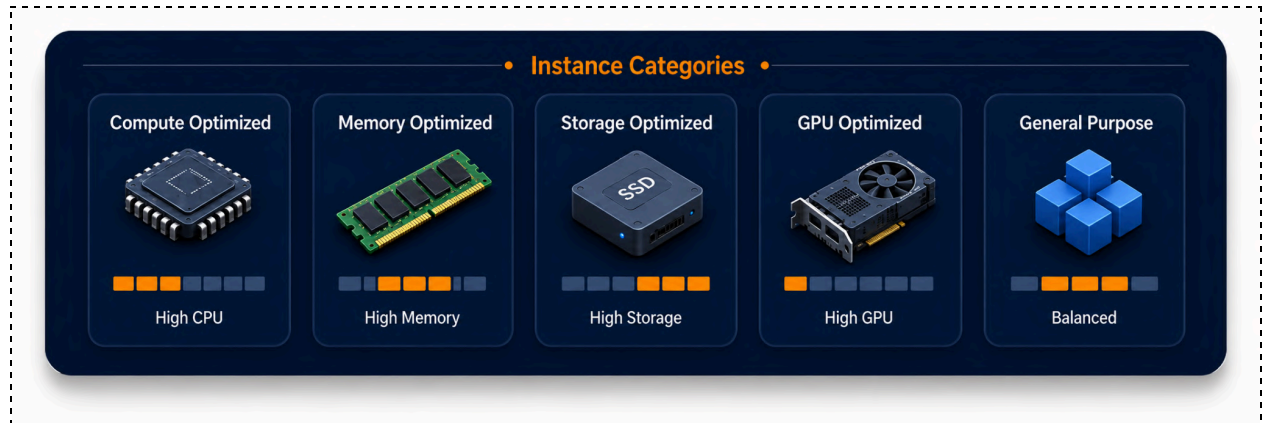


الـ EC2 Instance كأنك بتأجر Virtual Machine أو Server على Amazon. وقت ما بتيجي تختار الـ Instance، بتحدد أربع حاجات أساسية:

- حجم الـ CPU.
- حجم الـ RAM.
- الـ Operating System، سواء Linux أو Windows.
- حجم الـ Storage.

الـ EC2 Instances متقسمة لـ Categories مختلفة. في كل Category ممكن Resource معين يبقى أعلى من الباقي، وده بيحدد حسب الـ Job أو الـ Task اللي هتستخدم فيها الـ Instance.

Instance Categories



General Purpose

الـ General Purpose Instances بيبقى فيها توازن بين الـ CPU والـ Memory، من غير ما واحدة فيهم تبقى أعلى بشكل واضح من الثانية.

بتستخدمها في حالات زي:

- Web Server.
- Development Environment للتجارب والـ Testing.
- Small أو Medium Database.

من أمثلة الـ Instance Families الخاصة بالـ General Purpose:

- T3
- T4
- M5

Compute Optimized

في الـ Compute Optimized Instances، الـ Compute أو الـ CPU يبقى هو أعلى Resource في الـ Instance.

بتستخدمها في Tasks محتاجة Processing عالي، زي:

- Batch Processing
- Media Transcoding
- تحويل Audio أو Video من Format لـ Format تاني.

من أمثلتها:

- C5
- C6g

Memory Optimized

في الـ Memory Optimized Instances، الـ Memory يتبقى أعلى من الـ Compute أو الـ CPU.

الـ Ideal Use Case ليها بيكون مع High-Performance Databases، لأن بعض الـ Databases بتحتاج Memory و Memory Cache بحجم كبير.

من أمثلتها:

- R5
- R6
- X1
- X1e

Storage Optimized

في الـ Storage Optimized Instances، الـ Storage بيكون هو أعلى Resource في الـ Instance.

بتستخدمها مع:

- Relational Databases.
- NoSQL Databases.

Accelerated Computing

الـ Accelerated Computing Instances بتستخدم في Tasks زي:

- Machine Learning.
 - Complex Calculations.
-

EC2 Payment Options

الـ EC2 فيها أكثر من Payment Option، والجزء ده مهم جدًا في امتحان AWS.

On-Demand

في الـ On-Demand بتطلب الـ EC2 Instance وقت ما تحتاجها، وبتدفع بالثانية، من غير أي Commitment.

مثلاً، ممكن تشغل الـ Instance الساعة 12 الظهر، وبعد ما تخلص الـ Tasks المطلوبة الساعة 1 تقفلها، وتتحاسب على وقت التشغيل فقط.

الـ On-Demand مناسبة لما:

- تكون محتاج الـ Instance لمدة ساعة أو نص ساعة.
- ما تكونش عارف هتحتاج تشغلها لحد إمتى.
- تكون عايز تقفلها بمجرد ما تخلص الـ Task.
- يكون عندك Short-Term Workload.
- يحصل Spike مفاجئ في الـ Load.
- يكون عندك Unpredictable Workload.
- تحتاج تضيف Serverين أو ثلاثة بشكل مؤقت بسبب الضغط العالي.

Reserved Instances

في الـ Reserved Instances بتعمل Commitment لمدة سنة أو ثلاث سنين على Instance Configuration معينة، وفي المقابل بتحصل على Discount مقارنة بالـ On-Demand.

الـ Reserved Instances مناسبة للـ Applications اللي هتشتغل معاك Long Term، خصوصاً لما يكون عندك Steady و Predictable Workload.

وقت شراء الـ Reserved Instance بتحدد Instance Type، زي **t3.medium** أو **c5.large**، بالإضافة للـ Region وبعض الـ Attributes التانية.

ممكن تختار طريقة الدفع:

- **Partial Upfront**: تدفع جزء في البداية، والباقي بيتحاسب بسعر مخفض خلال مدة الـ Commitment.
- **All Upfront**: تدفع المبلغ كله في البداية، وده بيدي أعلى Discount بشكل عام.
- **No Upfront**: ما تدفعش حاجة في البداية، وتدفع بسعر مخفض خلال مدة الـ Commitment.

Spot Instances

الـ Spot Instance يستخدم الـ Spare EC2 Capacity الموجودة عند AWS بسعر أقل من الـ On-Demand، وسعرها ممكن يتغير مع الوقت.

الـ Spot Instances بتوفر Discount كبير مقارنة بالـ On-Demand، وممكن توصل لحد 90%.

في المقابل، AWS ممكن تعمل Interrupt للـ Spot Instance لما تحتاج الـ Capacity، وعادةً بتديك Interruption Notice قبلها بدقيقتين.

الـ Spot Instances مناسبة لما:

- تكون عايز تقلل التكلفة.
- يكون عندك Flexible Workload.
- يكون عندك Fault-Tolerant Workload.
- يكون الـ Application قادر يتعامل مع إن الـ Instance يحصلها Interruption.
- يكون عندك Batch Jobs أو Big Data أو Background Processing.

Savings Plans

في الـ Savings Plans بتعمل Commitment بكمية معينة من الـ Usage محسوبة بالدولار في الساعة لمدة سنة أو ثلاث سنين، وبتحصل على Rate أقل من الـ On-Demand Instances.

الـ Savings Plans بتديك Flexibility، والـ Flexibility المتاحة بتختلف حسب نوع الـ Savings Plan.

Compute Savings Plan

الـ Compute Savings Plan بيسمحك بتغير الـ Instance Family والـ Instance Size والـ Region والـ OS والـ Tenancy.

مثلاً، لو عندك c5، ممكن تغيرها بعد كده لـ m5.

الـ Instance Family بتكون مثلاً c5 أو m5 أو t3.

لو متوقع إنك تحتاج تغير الـ Instance Family أو الـ Region خلال مدة الالتزام، تختار Compute Savings Plan.

EC2 Instance Savings Plan

EC2 Instance Savings Plan يكون محدود بنفس الـ Instance Family والـ Region.

ما تقدرش تغيّر الـ Instance Family، لكن تقدر تغيّر الـ Instance Size داخل نفس الـ Family.

مثلاً:

c5.xlarge → c5.2xlarge (**Allowed**)

c5 → c6 (**Not Allowed**), c5 → m5 (**Not Allowed**)

وتقدر كمان تغيّر الـ OS والـ Tenancy مع الاستمرار في الاستفادة من الـ Savings Plan.

Dedicated Host

في الـ Dedicated Host بتأخذ Physical Server كامل مخصص ليك، وما بيكونش Shared مع AWS Customers ثانيين إلا لو أنت اخترت تشاركه مع AWS Account ثاني.

الـ Dedicated Host بيديك Visibility وControl أكبر على الـ Physical Server والـ Instance Placement.

الـ Dedicated Host مناسب لما تكون عايز:

- تتحكم في مكان تشغيل الـ Instances على الـ Physical Server.
- تستخدم (BYOL (Bring Your Own License).
- تستخدم Licenses مرتبطة بالـ Socket أو الـ Core أو الـ VM.
- تمنع مشاركة الـ Physical Server مع Customers ثانيين.
- تستخدم Server مخصص لأسباب مرتبطة بالـ Compliance.

Dedicated Instances

في الـ Dedicated Instances بتأخذ EC2 Instance شغالة على Hardware مخصص للـ AWS Account بتاعك.

لكن ما بيكونش عندك نفس الـ Visibility والـ Control على الـ Physical Server والـ Instance Placement الموجودين في الـ Dedicated Host.

الـ Dedicated Instances عندها Limited Support للـ BYOL، بينما الـ Dedicated Host بيوفر Comprehensive Support للـ BYOL.

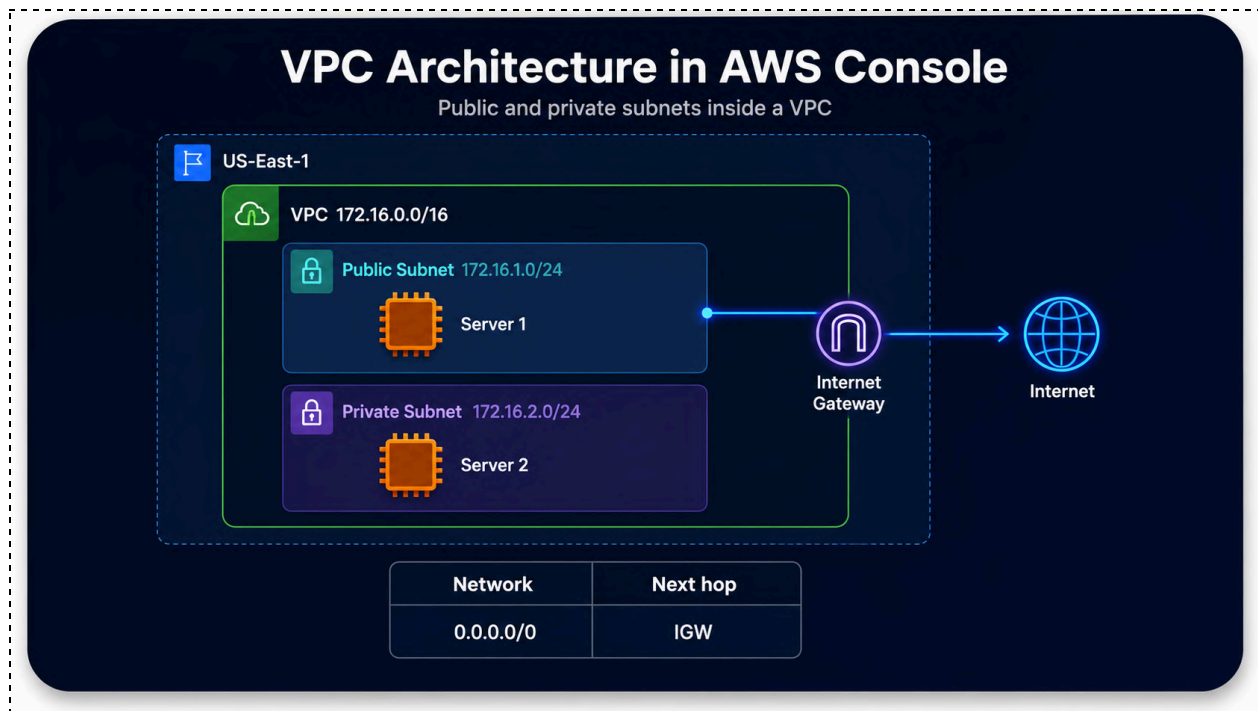
Video 2 – EC2 LAB

EC2 Networking Foundations

الـ EC2 Instance هي الـ Virtual Machine أو الـ Server اللي بتأجره من AWS. بعد ما تختار نوعها وطريقة الدفع، لازم تحدد المكان والـ Network اللي هتشتغل جواها.

الـ Networking Components المرتبطة بالـ EC2 Instance هنتعامل معاها هنا بشكل High Level، وبعد كده ممكن ندخل في تفاصيل أكثر عن الـ VPC وباقي مكوناتها.

Virtual Private Cloud



الـ VPC أو الـ Virtual Private Cloud هي الـ Network الخاصة بيك داخل AWS، وبتعتبر Logically Isolated Virtual Network.

الفكرة شبه الـ Network الموجودة عندك في الـ Data Center:

- بتحدد الـ Network Address.
- يتقسم الـ Network إلى Subnets.
- يتحط الـ Servers داخل الـ Subnets.
- بتحدد الـ Routing Tables.

الـ VPC يتحط داخل Region معين. في المثال اخترت Region اسمه **us-east-1**، وهو North Virginia. بعد اختيار الـ Region، بتحدد الـ CIDR Range الخاص بالـ VPC. في المثال استخدمت:

172.16.0.0/16

الـ VPC بتديك Control على الـ Network Environment، ومنها:

- تحديد الـ IP Ranges.
- إنشاء الـ Subnets.
- تعريف الـ Routing Tables.

Subnets

الـ Subnet هي جزء من الـ CIDR Range الخاص بالـ VPC.

في المثال، الـ VPC Range هو:

172.16.0.0/16

وتم تقسيمه إلى Subnets أصغر، كل واحدة منها واحدة جزء من الـ Range بحجم **24**.

تقدر تحط داخل الـ Subnet:

- EC2 Instances.
- أي AWS Resources ثانية محتاجة تكون جوه الـ Network.

كل Subnet بتكون موجودة داخل Availability Zone معينة. ممكن مثلاً تحط Subnet في Availability Zone أولى، و Subnet ثانية في Availability Zone مختلفة.

Availability Across Zones

يمكن تشغيل الـ Application على EC2 Instances موجودة في أكثر من Availability Zone. لو الـ Availability Zone الأولى وقعت، الـ Subnet والـ Server الموجودين فيها هيبقوا غير متاحين، لكن الـ Application يقدر يفضل شغال على الـ Server الموجود في الـ Availability Zone الثانية. في Mechanisms مختلفة لتشغيل الـ Application على أكثر من Server، ومن أهمها الـ Load Balancer.

Private IP Address

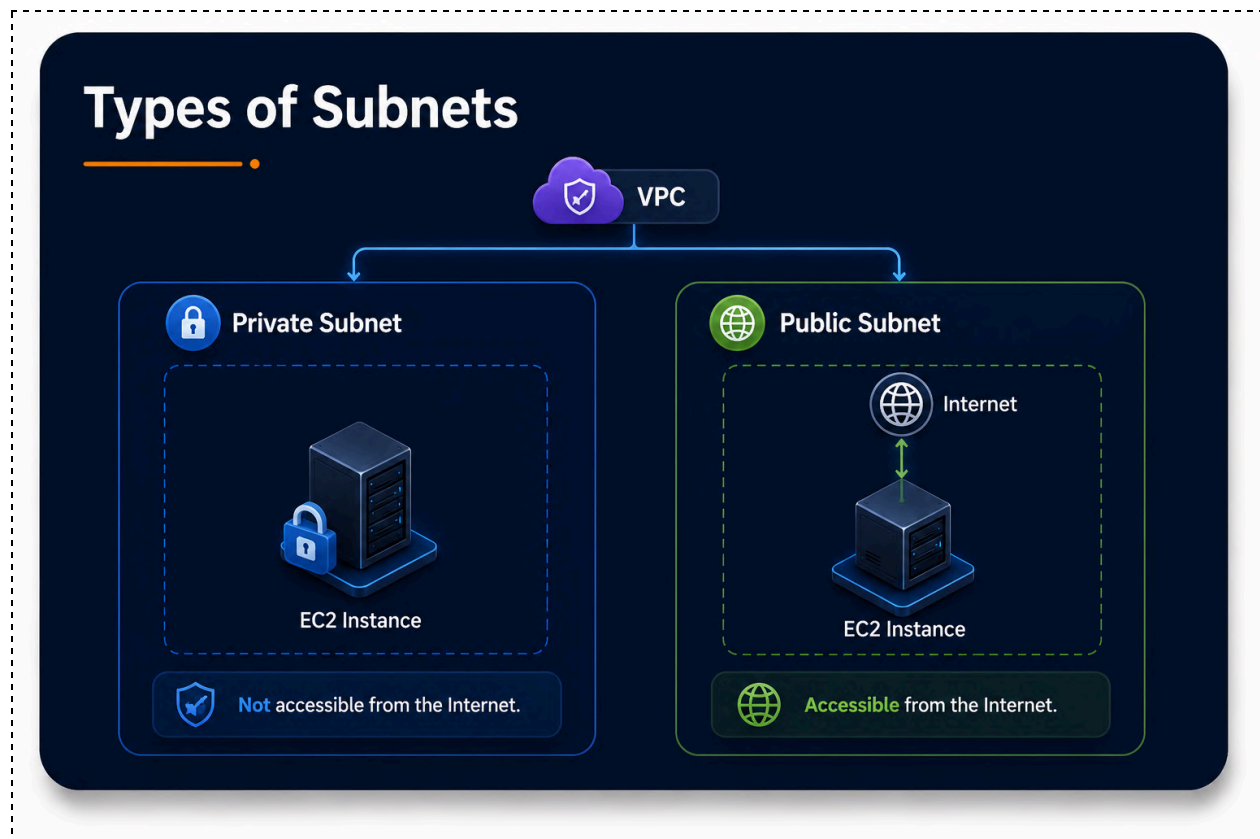
لما تحط EC2 Instance داخل Subnet، بتاخذ Private IP Address من الـ CIDR Range الخاص بالـ Subnet.

مثلاً، ممكن الـ Instance تاخذ عنوان زي:

172.16.1.3

الـ Private IP بيستخدم علشان الـ Instance تتواصل مع باقي الـ Resources الموجودة داخل الـ VPC.

Internet Connectivity



في نوعين من Subnets:

- Private Subnet.
- Public Subnet.

الـ Private Subnet ما بتوصلش للإنترنت مباشرة، ويمكن توصل من خلال NAT Gateway، لكن التفاصيل دي مش داخله في الجزء الحالي.

الـ Public Subnet بتستخدمها لما تكون محتاج الـ Resource يبقى Reachable من الإنترنت، زي Web Server الناس تقدر توصله.

Internet Gateway

علشان توصل الـ Public Subnet بالإنترنت، بتستخدم Internet Gateway.

الـ Internet Gateway هو Component يوصل الـ VPC والـ Resources الموجودة جواها بالإنترنت، وبيكون مهم للـ Public-Facing Resources.

Route Table

 Type	 Protocol	 Port range	 Source
HTTP	TCP	80	0.0.0.0
SSH	TCP	22	122.x.x.x

الربط بالـ Internet Gateway سيتم من خلال Route Table مرتبطة بالـ Subnet.

داخل الـ Route Table بتحت Default Route، بحيث أي Traffic رايج Destination مش معروف أو رايج الإنترنت يتم توجيهه من خلال الـ Internet Gateway.

بالتالي:

- الـ EC2 Instance بتطلع للإنترنت من خلال الـ Internet Gateway.
- المستخدمين الموجودين على الإنترنت يقدروا يوصلوا للـ Resources الموجودة في الـ Public Subnet.

Public IP Address

الـ EC2 Instance داخل الـ Public Subnet بتكون واخدة Private IP من الـ Subnet Range، ويمكن تاخذ كمان Public IP علشان تتواصل مع الإنترنت.

مثال على Public IP:

122.22.x.x

المستخدم الموجود على الإنترنت يقدر يوصل للـ Web Server باستخدام الـ Public IP.

بالتالي، الـ EC2 Instance ممكن يكون عندها:

- Private IP للتواصل داخل الـ VPC.
- Public IP للتواصل مع الإنترنت.

Network Security Controls

في نوعين أساسيين للتحكم في مين يقدر يوصل للـ Resources الموجودة داخل الـ VPC:

- Security Group.
- Network Access Control List (NACLs).

الـ Security Group هي الأكثر استخدامًا، بينما استخدامات الـ Network Access Control List بتكون Limited شوية.

Security Groups

الـ Security Group بتعتبر Stateful Firewall.

Stateful Behavior

كون الـ Security Group Stateful معناه إن Return Traffic الخاص بالـ Flow المسموح بـ يكون Allowed تلقائيًا.

مثلاً، لو عملت Rule تسمح بـ HTTP Traffic:

- Protocol: TCP.
- Port: 80.
- Source: Anywhere.

ده معناه إن أي مستخدم على الإنترنت يقدر يوصل للـ EC2 Instance باستخدام HTTP على Port 80.

الـ Return Traffic الخاص بنفس الاتصال بـ يكون Allowed تلقائيًا.

Instance-Level Protection

الـ Security Group بتتطبق على مستوى الـ Instance.

بتعمل Attach للـ Security Group على الـ EC2 Instance، وبتحدد من خلالها الـ Traffic المسموح يوصل للـ Instance أو يخرج منها.

Allow Rules

الـ Security Group بتدعم Allow Rules فقط.

بتحدد الـ Traffic اللي عايز تسمح بيه، زي:

- HTTP.
- SSH من IP معين.
- SSH من Subnet معينة.

أي Traffic مش موجود له Allow Rule بـ يكون Denied.

Inbound and Outbound Rules

كل Security Group عندها:

- Inbound Rules للـ Traffic اللي داخل على الـ EC2 Instance.
- Outbound Rules للـ Traffic اللي خارج من الـ EC2 Instance.

بشكل افتراضي:

- الـ Inbound Traffic بيكون Denied.
- الـ Outbound Traffic بيكون Allowed.

علشان تسمح بالوصول، بتضيف Inbound Rules زي:

- Port 80 للـ HTTP.
 - Port 22 للـ SSH.
-

Network Access Control Lists (NACLs)

الـ Network Access Control List أو الـ NACL بتتطبق على مستوى الـ Subnet، مش على مستوى الـ EC2 Instance.

من خلالها بتحدد:

- الـ IP Addresses اللي تقدر تتواصل مع الـ Subnet.
- الـ Destinations اللي الـ Subnet تقدر تتواصل معاها.

Stateless Behavior

الـ NACL بتعتبر Stateless Firewall.

ده معناه إن الـ Inbound والـ Outbound بيتعاملوا بشكل منفصل. لو سمحت بـ SSH في الـ Inbound لكن عملت Deny للـ Traffic في الـ Outbound، الاتصال مش هيشغل.

الـ Reply Traffic مش بيكون Allowed تلقائيًا، ولازم تعرّف الـ Inbound والـ Outbound Rules بشكل منفصل.

Rule Priority

الـ NACL بتحتوي على مجموعة Rules، وكل Rule ليها رقم.

كل ما رقم الـ Rule يكون أقل، الـ Priority بتاعتها بتكون أعلى.

AWS بتعمل Rules Processing من أقل رقم لأعلى رقم. أول Rule تعمل Match مع الـ Traffic بيتم تنفيذها، وبعد كده باقي الـ Rules ما بيتمش فحصها.

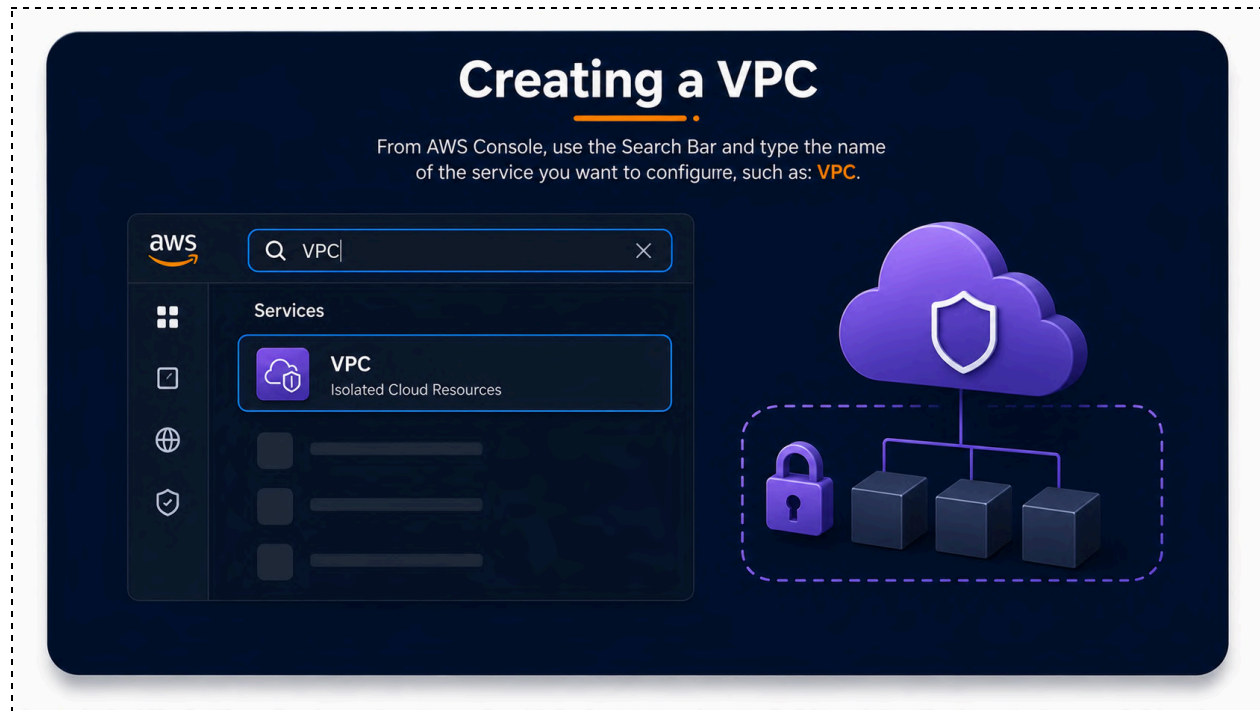
مثلاً، لو عندك:

- Rule 100 بتعمل Deny لأي Traffic.
- Rule 200 بتعمل Allow للـ Traffic الجاي من **10.0.0.0/16**.

الـ Traffic الجاي من **10.0.0.0/16** مش هيعدي، لأن Rule 100 اتنفذت الأول وعملت Deny.

علشان تسمح للـ Range ده، لازم تخلي Allow Rule برقم أقل من Deny Rule.

Creating a VPC



من AWS Console تقدر تستخدم الـ Search Bar وتكتب اسم الـ Service اللي عايز تعمل لها Configuration، زي:

- VPC.
- EC2.

أهم خطوة قبل إنشاء الـ VPC هي اختيار الـ Region الصح. في المثال اخترت North Virginia.

بعد كده:

1. افتح VPC Service.
2. اختار Create VPC.
3. اختار إنشاء VPC ومعها باقي الـ Components.
4. حدد اسم أو Tag للـ Resources.
5. حدد الـ CIDR Range.
6. حدد عدد الـ Availability Zones والـ Subnets.
7. أنشئ الـ VPC.

VPC Configuration

في المثال استخدمت اسم:

CloudCode

وده خلى أسماء الـ Resources تبقى بالشكل ده:

- CloudCode VPC.
- CloudCode Subnet.
- CloudCode Route Table.

الـ CIDR Range المستخدم كان:

172.16.0.0/16

اخترت:

- Availability Zone واحدة.
- Public Subnet واحدة.
- صفر Private Subnets.

بعد إنشاء الـ AWS، VPC أنشأت معها:

- Subnet.
- Internet Gateway.
- Route Table.
- باقي الـ Components المطلوبة.

Default Network ACL

الـ NACL المرتبطة بالـ Subnet كانت بشكل افتراضي بتسمح بأي Traffic.

كان فيها Rule 100 بتسمح بالـ Traffic من أي مكان في:

- Inbound.
- Outbound.

Route Table Configuration

الـ Route Table الخاصة بالـ Subnet كانت تحتوي على Default Route بتوجّه الـ Traffic إلى الـ Internet Gateway.

Launching an EC2 Instance

بعد إنشاء الـ VPC والـ Subnet، الخطوة التالية هي إنشاء EC2 Instance ووضعها داخل الـ Subnet.

Instance Name

سميت الـ EC2 Instance:

web-server

الهدف منها إنها تشتغل كـ Web Server.

Operating System

اخترت Amazon Linux كـ Operating System. ممكن تختار أنظمة ثانية، زي:

- Ubuntu.
- Windows.
- Red Hat.

Instance Type

اخترت:

t3.micro

الـ t3.micro كانت ظاهرة كـ Free Tier Eligible.

في أول سنة من الـ Free Tier، متاح 750 ساعة في الشهر على:

- t2.micro.
- t3.micro.

ممكن تختار Instance Type مختلف. مثلاً، الـ c5.large من C5 Family كانت تحتوي على:

- 2 Virtual CPUs.
- 4 GiB Memory.

تكلفة التشغيل بالساعة بتختلف حسب استخدام Windows أو Linux.

Key Pairs

الـ Key Pair بتستخدمها علشان تعمل Access على الـ EC2 Instance.

بتتكون من:

- Public Key.
- Private Key.

الـ Public Key بيكون موجود على الـ Machine، والـ Private Key بتنزلها عندك وتحطها على الـ Laptop أو الـ Linux Machine اللي هتتصل منها.

وقت إنشاء Key Pair جديدة، بتختار Format مناسب:

- **pem.** عند استخدام OpenSSH.
- **ppk.** عند استخدام PuTTY على Windows.

في المثال استخدمت Key Pair تم إنشاؤها قبل كده، وكان الاتصال من Mac Machine، فتم استخدام OpenSSH.

EC2 Network Settings

داخل Network Settings بتحدد المكان اللي هتتخط فيه الـ EC2 Instance.

VPC Selection

ممکن يكون عندك أكثر من VPC، لذلك اخترت الـ VPC اللي تم إنشاؤها.

Subnet Selection

ممکن يكون داخل نفس الـ VPC أكثر من Subnet، لذلك اخترت الـ Public Subnet المطلوبة.

Public IP Assignment

فعلت Public IP لأن الـ Server مطلوب يكون Public Facing و Reachable من الإنترنت.

الـ EC2 Instance بتأخذ بشكل افتراضي Private IP من الـ Range الـ Subnet الموجودة فيها.

في المثال، الـ Subnet Range كان:

172.16.0.0/20

الـ Instance بتأخذ Private IP من الـ Range ده، وبتأخذ كمان Public IP علشان:

- تقدر توصل للإنترنت.
- المستخدمين يقدرُوا يوصلوا لها من الإنترنت.

Web Security Group

أنشأت Security Group باسم:

web

وضفت فيها Inbound Rules للـ SSH والـ HTTP.

SSH Rule

سمحت بـ SSH من Anywhere علشان نقدر نعمل SSH على الـ Instance.

ممکن بدل Anywhere تحدد:

- IP Address معين.
- Subnet معينة.

HTTP Rule

سمحت بـ HTTP من Anywhere على Port 80، علشان أي مستخدم يقدر يوصل للـ Web Server.

الـ Security Group كانت تحتوي على:

- SSH على Port 22 من Anywhere.
- HTTP على Port 80 من Anywhere.

ممکن تضيف Description لكل Rule، لكن ده مش شرط.

User Data

داخل Advanced Settings فيه اختيار اسمه User Data.

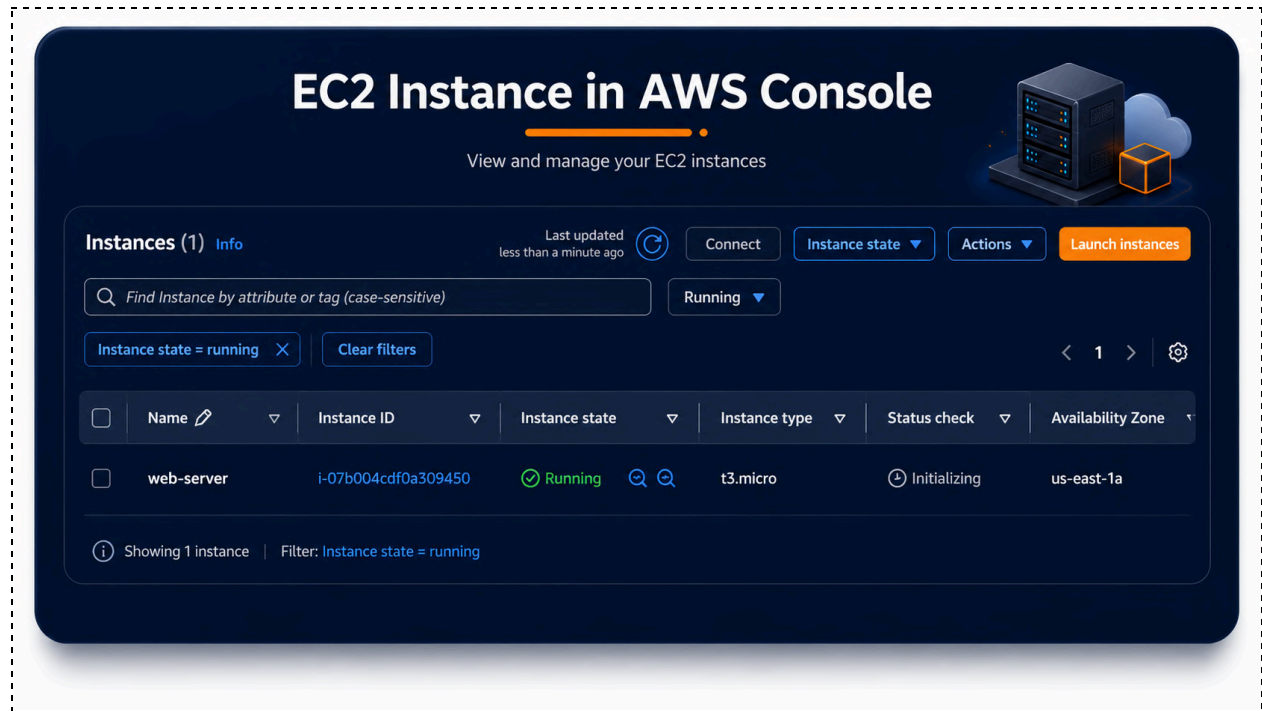
الـ User Data عبارة عن Script تقدر تصيفه وقت إنشاء الـ Instance، والـ Machine تنفذه أثناء التشغيل.

في المثال، الـ Script كان بيعمل:

- تثبيت Apache.
- إنشاء HTML Page.
- إضافة رسالة داخل الصفحة.

الهدف كان تشغيل Web Server تلقائيًا على الـ EC2 Instance.

Launching and Viewing the Instance



بعد مراجعة الإعدادات، عملت Launch للـ EC2 Instance.

داخل صفحة الـ Instances تقدر تعرض الـ Instances حسب حالتها، زي:

- Running.
- Stopped.
- Terminated.

الـ Instance ظهرت باسم **web-server** وكانت في حالة Running. داخل تفاصيل الـ Instance ظهر:

- Instance ID.
- Instance Type.
- Public IP.
- Private IP.
- VPC.
- Subnet.
- Security Group.

الـ Private IP يُستخدم للتواصل مع الـ Resources داخل الـ VPC، والـ Public IP يُستخدم للوصول من الإنترنت.

Security Rules Review

EC2 Security in AWS Console

[Details](#)[Status and alarms](#)[Monitoring](#)[Security](#)[Networking](#)[Storage](#)[Tags](#)

Security details

IAM Role
-

Security groups
sg-0f6723da572890760 (Web-SG) [↗](#)

Owner ID
[639233317097](#)

Launch time
Sat Jul 12 2025 15:06:04 GMT+0200

Inbound rules

Name	Security group rule ID	Port range	Protocol	Source	Security groups
-	sgr-0c5719de3132ce4e	80	TCP	0.0.0.0/0	Web-SG ↗
-	sgr-081e75d23c831df6a	22	TCP	0.0.0.0/0	Web-SG ↗

Outbound rules

Name	Security group rule ID	Port range	Protocol	Destination	Security groups
-	sgr-045517e93e370a713	All	All	0.0.0.0/0	Web-SG ↗

داخل Security Tab ظهرت Inbound Rules التي تم إعدادها:

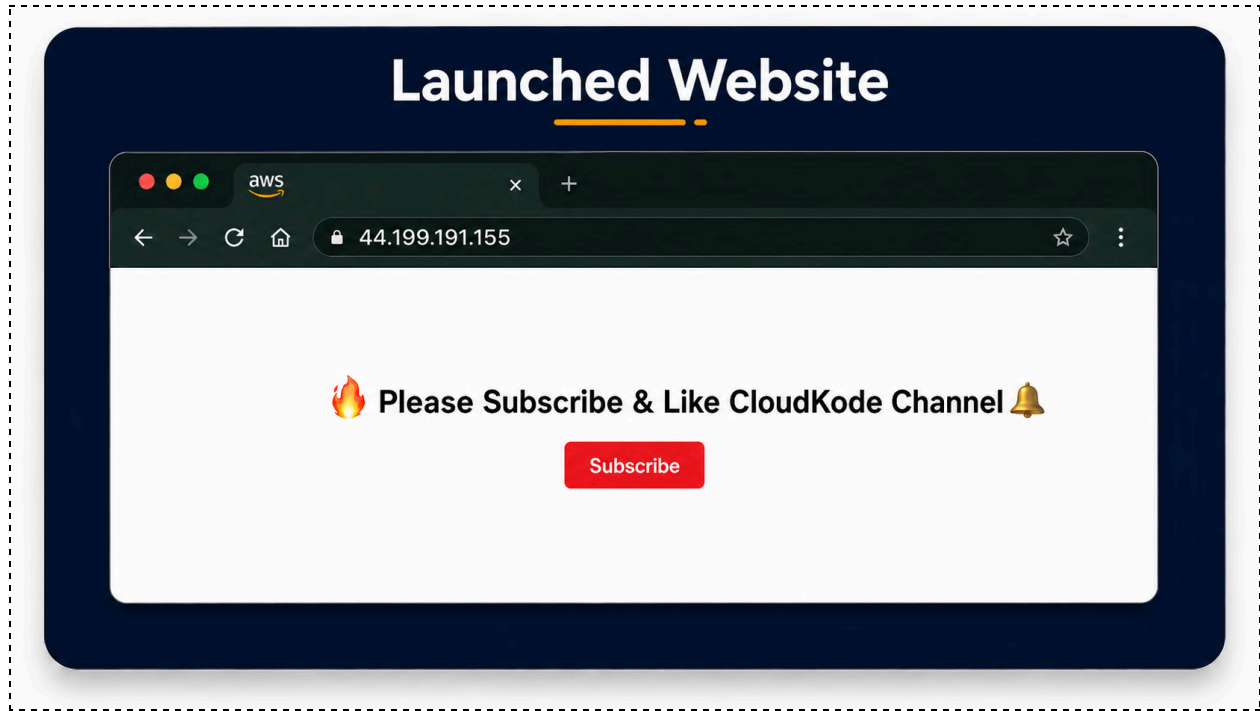
- Port 80 Allowed من أي مكان.
- Port 22 Allowed من أي مكان.

الـ Outbound Rules كانت بتسمح للـ Instance تتواصل مع أي Destination على الإنترنت بشكل افتراضي.

Testing the Web Server

لاختبار الـ Web Server، نسخت الـ Public IP وحطينه في الـ Browser باستخدام HTTP.

الصفحة اشتغلت وظهرت الرسالة:

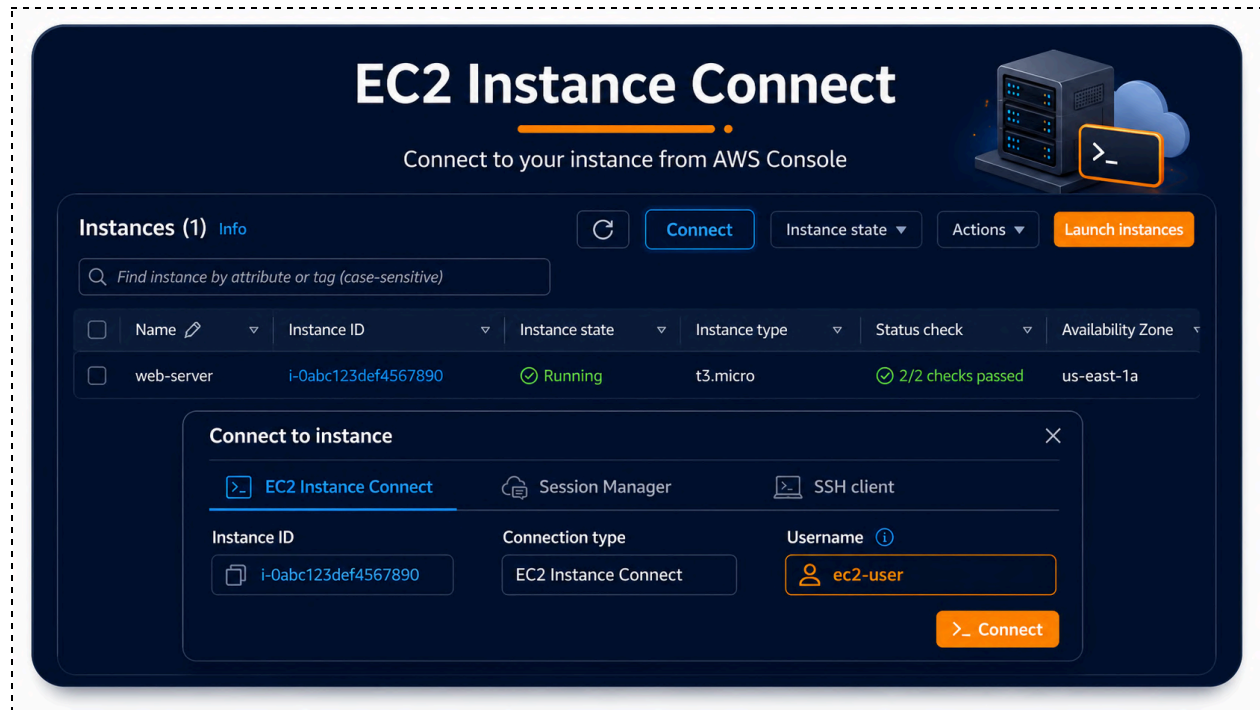


ده أكد إن الـ Web Page شغالة على HTTP وإن الوصول من الإنترنت للـ EC2 Instance شغال.

Connecting to the Instance

في أكثر من طريقة تعمل بينها Connect على EC2 Instance.

EC2 Instance Connect



من AWS Console تقدر تختار Connect وتستخدم EC2 Instance Connect.

الـ Default Username الخاص بالـ Amazon Linux Image المستخدمة كان:

ec2-user

بما إن الـ Security Group كانت بتسمح بالـ SSH من الإنترنت، تم الاتصال بالـ Machine من خلال الـ Browser.

SSH From macOS

SSH Terminal Commands

Prepare the PEM key and connect to the EC2 instance

1 Move the key file

```
Mo@CloudKode ~ % mv ~/Downloads/test-key.pem ~/.ssh/
```

2 Set secure permissions

```
Mo@CloudKode ~ % chmod 400 ~/.ssh/test-key.pem
```

3 Connect with SSH

```
Mo@CloudKode ~ % ssh -i ~/.ssh/test-key.pem ec2-user@44.199.191.155
```

علشان نتصل من Mac Laptop:

1. نزل ملف الـ Private Key.
2. انقل الـ Key من Downloads إلى الـ SSH Directory أو أي Directory تختاره.
3. اضبط الـ Permissions الخاصة بالـ Key.
4. استخدم SSH مع مسار الـ Key.
5. استخدم الـ Default Username الخاص بالـ Image.
6. استخدم الـ Public IP الخاص بالـ EC2 Instance.

شكل الاتصال بيكون معتمد على:

- مسار الـ Private Key.
- اسم المستخدم **ec2-user**.
- الـ Public IP الخاص بالـ Instance.

بعد تنفيذ الاتصال، قدرت أدخل على الـ Machine مباشرة من الـ Laptop.

SSH From Windows

على Windows تقدر تستخدم PuTTY للاتصال بالـ Instance.

لازم يكون معاك الـ Key Pair، وتحط الـ Key في إعدادات PuTTY، وبعد كده تتصل باستخدام الـ Public IP.

SSH Source Restrictions

تقدر تتصل بالـ Instance من أي مكان في العالم بشرط إن الـ IP الخاص بيك يكون Allowed في الـ Security Group.

السماح باستخدام:

```
0.0.0.0/0
```

معناه إن أي IP يقدر يحاول يعمل SSH، وده مش أفضل اختيار.

الأفضل تقدر تحدد IP معين، زي الـ Public IP الخاص بالبيت أو المكان اللي بتتصل منه، وتسمح له فقط بالوصول للـ Machine.

Video 3 – EC2 Storage types

Amazon Machine Images

الـ AMI هي نسخة Customized من الـ EC2 Instance. بدل ما كل مرة تعمل Launch لـ EC2 Instance وتدخل كل الـ Parameters من جديد، تقدر تجهز Instance مرة واحدة وتحفظها كـ Image.

ممکن مثلاً تحدد:

- الـ Operating System.
- Linux Version معينة.
- User Data فيها Web Script.
- Software Configuration معينة.
- أي Customization محتاجها.

بعد كده تحفظ الـ Instance كـ AMI. لما تحتاج تعمل Deployment لنفس الـ Application مرة ثانية، بتعمل Launch من الـ AMI مباشرة بدل ما تدخل كل الـ Configuration من الأول.

Benefits

الـ AMI بتوفر ميزتين أساسيتين:

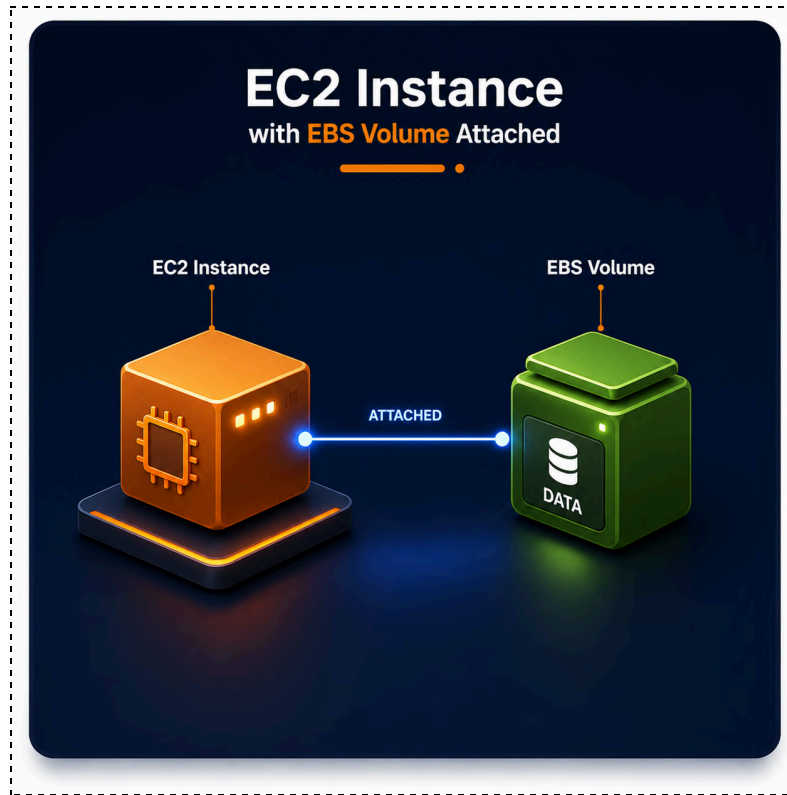
- **Speed:** بتسرّع الـ Deployment لأنك بتشغل الـ Instance من Image جاهزة.
- **Consistency:** كل الـ EC2 Instance بتقوم من نفس الـ AMI هتشغل بنفس الـ Configuration بالضبط، من غير ما يحصل اختلاف بسبب إدخال الـ Parameters يدويًا كل مرة.

AMI Types

فيه أكثر من نوع للـ AMI:

- **Public AMI:** بتكون متاحة لأي حد يقدر يستخدمها.
- **Private AMI:** بتعملها بنفسك وبتفضل خاصة بيك، ومش لازم تخليها Public.
- **AWS Marketplace AMI:** هي Image بيعملها شخص أو جهة بـ Specifications معينة وبيبيعها للناس.

EBS Volumes



الـ EBS Volume هو Network Drive بيتعمله Attach للـ EC2 Instance.

ممکن تختار حجم الـ Volume اللي محتاجه. مثلاً، لو اخترت Volume حجمه 10 GB وربطته بـ EC2 Instance، الـ Instance تقدر تحفظ الـ Data بتاعتها جواه.

Data Persistence

الـ EBS Volume يمكن يحتفظ بالـ Data حتى لو حصل Termination للـ EC2 Instance.

يمكن تمسح الـ Instance وتسبب الـ Volume موجود، وبعد كده تعمل الـ EC2 Instance Launch جديدة وتعمل Attach لنفس الـ Volume. الـ Instance الجديدة هتقدر تشوف الـ Data القديمة الموجودة عليه.

Instance Attachment

بشكل أساسي، الـ EBS Volume بيتعمله Mount على Instance واحدة في نفس الوقت.

يعني ما ينفعش تعمل Attach لنفس الـ EBS Volume على أكثر من EC2 Instance في نفس الوقت، إلا باستخدام Feature مخصصة هنتكلم عنها بعدين.

Availability Zone

الـ EBS Volume بيتعمل داخل Availability Zone محددة.

لو عملت EBS Volume في **us-east-1a**، لازم الـ EC2 Instance اللي هيتعمله Attach تكون موجودة في نفس الـ Availability Zone.

لو الـ Instance موجودة في **us-east-1b**، مش هتقدر تشوف أو تعمل Attach للـ Volume الموجود في **us-east-1a**.

Network USB Stick Analogy

يمكن نشبّه الـ EBS Volume بـ Network USB Stick.

زي ما بتوصل USB Stick أو External Hard Disk بالـ Laptop علشان تزود الـ Storage، بتعمل Attach للـ EBS Volume على الـ EC2 Instance علشان تزود مساحة التخزين المتاحة ليها.

Delete on Termination

لما بتعمل EC2 Instance Launch، بيتم إنشاء Root Volume معاها.

أثناء إنشاء الـ Instance، بتحدد:

- حجم الـ Root Volume، زي 8 GB أو 80 GB.
- نوع الـ Volume، زي General Purpose.

Root Volume Behavior

بشكل افتراضي، خيار **Delete on Termination** بيكون مفعل على الـ Root EBS Volume.

معنى كده إنك لما تعمل EC2 Instance Terminate:

- الـ Root Volume هيتمسح.
- كل الـ Data الموجودة عليه هتتمسح.

لو عايز الـ Root Volume يفضل موجود بعد حذف الـ Instance، لازم تعمل Uncheck لخيار **Delete on Termination**.

بعد كده تقدر تعمل EC2 Instance Launch ثانية وتعمل Attach للـ Root Volume القديم عليها.

Additional Volume Behavior

ممكن تضيف Volume إضافي غير الـ Root Volume مثلاً:

1. تعمل EC2 Instance Launch والـ Root Volume موجود معاها.
2. تعمل EBS Volume Create إضافي حجمه 20 GB في نفس الـ Availability Zone.
3. تعمل Attach للـ Volume الجديد على الـ Instance.

في الحالة دي، الـ Instance هيبقى عندها:

- Root Volume.
- Additional EBS Volume.

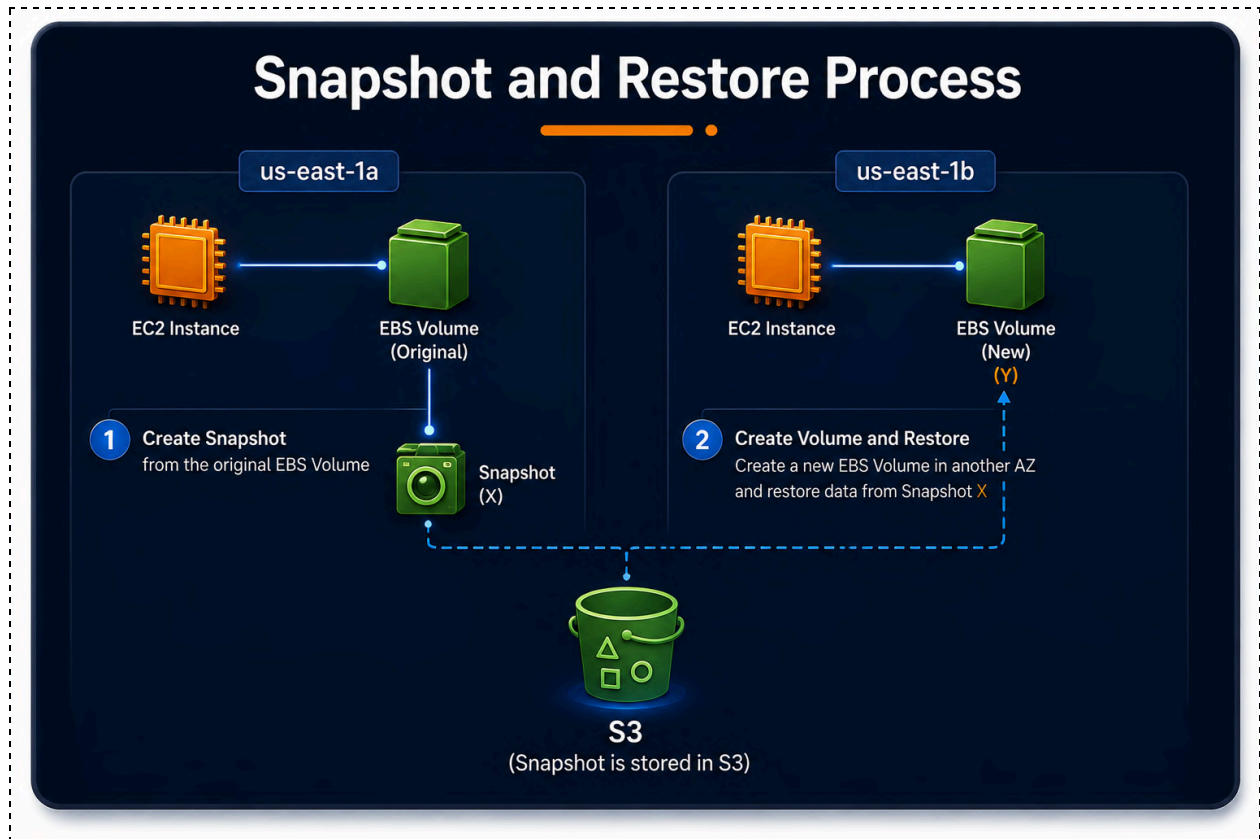
بشكل افتراضي، لما تعمل EC2 Instance Terminate، الـ Additional EBS Volume ما بيتمسحش.

EBS Snapshots

الـ EBS Snapshot يستخدم علشان تاخذ Backup من EBS Volume وتنقل الـ Data من Availability Zone إلى Zone أخرى.

الـ EBS Volume نفسها ما ينفعش تعمل Attach ليها على EC2 Instance في Availability Zone مختلفة، لذلك بنستخدم Snapshot.

Snapshot and Restore Process



نقل الـ Data بيتم على خطوتين:

1. تعمل Snapshot من الـ EBS Volume الأصلية.
2. تعمل Create الـ EBS Volume جديدة في الـ Availability Zone الثانية وتعمل Restore للـ Snapshot عليها.

مثلاً، لو عندك EBS Volume في **us-east-1a**:

1. تعمل Snapshot للـ Volume وتسميها **X**.
 2. تعمل Create Volume جديدة اسمها **Y** في **us-east-1b**.
 3. أثناء إنشاء الـ Volume الجديدة، تختار Restore للـ Data الموجودة في **X** Snapshot.
- كده كل الـ Data اللي كانت في الـ EBS Volume الموجودة في **us-east-1a** هتنتقل للـ EBS Volume الجديدة الموجودة في **us-east-1b**.
- الـ Snapshot بتكون محفوظة في **S3**.
-

EBS Volume Types

GP2 and GP3

الـ GP2 والـ GP3 هما General Purpose SSD Volumes.

يكونوا مناسبين لما تحتاج SSD Volume بـ Cost مناسب.

IO1 and IO2

الـ IO1 والـ IO2 هما SSD Volumes مخصصة للحالات التي تحتاج:

- High Performance.
- Low Latency.
- High Throughput.

ST1

الـ ST1 هو **Throughput Optimized HDD**. تكلفته أقل من GP و IO، وبيستخدم لما يكون عندك:

- Big Data.
- High Throughput.
- نقل Data كبيرة بين الـ Volume والـ EC2 Instance.
- Latency مش عامل أساسي.

الـ ST1 يوفر **High Throughput**، لكن لو محتاج Low Latency أو High IOPS تستخدم SSD زي GP أو IO بدل ST1.

SC1

الـ SC1 هو **Cold HDD**، وبيستخدم لما يكون عندك:

- Big Data.
- Latency مش مهمة.
- Data مش بتعمل Access عليها بشكل متكرر.

الـ SC1 مناسب للـ **Infrequently Accessed Data**، وبيعتبر أقل EBS Volume من حيث التكلفة.

EBS Multi-Attach



بشكل أساسي، الـ EBS Volume يتعمله Attach على EC2 Instance واحدة.

لو محتاج أكثر من EC2 Instance يشوفوا نفس الـ EBS Volume، لازم تفعّل خيار **Multi-Attach** أثناء إنشاء الـ Volume.

Requirements

علشان تستخدم EBS Multi-Attach:

- نوع الـ EBS Volume لازم يكون IO1 أو IO2.
- كل الـ EC2 Instances لازم تكون في نفس الـ Availability Zone.

في أسئلة الامتحان، لو المطلوب EBS Volume يتشارك بين أكثر من EC2 Instance، الاختيار بيكون IO1 أو IO2.

Instance Store

الـ Instance Store هو Storage موجود Local جوه الـ Hardware الخاص بالـ EC2 Instance. الـ EBS يكون Volume منفصل بيتعمله Attach للـ Instance، لكن الـ Instance Store يكون موجود جوه الـ EC2 Instance نفسها.

Performance

الـ Instance Store بيقدم High Performance لأنه موجود مباشرة مع الـ EC2 Instance.

Data Loss

المشكلة الأساسية إنك بتفقد الـ Data لو الـ EC2 Instance اتوقفت. لو عملت Stop للـ Instance، كل الـ Data الموجودة في الـ Instance Store هتتمسح.

Use Cases

الـ Instance Store مناسب للحالات اللي تحتاج:

- Fast Buffer
- Cache سريع
- Temporary Data

مش مناسب للـ Data اللي لازم تفضل محفوظة بعد توقف الـ Instance.

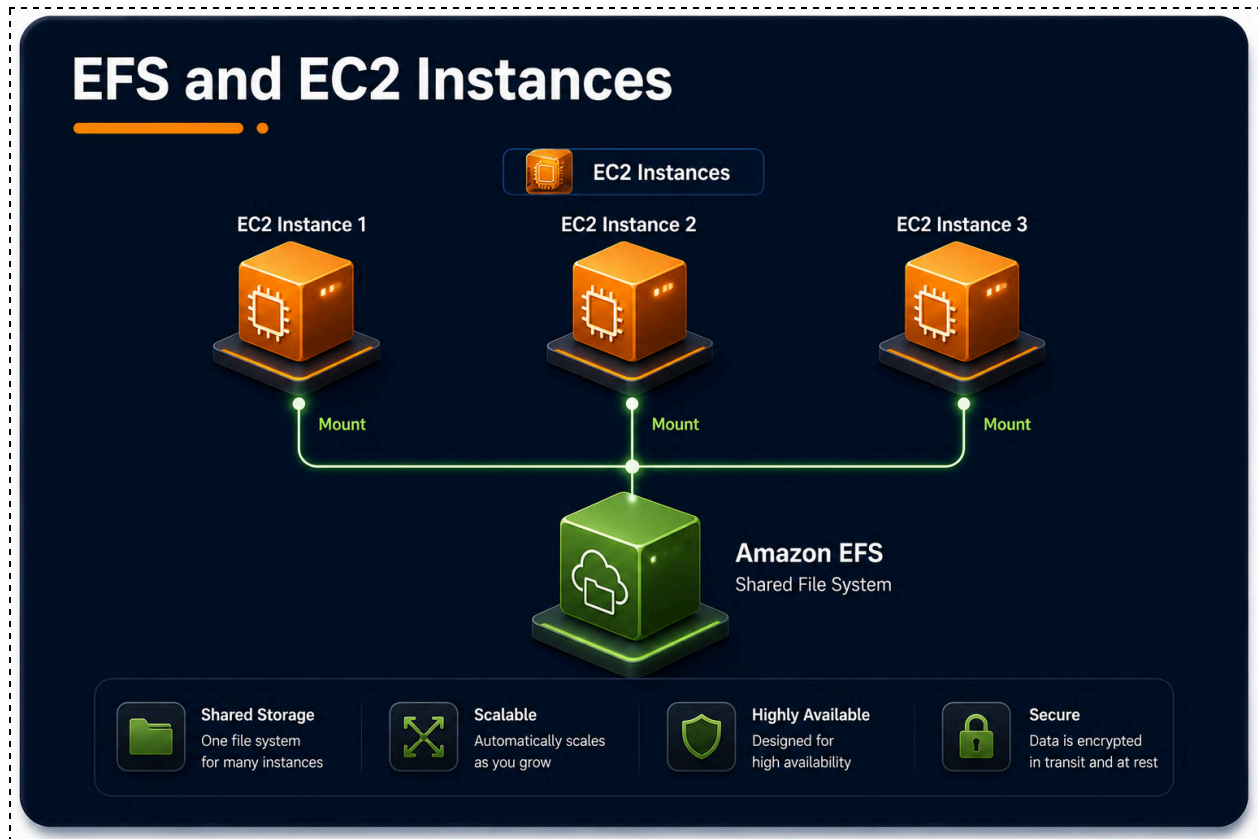
EBS and EFS Storage Models

الـ EBS بيعتبر Block Storage، بينما الـ EFS بيعتبر File Sharing.

في أسئلة الامتحان:

- أي حاجة مرتبطة بـ Block Storage بتنشيط للـ EBS.
 - أي حاجة مرتبطة بـ File Sharing بتنشيط للـ EFS.
-

Elastic File System



الـEFS، أو Elastic File System، هو Managed Network File System بتديره AWS.

يمكن أكثر من EC2 Instance تعمل Mount لنفس الـFile System، والـInstances دي ممكن تكون موجودة في Availability Zones مختلفة.

الميزة دي مش موجودة بنفس الشكل في EBS، لأن EBS لازم يكون في نفس الـAvailability Zone، والـSharing بين أكثر من Instance بيحتاج Multi-Attach.

Pricing and Compatibility

الـEFS:

- أعلى في التكلفة.
- بيتحاسب حسب الاستخدام.
- Compatible مع Linux فقط.
- مش بيشتغل مع Windows EC2 Instances.

File Sharing

الـ EFS مناسب لما يكون عندك مجموعة Files عايز أكثر من EC2 Instance يشاركوا فيها. ممكن تعتبره Shared Folder، وكل الـ EC2 Instances تقدر تتعامل مع نفس الـ Files الموجودة عليه.

EFS Performance Modes

أثناء إعداد الـ EFS، بتختار Performance Mode.

General Purpose

الـ General Purpose يوفر أداء Balanced.

Max I/O

الـ Max I/O يستخدم لما تحتاج High Throughput، ويكون أعلى شوية.

EFS Throughput Modes

الـ Throughput هو سرعة نقل الـ Data، ومالوش علاقة مباشرة بحجم الـ Storage.

مثلاً، ممكن يكون عندك EFS حجمه 10 GB، لكن سرعة إرسال الـ Data تكون 1 GB per second.

أثناء إعداد الـ EFS، بتختار Throughput Mode من 3 اختيارات.

Bursting

الـ Bursting مناسب لما يحصل Traffic عالي بشكل مفاجئ على الـ EFS.

لو متوقع Bursts مفاجئة، زي إن كمية Data كبيرة تنتقل فجأة، تختار Bursting Mode.

Provisioned

الـ Provisioned يستخدم لما تكون عارف من البداية الـ Throughput المطلوب.

مثلاً، لو متوقع إن الـ EFS يحتاج سرعة ثابتة 1 GB، تختار Provisioned وتحدد الـ Throughput من البداية.

الـ Provisioned Throughput مستقل عن حجم الـ Storage.

Elastic

الـ Elastic يستخدم لما تكون مش عارف حجم الـ Workload أو سرعة الـ Download المطلوبة.

في الحالة دي، الـ EFS ييضبط الـ Throughput تلقائياً حسب الـ Workload بدل ما تحدد قيمة ثابتة.

EFS Storage Tiers

أثناء إعداد الـ EFS، يتحدد هل الـ Files ستتعمل Access عليها بشكل متكرر ولا لأ.

Standard

الـ Standard مناسب للـ Frequently Accessed Files.

يستخدم لما تكون محتاج الـ Files تبقى Available في أي وقت، ويكون أعلى.

Infrequent Access

الـ Infrequent Access مناسب للـ Files التي ستتعمل Access عليها كل فترة، زي مرة كل يوم أو كل كام يوم.

تكلفته أقل من Standard.

EFS Availability

أثناء إعداد الـ EFS، بتحدد مستوى الـ Availability.

Standard Availability

الـ Standard بيخلي الـ EFS Available في أكثر من Availability Zone. لو Availability Zone وقعت، الـ EFS يفضل شغال من الـ Availability Zones التانية. الاختيار ده:

- أعلى.
- مناسب للـ Production Environment.
- بيوفر Availability أعلى.

One Zone

الـ One Zone بيخلي الـ EFS موجود في Availability Zone واحدة. مناسب لحالات زي:

- Testing.
 - Development Environment.
 - Backup.
-

EBS and EFS Comparison

EFS	EBS	Feature
File Storage	Block Storage	Storage Model
Shared File Storage File Workloads و	Block-Level و Databases Storage	Main Use Case
Multi-Gigabit يدعم Throughput و ييقدر يعمل Performance لل Scale	ييعتمد على Volume Type، و IO1 و IO2 ييقدموا أعلى Performance و Throughput و	Performance
Scale و Scale Up يعمل يقدر يعمل Down تلقائيًا	تغيير الحجم محتاج Manual Intervention	Scalability
يمكن يكون Available في أكثر من Availability Zone	يكون موجود في Availability Zone واحدة	Availability
يمكن يتعمله Mount على أكثر من EC2 Instance في Availability Zones مختلفة	ييتعمله Attach على Instance واحدة، إلا مع Multi-Attach	Access
يتدفع حسب كمية ال Storage المستخدمة	يتدفع مقابل ال Provisioned Storage اللي حددته من البداية	Pricing

EBS Pricing

في EBS، يتحدد حجم الـ Storage من البداية.

مثلاً، لو اخترت EBS Volume بحجمه 80 GB، بتدفع مقابل الـ 80 GB اللي عملت Provision ليها، حتى لو ما استخدمتش المساحة كلها.

EFS Pricing

في EFS، بتدفع حسب الـ Storage المستخدم فعلياً.

لو أنشأت EFS وما حطيتش عليه Files، مش هتدفع مقابل Storage غير مستخدم.

Video 4 – S3

Amazon S3

Amazon S3 واحدة من أهم Services التي بتقدمها AWS، وهي Storage Service تقدر تستخدمها لتخزين كمية كبيرة جدًا من البيانات من غير ما يكون عندك مشكلة في Scaling أو Limit واضح للStorage.

Main Use Cases

يمكن استخدام Amazon S3 في أكثر من حالة:

- تخزين Backups.
- عمل Archive للFiles.
- تخزين ملفات Media Applications.
- استضافة Static Website.

Buckets and Objects

الFiles التي برفعها على S3 بتسمى Objects، وعشان كده S3 بتتصنف كObject Storage.

قبل ما أرفع أي Object، لازم أعمل Bucket. الBucket هي الTop Directory التي بتحفظ جواها الObjects الخاصة بيا.

ترتيب التخزين بيكون بالشكل ده:

- Bucket.
- Folders عند الحاجة.
- Objects، وهي الملفات نفسها.

Bucket Naming Rules

اسم الBucket لازم يكون Unique على مستوى AWS كلها، وده معناه إن الاسم ماينفعش يكون مستخدم في أي Account ثاني أو أي Region ثانية.

لما أختار اسم للBucket، لازم أراعي الآتي:

- ماستخدمش Uppercase Letters.
- ماستخدمش Underscore.

- أختار اسم Unique مش مستخدم قبل كده.

مثال على اسم الـ Bucket:

```
S3-CloudKode
```

Global Service and Bucket Region

Amazon S3 بتظهر كـ Global Service، على عكس Services زي EC2 اللي لازم أختار الـ Region قبل ما أبدأ أتعامل معاها.

لكن لما أعمل Create للـ Bucket نفسها، لازم أحدد الـ Region اللي الـ Bucket هتتخزن فيها.

يعني ممكن أفتح S3 كـ Global Service، وبعد كده أعمل Bucket وأحدد إنها تكون موجودة في Frankfurt Region.

Object Paths

كل Object داخل S3 بيكون ليه Full Path أو Key.

مثال:

```
s3://cloudkode-bucket/fileXYZ.txt
```

الـ Path بيتكوّن من:

1. اسم الـ Bucket.
2. اسم الـ Folder لو الـ Object موجود داخل Folder.
3. اسم الـ Object أو الـ File.

مثال على Object داخل Folder:

```
s3://cloudkode-bucket/folder-x/fileXYZ.txt
```

Versioning Overview

يمكن عمل Enable للـ Versioning على الـ Bucket.

لو رفعت File وبعد كده عدلت عليه ورفعت Version جديدة بنفس الاسم، الـ Version القديمة بتفضل محفوظة، وأقدر أرجع لها أو أشوفها وقت الحاجة.

الـ Versioning لازم يتعمل لها Enable الأول قبل ما أستفيد منها.

Creating an S3 Bucket

Bucket Configuration

عشان عمل Bucket من AWS Console:

1. أكتب **S3** في الـ Search Bar.
2. أفتح Amazon S3.
3. أضغط على **Create bucket**.
4. أكتب اسم الـ Bucket.
5. أحدد الـ Region.

في التطبيق، اخترت Frankfurt Region، وسميت الـ Bucket:

S3-CloudKode

أثناء كتابة الاسم اتأكدت إن مفيش:

- Uppercase Letters.
- Underscore.

في إعدادات الـ Public Access، خليت الـ Public Access معمول له Block، بحيث إن أي حد من الإنترنت ما يقدرش يعمل Access للـ Objects.

في إعدادات الـ Encryption، اخترت الـ Default Encryption، بحيث إن أي Object يترفع على الـ Bucket يتعمل له Encryption من Amazon.

بعد كده عملت Create للـ Bucket، وظهرت الـ Bucket في Frankfurt Region.

Uploading an Object

بعد ما فتحت الـ Bucket، قدرت تعمل Folder أو أرفع الـ Files مباشرة داخل الـ Bucket.

رفعت File اسمه:

CloudKode Logo Dark

خطوات رفع الـ File:

1. أفتح الـ Bucket.
2. أضغط على **Upload**.
3. أختار الـ File.
4. أتأكد إن الـ Destination هي الـ Bucket المطلوبة.
5. أضغط على **Upload**.

بعد انتهاء الـ Upload، ظهر الـ File داخل الـ Bucket كـ Object.

Object URL and Access

لما أفتح الـ Object، بقدر أشوف معلومات عنه زي:

- اسم الـ Object.
- الـ Size.
- الـ URL الخاص بالـ Object.

الـ URL هو الرابط اللي ممكن أستخدمه للوصول للـ Object من الإنترنت.

لكن بما إن الـ Public Access كان معمول له Block، استخدام الـ URL مباشرة من الإنترنت بييرجع:

Access Denied

Presigned URL

لما أفتح الـ Object من داخل AWS Console عن طريق زر **Open**، بقدر أشوف الـ Object حتى لو الـ Public Access مقفول.

الرابط اللي بيتستخدم في الحالة دي بيكون فيه معلومات إضافية، وده اسمه Presigned URL.

بما إنني صاحب الـ Resource وعندي Access عليه، AWS بتسمحلي أفتح الـ Object باستخدام الـ Presigned URL من غير ما أخلي الـ Object Public لكل الناس على الإنترنت.

S3 Access Control

في طريقتين أقدر أتحكم بيهم في الـ Access على S3.

User-Based Access

الـ User-Based Access بيتستخدم لما أكون عارف مين الـ User أو الـ Resource اللي هيعمل Access داخل AWS.

أمثلة:

- IAM User.
- EC2 Instance.

في الحالة دي، بستخدم IAM Policy أو IAM Role، وبعمل لها Attach للـ User أو الـ EC2 Instance.

جوا الـ Policy بحدد:

- الـ S3 Bucket اللي يقدر يعمل عليها Access.
- الـ Actions اللي يقدر ينفذها.
- هل يقدر يعمل Get.
- هل يقدر يعمل Put.
- أي Permissions تانية مسموحة له.

Resource-Based Access

الـ Resource-Based Access بيتستخدم لما أحتاج أتحكم في الـ Access من ناحية الـ S3 Bucket نفسها.

ده مناسب لما يكون عندي Users من خارج AWS Account أو ناس جايين من الإنترنت ومش عندي كنترول مباشر عليهم.

في الحالة دي، بستخدم:

S3 Bucket Policy

بدل ما أحط الـ Policy على الـ User، بحطها على الـ Bucket نفسها.

Bucket Policy Structure

الـ Bucket Policy بتكتب بصيغة JSON، وبتحتوي على معلومات أساسية.

Resource

الـ Resource بيحدد الـ Bucket أو الـ Objects اللي الـ Policy هتطبق عليها.

مثال:

```
arn:aws:s3:::s3-cloudkode/*
```

الـ /* معناها إن الـ Policy تنطبق على كل الـ Objects اللي موجودة داخل الـ Bucket.

Effect

الـ Effect بيحدد هل الـ Action مسموح ولا مرفوض.

مثال:

```
Allow
```

Action

الـ Action بيحدد العملية اللي مسموح تنفيذها.

أمثلة:

- `GetObject`
- `PutObject`

لو استخدمت `GetObject`، يبقى الـ User يقدر يقرأ أو يفتح الـ Object.

Principal

الـPrincipal يحدد مين اللي مسموح له ينفذ الـAction.

لو استخدمت:

*

ده معناه إن أي حد يقدر يعمل Access.

Public Read Policy Example

عشان أخلي كل الناس تقدر تقرأ الـObjects داخل Bucket معينة، الـPolicy بتحدد:

- الـResource هو كل الـObjects داخل الـBucket.
- الـEffect هو **Allow**.
- الـAction هو **GetObject**.
- الـPrincipal هو *****.

بالشكل ده، أي حد يقدر يعمل Read للـObjects، لكن ده مش معناه إنه يقدر يعمل Delete أو أي Action تانية مش موجودة في الـPolicy.

الـBucket Policy ممكن تستخدم كمان لفرض شروط معينة، زي إن أي Object يترفع على الـBucket لازم يتعمل له Encryption.

Applying a Bucket Policy

Disabling Block Public Access

لما استخدمت الـ Object URL بعد رفع الـ Logo، ظهر:

Access Denied

عشان أخلي الـ Object متاح من الإنترنت:

1. فتحت إعدادات الـ Bucket.
2. فتحت إعدادات الـ Public Access.
3. عملت Edit.
4. قفلت **Block Public Access**.
5. أكدت التغيير.

بعد تعطيل Block Public Access، الـ Object فضل بيرجع **Access Denied**، لأن تعطيل الـ Block لوحده مش كفاية.

لازم أضيف Bucket Policy تسمح بالـ Access.

Using the Policy Generator

عشان أعمل الـ Bucket Policy من غير ما أكتب JSON يدويًا، استخدمت الـ Policy Generator. كل Resource داخل AWS بيكون ليه ARN، وهو اسم Unique يميز الـ Resource داخل AWS.

استخدمت الـ ARN الخاص بالـ Bucket أثناء إنشاء الـ Policy.

خطوات إنشاء الـ Policy:

1. أفتح **Bucket policy**.
2. أضغط على **Policy generator**.
3. أختار إن نوع الـ Policy هو **S3 Bucket Policy**.
4. أحط * في الـ Principal عشان أسمح لأي حد.
5. أختار الـ Action:

GetObject

6. أنسخ الـARN الخاص بالـBucket.
7. أضيف بعد الـARN:

/*

8. أضغط على **Add Statement**.
9. أعمل Generate للـPolicy.
10. أنسخ الـJSON الناتج.
11. أرجع للـBucket Policy.
12. أعمل Paste للـPolicy.
13. أضغط على **Save**.

استخدام **/*** بعد الـARN الخاص بالـBucket معناه إن الـPolicy هتسمح بالـAccess على كل الـObjects الموجودة داخل الـBucket، بدل ما أحدد File واحد فقط.

بعد حفظ الـPolicy وعمل Refresh للـObject URL، ظهرت الصورة من الإنترنت.

Access Allowed

Static Website Hosting

Uploading Website Files



عشان أعمل Static Website على S3، رفعت File بين داخل الـ Bucket:

- ملف الـ Logo.
- ملف `index.html`.

ملف `index.html` كان عبارة عن HTML Page بتشاور على ملف الـ Logo الموجود داخل نفس الـ Bucket.

بعد رفع الـ File، قدرت أفتح `index.html` وأشوف الـ Logo اللي الصفحة بتشاور عليه.

Enabling Static Website Hosting

عشان أعمل Static Website:

1. فتحت الـ Bucket.
2. دخلت على `Properties`.

3. نزلت لحد Static website hosting.

4. ضغطت على Edit.

5. فعلت Static website hosting.

6. حددت الـ Index Document:

```
index.html
```

7. حفظت الإعدادات.

بعد التنفيع، ظهر Website Endpoint خاص بالـ Bucket.

لما فتحت الـ Endpoint، ظهرت الـ Static Web Page التي فيها المحتوى المكتوب داخل ملف الـ HTML والـ Logo الموجود داخل الـ Bucket.

الـ Static Website Hosting بيتم عمل له Enable على مستوى الـ Bucket.

S3 Versioning

Version History

لو عندي File اسمه:

XYZ.txt

ورفعته على S3، وبعد كده عدلت محتواه ورفعته مرة ثانية بنفس الاسم، تفعيل الـ Versioning بيخلي عندي:

- Version قديمة بالمحتوى الأول.
- Version جديدة بالمحتوى المعدل.

لما أفتح الـ Bucket بشكل طبيعي، بشوف أحدث Version فقط.

لكن من خلال **Show versions**، أقدر أشوف كل الـ Versions القديمة الخاصة بالـ Object.

الـ Versioning بيساعدني أعمل Rollback لأي Version قديمة وقت الحاجة.

Delete Marker

لو مسحت Object من Bucket عليها Versioning، الـ Object مش بيتحذف نهائياً مباشرة.

بدل كده، بيتعمل:

Delete Marker

الـ Delete Marker بيخلي الـ Object ما يظهرش في العرض العادي للـ Bucket، لكن أقدر أرجع للـ Versions وأستعيد الـ Object اللي اتمسح.

ده مفيد لو مسحت File بالغلط أو احتجت أرجعه بعد الحذف.

Stopping Versioning

لو وقفت الـ Versioning بعد ما كانت مفعلة، الـ Objects والـ Versions القديمة اللي اتعملت قبل إيقافها مش بتتمسح. الـ Files اللي اترفعت قبل تفعيل الـ Versioning بيكون الـ Version ID الخاص بيها:

Null

أي File يترفع بعد تفعيل الـ Versioning بياخد Version ID جديد.

Enabling Versioning

في التطبيق، كان عندي داخل الـ Bucket:

- ملف الـ Logo.
- ملف الـ `index.html`.

الـ Fileين كانوا مستخدمين في الـ Static Website.

عشان أفعل الـ Versioning:

1. فتحت الـ Bucket.
2. دخلت على `Properties`.
3. فتحت إعدادات `Bucket Versioning`.
4. ضغطت على `Edit`.
5. اخترت `Enable`.
6. حفظت الإعدادات.

بعد كده رجعت للـ Objects، وفعلت:

Show versions

Existing Object Version IDs

الـ Logو ملف `index.html` كانوا مرفوعين قبل تفعيل الـ Versioning، عشان كده ظهر الـ Version ID الخاص بيهم:

Null

من لحظة تفعيل الـ Versioning، أي Upload جديد لنفس الـ Files بياخد Version ID مختلف.

Uploading New Versions

عدلت ملف `index.html` ورفعت النسخة الجديدة بنفس الاسم.

لما قفلت `Show versions`، ظهر File واحد فقط باسم:

index.html

لكن لما فعلت `Show versions`، ظهر:

- الـ Version القديمة، وكان الـ Version ID الخاص بيها `null`.
- الـ Version الجديدة، وكان ليها Version ID جديد.

بعد كده عدلت ملف `index.html` مرة ثانية، وضفت داخله:

I Hope You Like CloudKode

ورفعت الملف بنفس الاسم، فتم إنشاء Version جديدة من `index.html`، وظهر المحتوى الجديد على الـ Static Website.

Restoring Deleted Objects

لو مسحت Object زي **CloudKode Logo** من الـ Bucket، هأكد عملية الحذف عن طريق كتابة **delete**.

بعد الحذف، الـ Object مش هيتظهر في العرض العادي للـ Bucket.

لما أفعل **Show versions**، هلاقي إن S3 عملت حاجة اسمها **Delete Marker** للـ Object.

عشان أرجع الـ Object:

1. أفعل **Show versions**.
2. أعدد الـ Delete Marker.
3. أعمل Delete للـ Marker.
4. أكتب **permanently delete** لتأكيد حذف الـ Marker.
5. أفعل **Show versions**.

بعد حذف الـ Delete Marker، الـ Object بيرجع يظهر تاني داخل الـ Bucket.

الـ Versioning بيخلي حذف الـ Object يعمل Delete Marker بدل ما يمسخ الـ Object نهائياً، ولما أمسح الـ Marker أقدر أرجع الـ File.

S3 Replication

الـ S3 Replication بتسمحلي أنقل نسخة من الـ Files من Bucket إلى Bucket ثانية.

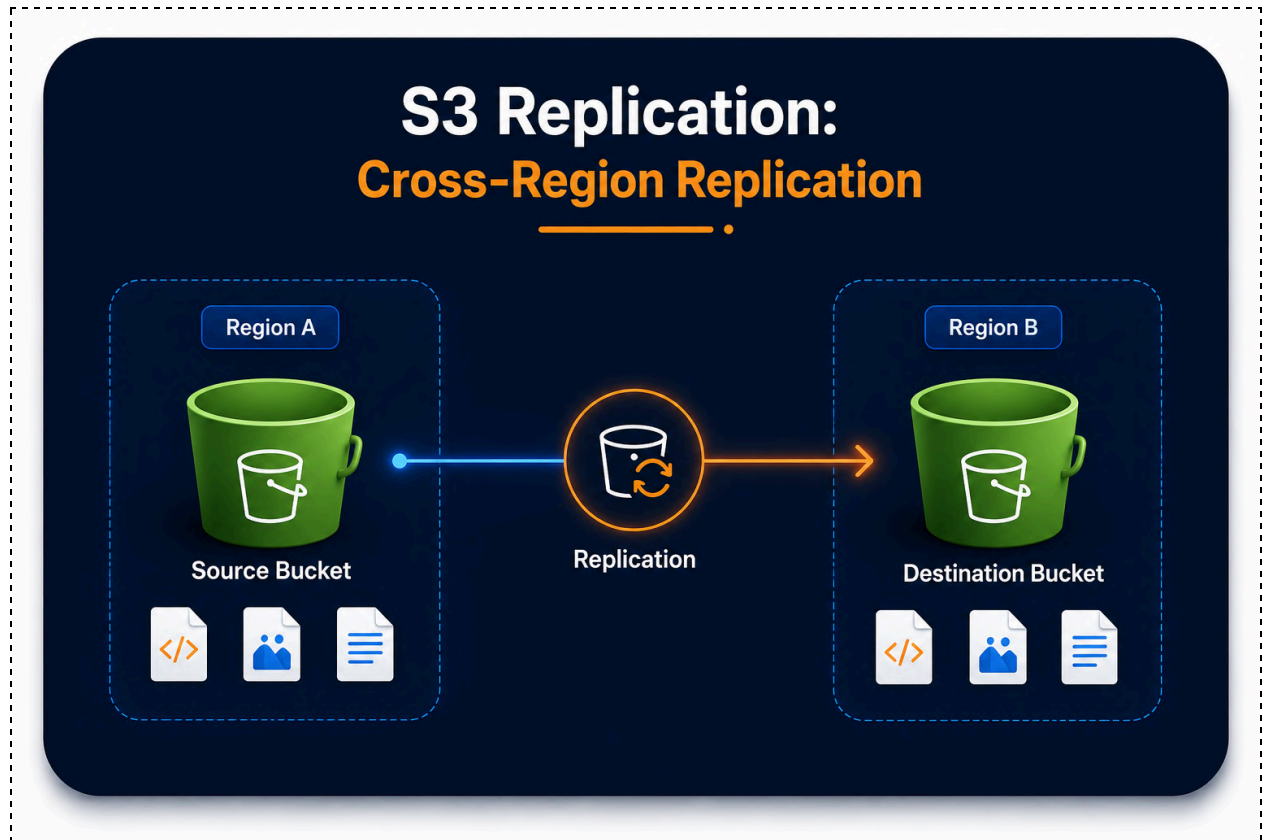
أي File موجود في الـ Source Bucket ممكن يتعمل له Replication ويتنقل إلى الـ Destination Bucket.

قبل ما أعمل Replication، لازم الـ Versioning يكون مفعل.

Replication Types

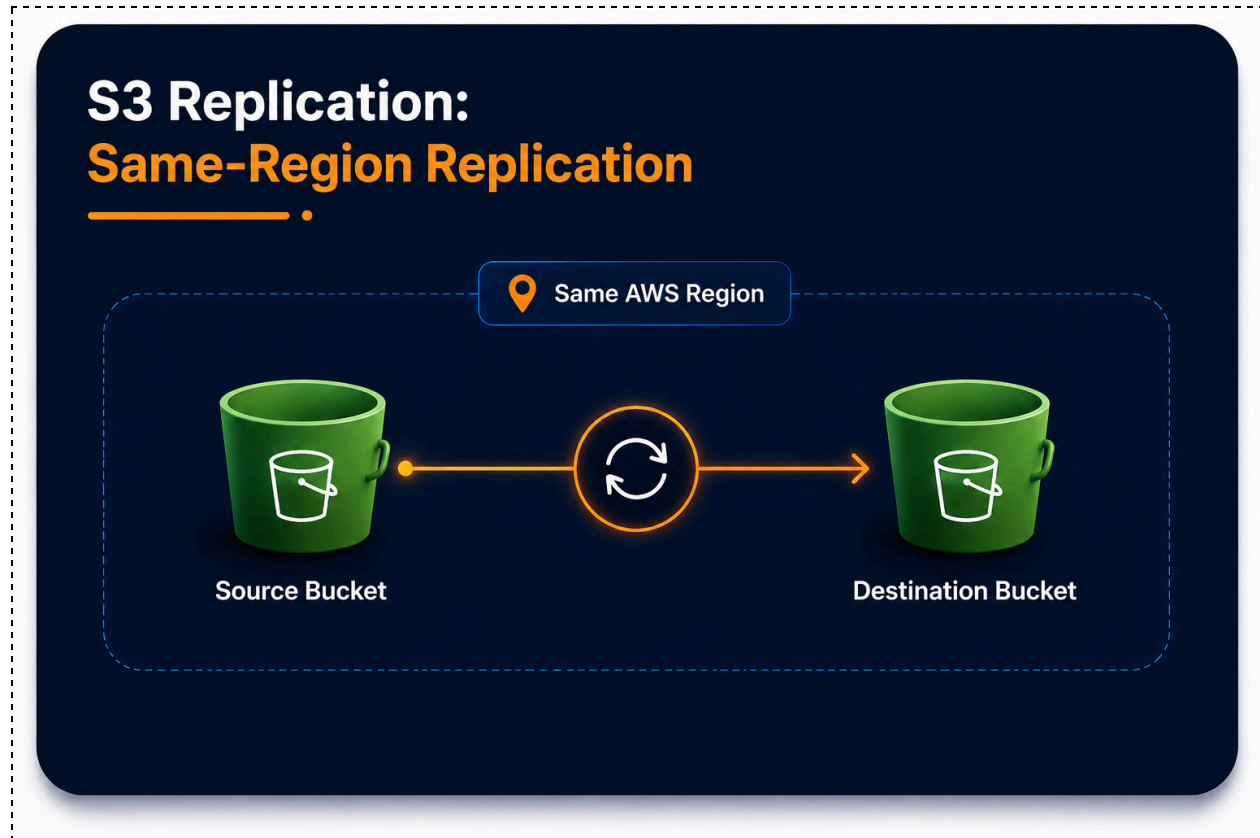
في نوعين من الـ Replication:

Cross-Region Replication



الـ Cross-Region Replication بيتم لما الـ Source Bucket والـ Destination Bucket يكونوا موجودين في Regions مختلفة.

Same-Region Replication



الـ Same-Region Replication سيتم لما الـ Source Bucket والـ Destination Bucket يكونوا موجودين في نفس الـ Region.

Creating a Replication Rule

كان عندي الـ Source Bucket:

S3-CloudCode

وعملت Bucket جديدة عشان تكون الـ Destination:

S3-CloudCode-Replica

الـ Bucketين كانوا موجودين في نفس الـ Region، وبالتالي ده الـ Same-Region Replication.

عشان أعمل Replication Rule:

1. أفتح الـ Source Bucket.
2. أدخل على **Management**.
3. أختار **Create replication rule**.
4. أكتب اسم للـ Rule.
5. أحدد إن الـ Source هي **S3-CloudKode**.
6. أطبق الـ Rule على كل الـ Objects داخل الـ Bucket.
7. أحدد إن الـ Destination Bucket موجودة في نفس الـ Account.
8. أختار **S3-CloudKode-Replica** كـ Destination.
9. أعمل Create للـ IAM Role.
10. أحفظ الـ Rule.

أثناء إنشاء الـ S3، Rule بتسأل هل عايز أعمل Replication للـ Objects القديمة الموجودة بالفعل، ولا للـ Objects الجديدة فقط.

اخترت إن الـ Objects القديمة زي الـ Logo وملف **index.html** مايتعملهاش Replication، وأي Object جديد يترفع بعد إنشاء الـ Rule ينتقل للـ Destination Bucket.

بعد ما رفعت HTML File جديد على **S3-CloudKode**، انتظرت ثواني وعملت Refresh للـ Destination Bucket، ولقيت الـ File موجود داخل **S3-CloudKode-Replica**.

Storage Classes

الـ Storage Classes بتوضح الطرق المختلفة اللي أقدر أأخزن بيها الـ Data داخل S3.

كل Storage Class بتختلف عن الثانية في:

- الـ Cost.
- الـ Availability.
- الـ Latency.
- الوقت المطلوب عشان أعمل Retrieve للـ File.
- هل الـ Data هتكون موجودة في أكثر من Availability Zone ولا Availability Zone واحدة.

S3 Standard

الـ S3 Standard أو الـ General Purpose هي أعلى Storage Class.

بتتميز بـ:

- الـ Availability عالية جدًا.
- الـ File بيكون متاح تقريبًا طول الوقت.
- أقدر أعمل Retrieve للـ File بسرعة.
- مفيش Latency ملحوظة عند الوصول للـ Data.

S3 Standard-Infrequent Access

الـ S3 Standard-IA بتستخدم مع الـ Files اللي محتاج أأخزنها، لكن مش بعمل لها Access بشكل متكرر.

مثال: File موجود داخل S3، لكن بفتحها كل أسبوع أو كل أسبوعين.

مقارنة بـ S3 Standard:

- الـ Cost أقل.
- الـ Availability عالية جدًا.
- الـ Retrieval ممكن ياخد بعض الـ Milliseconds.

S3 One Zone-Infrequent Access

في الـ S3 Standard-IA والـ Standard، الـ Data بتكون موجودة في أكثر من Availability Zone داخل الـ Region.

في **S3 One Zone-IA**، الـ File بيتخزن في Availability Zone واحدة فقط. الـ Cost بتاعها أقل من **S3 Standard-IA**.

S3 Glacier

S3 Glacier بتستخدم أكثر في Archiving وتخزين الـ Files لمدة طويلة.

فيها أكثر من نوع:

Glacier Instant Retrieval

الـ Data ممكن تتعمل لها Retrieval خلال Milliseconds.

أي File بيتخزن باستخدام **Glacier Instant Retrieval** لازم يفضل متخزن لمدة 90 Minimum يوم.

Glacier Flexible Retrieval

الـ Minimum Storage Duration هي 90 يوم.

فيها أكثر من Retrieval Option:

- **Expedited**: الـ File ممكن يظهر خلال دقائق.
- **Standard**: ممكن ياخذ من 3 إلى 5 ساعات.
- **Bulk**: ممكن ياخذ من 5 إلى 12 ساعة.

كل اختيار بيختلف في طريقة الاستخدام والتسعير.

Glacier Deep Archive

الـ Minimum Storage Duration هي 180 يوم.

خيارات الـ Retrieval:

- **Standard**: ممكن ياخذ 12 ساعة.
- **Bulk**: ممكن ياخذ 48 ساعة.

S3 Intelligent-Tiering

S3 Intelligent-Tiering ممكن أعمل لها Enable عشان تراقب الـ Files وتنقلها أوتوماتيك بين الـ Storage Classes.

لو File بقاله فترة محدش بيعمل له Access، ممكن يتنقل إلى Storage Class تكلفتها أقل بدل ما يفضل في Class تكلفتها عالية.

Changing Storage Classes

لما أرفع Object جديد، السـ Default Storage Class بتكون **S3 Standard**.

أثناء الـ Upload، أقدر أنزل إلى **Properties** وأختار السـ Storage Class اللي محتاجها، زي:

- S3 Standard.
- S3 Intelligent-Tiering.
- S3 Standard-IA.
- S3 One Zone-IA.
- S3 Glacier.

لو السـ Object مرفوع بالفعل، أقدر أفتح خصائصه وأعمل Edit للسـ Storage Class وأغيرها.

Lifecycle Rules

السـ Lifecycle Rule بتسمحلي أعمل Automation لنقل السـ Objects بين السـ Storage Classes بعد عدد أيام معين.

عشان أعمل Lifecycle Rule:

1. أفتح السـ S3 Bucket.
2. أدخل على **Management**.
3. أختار **Create lifecycle rule**.
4. أكتب اسم للسـ Rule، زي **CloudCode**.
5. أحدد السـ Objects اللي السـ Rule هتتطبق عليها.
6. أضيف السـ Transitions المطلوبة.

مثال:

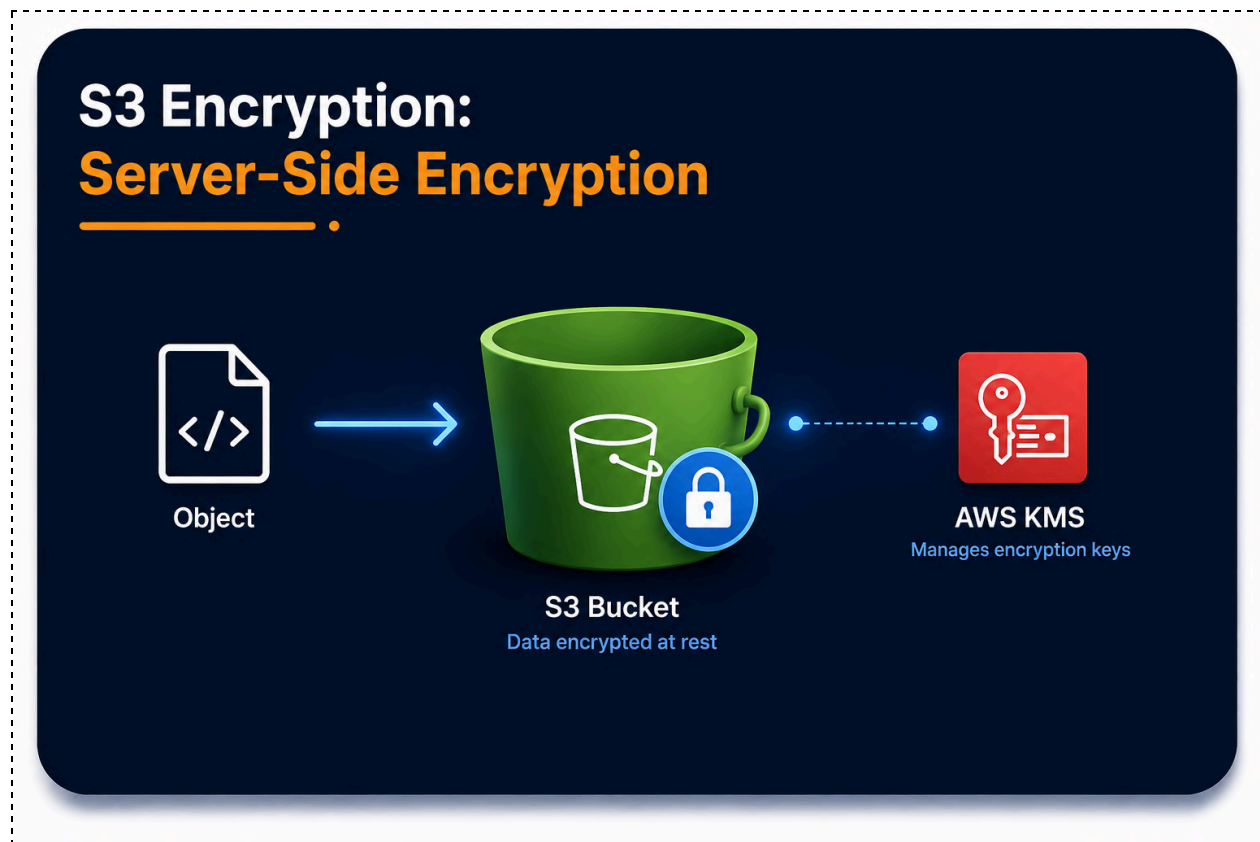
- السـ Objects تبدأ على **S3 Standard**.
- بعد 7 أيام تنتقل إلى **S3 Standard-IA**.
- بعد 18 يوم تنتقل إلى **S3 One Zone-IA**.

بالطريقة دي أقدر أتحكم في انتقال الـ Objects بين الـ Storage Classes بناءً على عدد الأيام.

S3 Encryption

في نوعين من الـ Encryption:

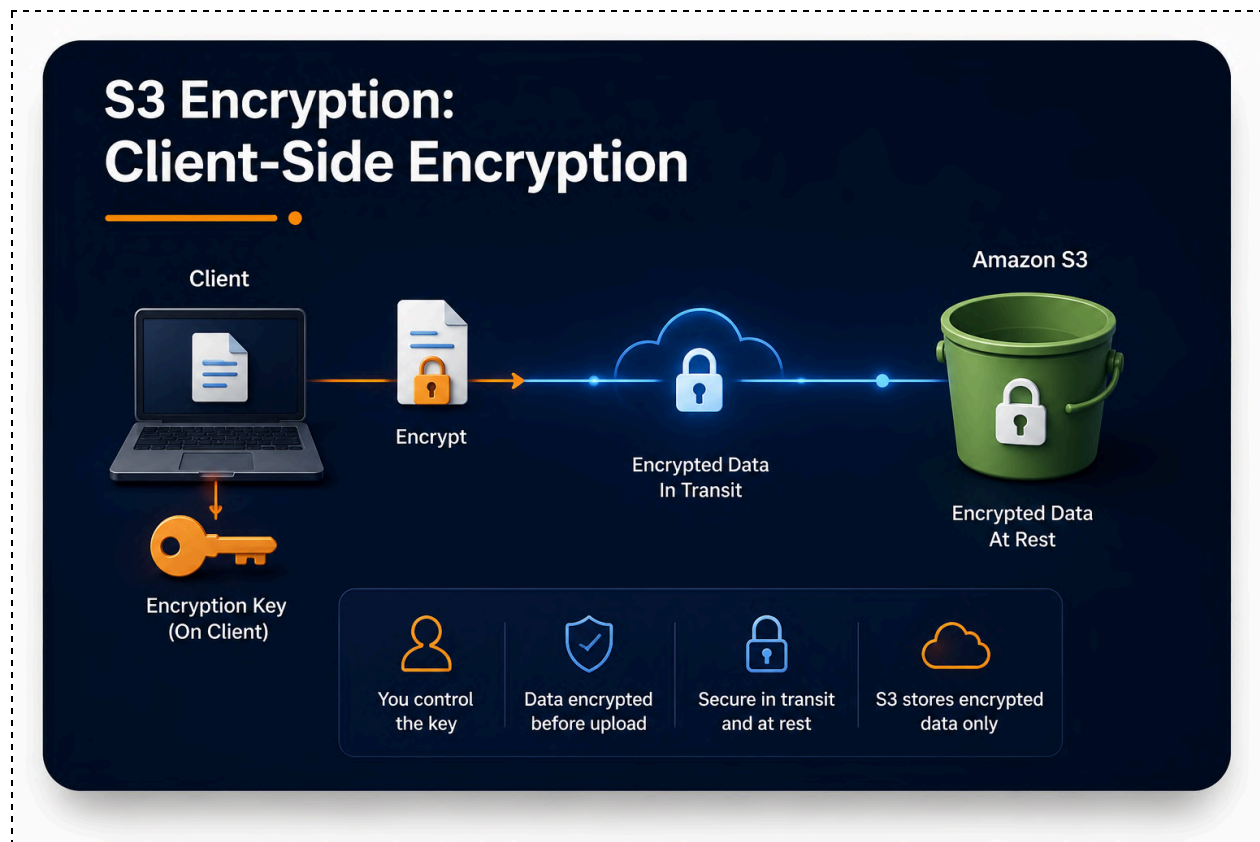
Server-Side Encryption



في الـ Server-Side Encryption، الـ User بيرفع الـ Document إلى S3، والـ Encryption Key بتكون ناحية الـ Server.

S3 بتعمل Encryption للـ Data عند الـ Server بعد رفعها.

Client-Side Encryption



في Client-Side Encryption، الـ User سيكون عنده الـ Encryption Key.

الـ User بي عمل Encryption للـ File قبل ما يرفعه، وبعد كده يرفع الـ Encrypted File إلى S3.

Video 5 – Identity & Access Management

Identity and Access Management

IAM اختصار لـ Identity and Access Management، وفكرتها إنني أعرف Users وGroups، وبعد كده أحدد لهم يقدرُوا يعملُوا إيه وما يقدرُوا يعملُوا إيه على الـ Infrastructure بتاعتي في AWS.

IAM هي Global Service، يعني مش محتاج أختار Region الأول عشان أعمل Configuration لـ IAM. من خلال IAM بقدر أعمل:

- Users.
- Groups.
- Roles.
- Permissions.

Root Account

أول ما بعمل AWS Account، الحساب الأساسي بيكون هو الـ Root Account.

الـ Root Account بيكون عنده Full Privileges، ومن خلاله أقدر أبدأ أعمل Configuration للـ Resources والخدمات اللي محتاجها.

بعد كده، لو عندي Team أو موظفين في الشركة، ببدأ أعمل لهم Users وأحدد الصلاحيات المناسبة لكل واحد.

Users and Policies

ممكن أعمل User باسم **Ahmed** وأعمل له Policy.

الـ Policy عبارة عن Permissions بتحدد:

- إيه اللي الـ User يقدر يعمل.
- إيه اللي الـ User ما يقدرش يعمل.
- الـ Resources اللي يقدر يعمل عليها Access.

الـ Policy ممكن تتعمل Attach مباشرة على User من غير ما يكون موجود داخل Group.

Groups and Policies



يمكن عمل Group باسم **Operations**، وأضيف جواه Users زي:

- Rizk.
- Ali.

بعد كده عمل Attach للـ Group لـ **Operations Policy**.

الـ Policy دي بتحتوي على الصلاحيات اللي كل أعضاء الـ Group يقدرُوا يستخدموها. أي User يدخل الـ Group بياخد الـ Permissions الخاصة بالـ Group بشكل تلقائي. ممكن كمان عمل Group تانية باسم **Developers**، وأضيف جواها Users زي:

- Ramy.
- Amr.

وعمل Attach للـ Group لـ **Developers Policy**.

أي User يدخل الـ Developers Group بياخد صلاحيات الـ Developers Policy.

Multiple Group Membership

الـ User ممكن يكون موجود في أكثر من Group.

لو **Ali** موجود في:

- Operations Group.
- Developers Group.

يـ يحصل على Permissions الخاصة بالـ Operations Group والـ Developers Group مع بعض.

IAM Policies

الـ Users والـ Groups بيتعمل لهم Assign لـ Policies، والـ Policies بتكتب بصيغة JSON. الـ Policy بتدي Permissions للـ User أو الـ Group، وتحدد العمليات اللي يقدروا ينفذوها. الـ Policy ممكن تحتوي على Statement واحدة أو أكثر.

Effect

الـ **Effect** بيحدد هل العملية مسموحة ولا مرفوضة.

القيمة ممكن تكون:

- Allow
- Deny

في المثال، الـ Effect كان:

```
"Effect": "Allow"
```

Action

الـ **Action** بيحدد العمليات اللي مسموح للـ User أو الـ Group ينفذها.

في المثال، العمليات كانت:

```
GetObject  
PutObject
```

ده معناه إن الـ User أو الـ Group يقدر يقرأ Objects ويرفع Objects.

Resource

الـ **Resource** بيحدد الخدمة أو الـ Resource اللي العمليات هتتفد عليها.

في المثال، الـ Policy كانت بتسمح بعمل **GetObject** و **PutObject** على S3 Bucket باسم **CloudCode**.

الصلاحيات دي خاصة بالـ Bucket المحددة، مش بكل S3 Buckets.

لو عملت Attach للPolicy دي على User زي **Amr** أو على Developers Group، الـUser أو أعضاء الـGroup يقدرُوا ينفذوا العمليات المحددة على الـResource المحددة.

Multi-Factor Authentication



MFA اختصار لـ Multi-Factor Authentication.

من المهم أعمل Enable لـ MFA على الـ Accounts والـ Users، وخصوصاً الـ Root Account.

بدل ما تسجيل الدخول يعتمد على الـ Password فقط، يحتاج:

1. الـ Password.
2. Code بيتغير بشكل مستمر.

الكود ممكن يتغير كل دقيقة تقريباً.

حتى لو حد عرف الـ Password، مش هيقدر يدخل على الـ Account من غير الكود.

Virtual MFA

في أغلب الحالات، الـ MFA بيكون Virtual.

ممكن أستخدم تطبيق زي:

Google Authenticator

لما أعمل Enable للـ AWS، MFA بتعرض QR Code.

الخطوات بتكون:

1. أفتح Google Authenticator.
2. أعمل Scan للـ QR Code.
3. يظهر Entry جديد داخل التطبيق.
4. التطبيق يولد Code بيتغير باستمرار.
5. عند تسجيل الدخول، أدخل الـ Password والـ Code.

حتى لو حد عنده Password الخاصة بالـ Root Account، مش هيقدر يدخل من غير الكود الموجود على الموبايل.

Hardware MFA

ممكن الكود يتولد من Physical Device بدل تطبيق على الموبايل.

في Devices معينة بتكون Supported by AWS، وبتعرض الكود اللي لازم أدخله بعد الـ Password.

الـ MFA ممكن يتعمل لها Enable على:

- Root Account.
- IAM User Account.

IAM Console

بعد إنشاء AWS Account وتسجيل الدخول بالـ Root Account، أكتب في البحث:

Identity and Access Management (IAM)

بعد كده أفتح IAM Dashboard.

Creating an Admin Group

من IAM Dashboard أقدر أعمل Group.

في التطبيق، عملت Group باسم:

Admin

بعد كتابة اسم الـ Group، حددت الـ Permissions اللي عايز أديها للـ Group.

اخترت:

AdministratorAccess

بعد إنشاء الـ Group، ظهر عندي Admin Group من غير Users، ومعاه Administrator Permissions.

Creating a User

بعد كده عملت User باسم:

Amr

أثناء إنشاء الـ User، حددت إنه يقدر يعمل Access على AWS Management Console.

إتاحة Console Access معناها إن الـ User يقدر يدخل على واجهة AWS اللي بستخدمها من المتصفح.

في حالات تانية، ممكن الـ User يستخدم Configuration من Linux Machine أو من Visual Studio، وساعتها مش لازم يكون عنده Console Access.

User Password

أثناء إنشاء الـ User، قدرت أختار إن الـ Password تكون:

- Auto-generated.
- Custom.

اخترت Auto-generated Password.

يمكن كمان أحدد إن الـ User لازم يغير الـ Password في أول مرة يعمل فيها Sign In.

في الحالة دي، بيعت له الـ Auto-generated Password، وبعد تسجيل الدخول بيعمل Password خاصة بيه.

Assigning User Permissions

بعد إنشاء User باسم **Amr**، كان عندي اختيارين أساسيين لإضافة الصلاحيات:

1. أضيفه داخل Group.

2. أعمل Attach لـ Policy مباشرة على الـ User.

لو ضفته داخل Admin Group، هياخد Permissions الخاصة بالـ Group.

في التطبيق، اخترت أعمل Attach مباشرة لـ:

AdministratorAccess

بعد كده أنشأت الـ User، وظهر الـ Password والـ Sign-in URL الخاصين به.

Signing In as an IAM User

عشان الـ User يعمل Sign In، بيستخدم:

- Sign-in URL.
- Account ID.
- Username.
- Password.

دخلت باستخدام Username:

Amr

وبعد كتابة الـ Password، دخلت على الـ Account بصلاحيات Administrator.

Account Alias

الـ Sign-in URL سيكون فيه Account ID.

ممکن أغير الجزء الخاص بالـ Account ID باستخدام Account Alias.

عملت Alias باسم:

CloudKode

الـ Alias لازم يكون Unique، لأن الـ URL الناتج سيكون خاص بالـ AWS Account.

بعد إنشاء الـ Alias، أي User يقدر يستخدم الـ Sign-in URL الجديد بدل الـ URL اللي فيه Account ID. عند تسجيل الدخول، ممكن أستخدم:

- Account ID.
- Account Alias.

بعد كده أدخل الـ Username والـ Password.

Assigning Users to Groups

عملت User ثاني باسم:

Ali

أثناء إنشاء الـ User، اخترت إنه يكون موجود داخل Admin Group بدل ما أعمل Attach لـ Policy مباشرة عليه. بمجرد إضافة **Ali** إلى Admin Group، حصل على Administrator Permissions الخاصة بالـ Group.

User Security Settings

من إعدادات الـ User أقدر أعمل Configuration للـ MFA.

أختار Authenticator App، وبعد كده يظهر QR Code.

أعمل Scan للـ QR Code باستخدام التطبيق، وبعدها كل مرة أعمل Access للـ Account لازم أدخل:

- الـ Password.
- الـ Code الظاهر في التطبيق.

Password Policy

من خلال الـ Root Account أقدر أحدد Password Policy الخاصة بالـ Users.

الـ Default Policy يتطلب إن الـ Password تكون Minimum Length بتاعها 8 Characters، وتحتوي على عناصر زي:

- Uppercase Letters.
- Lowercase Letters.
- Numbers.

أقدر أغير الإعدادات، مثلاً:

- أخلي الـ Minimum Length تساوي 12.
- أضيف Requirements إضافية.
- أخلي الـ Password تنتهي بعد فترة.
- أجبر الـ User يغير الـ Password كل شهرين.

AWS Access Methods



في ثلاث طرق أقدر أستخدمهم للوصول إلى AWS Infrastructure.

AWS Management Console

AWS Management Console هي الواجهة التي يستخدمها من المتصفح.

عشان أدخل عليها، بستخدم:

- URL.
- Username.
- Password.
- MFA Code

من خلال الـ Console أقدر أعمل Configuration للـ Resources حسب الـ Privileges الخاصة بالـ User.

AWS CLI

AWS CLI هي Tool ممكن أنزلها على Machine زي Linux Machine. بعد تثبيت AWS CLI، لازم أوصلها بالـ AWS Account عن طريق Authentication. الـ User يقدر يعمل Create لحاجة اسمها Access Key. الـ Access Key بتتكوّن من:

- Access Key ID.
- Secret Access Key.

بعد تثبيت AWS CLI على الـ Machine، بستخدم الـ Access Key ID والـ Secret Access Key عشان أوصل الـ Machine بالـ AWS Account. بعد نجاح الـ Configuration، أقدر أنفذ تغييرات على AWS من خلال الـ CLI من غير ما أفتح AWS Management Console. كل العمليات بتكون حسب الـ Permissions الخاصة بالـ User.

AWS SDK

AWS SDK بتستخدم داخل الـ Applications. ممكن يكون عندي Application، وأستخدم الـ SDK عشان أوصله بالـ AWS Account. الـ Application بيحتاج Access Key عشان يعمل Authentication ويتصل بالـ Account.

AWS CLI Configuration

Installing AWS CLI

AWS CLI ممكن تثبيت على:

- Windows.
- Linux.
- macOS.

طريقة التثبيت بتختلف حسب الـ Operating System.

على Ubuntu، بستخدم الـ Commands الخاصة بتثبيت AWS CLI.

بعد التثبيت، بقدر أوصل الـ Linux Machine بالـ AWS Account.

Creating an Access Key

عشان أعمل Access Key للـ User:

1. أفتح IAM.
2. أفتح الـ User المطلوب.
3. أدخل على **Security credentials**.
4. أنزل إلى Access Keys.
5. أضغط على **Create access key**.
6. أحدد إن الاستخدام هيكون مع **Command Line Interface**.
7. أعمل Create للـ Access Key.

بعد الإنشاء، AWS بتعرض:

- Access Key ID.
- Secret Access Key.

القيم دي بستخدمها أثناء Configuration للـ AWS CLI.

Configuring AWS CLI

بعد تثبيت AWS CLI، أستخدم:

```
Aws configure
```

بعد تشغيل الـ AWS CLI، Command بتطلب:

1. Access Key ID
2. Secret Access Key
3. Default Region

بعد إدخال المعلومات، الـ Machine بتكون متصلة بالـ AWS Account.

Testing S3 Access

بعد عمل Configuration للـ CLI، أقدر أشوف الـ S3 Buckets الموجودة داخل الـ Account باستخدام:

```
aws s3 ls
```

في التطبيق، ظهر S3 Bucket باسم:

```
CloudASDX
```

ولما فتحت S3 من الـ AWS Management Console، لقيت الـ Bucket موجودة بنفس الاسم.

IAM Roles

في الوضع الطبيعي، يعمل User وأدي له Policy فيها Permissions تحدد بقدر يعمل إيه. مثال: لو عندي S3 Bucket، أقدر أدي User Permission عشان يعمل Access عليها. لكن ممكن أحتاج AWS Service نفسها تعمل Access على Service ثانية. مثال:

عندي EC2 Instance، وعايزها تعمل Access على S3 Bucket وترفع عليها Files. ممكن يكون عندي Files على VM، وعايز في نهاية اليوم تنتقل إلى S3. عشان أعمل ده، بستخدم IAM Role.

Role Purpose

IAM Role بتسمح لـ AWS Services إنها تنفذ Actions.

مثال: أدي EC2 Instance Role تسمح لها إنها:

- تقرأ Files من S3.
- تكتب Files على S3.

الـ Role بتشتغل باستخدام Permissions، بشكل مشابه للـ Policies الخاصة بالـ Users.

الـ Role ممكن تتعمل Assign لخدمات مختلفة، زي:

- EC2.
- Lambda.
- API Gateway.
- Amazon Q.

Creating a Role

عشان أعمل IAM Role:

1. أفتح IAM.
2. أدخل على Roles.
3. أضغط على Create role.
4. أختار إن الـ Trusted Entity هي AWS Service.

5. أعدد الخدمة اللي الـ Role هتتعمل Assign ليها.

في التطبيق، اخترت:

EC2

بعد كده ضغطت **Next**.

Assigning Role Permissions

بعد تحديد إن الـ Role خاصة بـ EC2، حددت الـ Permissions اللي الـ Role هتاخد.

في المثال، اخترت:

AmazonS3FullAccess

ده معناه إن الـ Role تقدر تعمل Full Access على S3.

بعد كده سميت الـ Role:

EC2-S3-Role

الـ Role دي:

- عندها Amazon S3 Full Access.
- معموله Assign لـ EC2 Instances.

من خلال الـ Trust Relationship أقدر أشوف الخدمة اللي مسموح لها تستخدم الـ Role.

Video 6 – Databases & Analytics

Database Fundamentals

Databases and Traditional Storage

الـ Database مختلفة عن الـ Storage العادية، لأنها بتخليني أحط الـ Data بشكل معين داخل الـ Tables، وأقدر أبني الـ Relationships بين الـ Tables والـ Data الموجودة جواها.

كل Database بيكون ليها الـ Features و Limitations مختلفة عن الأنواع التانية.

الهدف هنا مش شرح كل حاجة خاصة بالـ Databases، لكن توضيح الأنواع المختلفة الموجودة على AWS.

On-Premises Database Responsibilities

في الـ On-Premises، عشان أشغّل Database بكون مسؤول عن حاجات كتير:

1. أجهز الـ Physical Server.
2. أنزل عليه الـ Operating System.
3. أبني الـ High Availability.
4. أجهز الـ Primary Server و Backup Server.
5. أحدد الـ Storage اللي السيرفرات هتستخدمها.
6. أحدد الـ Storage Capacity تكفي الـ Data.
7. أجهز الـ Backup Server.
8. أعمل الـ Backups بشكل مستمر.

وجود الـ Backup مهم عشان لو السيرفرات وقعت أو حصلت فيها مشكلة، أقدر أرجع الـ Data المحفوظة.

Fully Managed Databases

AWS شالت مسؤوليات كثير من خلال الـ Fully Managed Database.

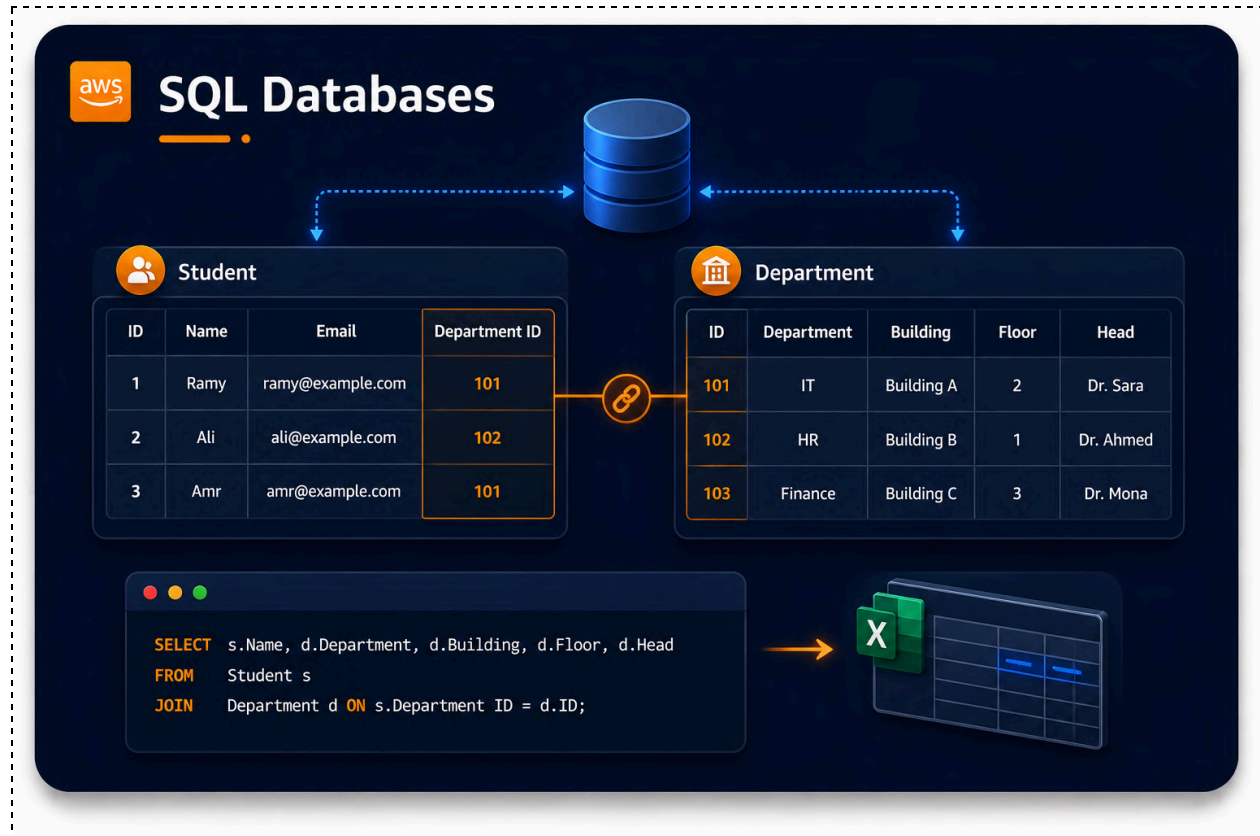
في الحالة دي، AWS بتكون مسؤولة عن:

- The Physical Servers.
- The Operating System.
- High Availability.
- Storage Scaling.
- Automated Backups.
- Upgrades.

أقدر أستخدم الـ Database وأخلي الـ Storage تعمل Scale حسب الـ Demand من غير ما أدير التفاصيل دي بنفسى.

SQL and NoSQL Databases

SQL Databases



SQL Databases هي Relational Databases.

الـ Data بتكون موجودة داخل Tables مرتبطة ببعض من خلال Relationships.

شكل الـ Table بيكون قريب من Excel Sheet، ويمكن يحتوي على Fields زي:

- ID.
- Department.
- Name.
- Email.

مثلاً ممكن يكون عندي Student Table و Department Table، وفي Relationship بينهم من خلال Department Field.

من خلال العلاقة دي أقدر أعرف معلومات عن الـ Student، وفي نفس الوقت أوصل لمعلومات الـ Department زي:

- مكانها في أنهي Building.
- موجودة في أنهي Floor.
- اسم الـ Head الخاص بيها.

اسمها SQL Database لأنني بقدر أعمل Query أو Search داخل الـ Tables باستخدام SQL Language.

NoSQL Databases



NoSQL Databases هي Non-Relational Databases.

الـ Data الموجودة جواها مش لازم تكون مرتبطة ببعض، ويمكن يكون شكلها مكتوب بصيغة JSON.

من مميزاتها:

- Horizontal Scaling.

- High Performance.

Horizontal Scaling معناه إنني أقدر أحط أكثر من Database داخل Cluster واحد.

Amazon RDS

Amazon RDS هي Relational Database Service، وهي Managed Database بقدر أستخدام معاهها SQL Language.

Amazon RDS بتدعم ستة Database Engines أقدر أختار منهم، ومن أشهرهم MySQL.

RDS Benefits

بدل ما أجيب EC2 Instance وأنزل عليها Database بنفسني، Amazon RDS بتوفرلي:

- Vertical Scaling.
- Horizontal Scaling.
- Multi-AZ Setup للDisaster Recovery.
- تشغيل الـDatabase في أكثر من Availability Zone.
- Automated Backups.
- Restore لنقطة زمنية محددة.
- إدارة الـOperating System.
- Monitoring Dashboards.

أقدر مثلاً أطلب Restore للـDatabase للحالة اللي كانت عليها الساعة 8:30 امبارح.

RDS Limitation

الـDrawback الموجود مع Amazon RDS إنني ماقدرش أعمل SSH للـDatabase، لأنها مش EC2 Instance أنا مسؤول عنها.

Application Architecture

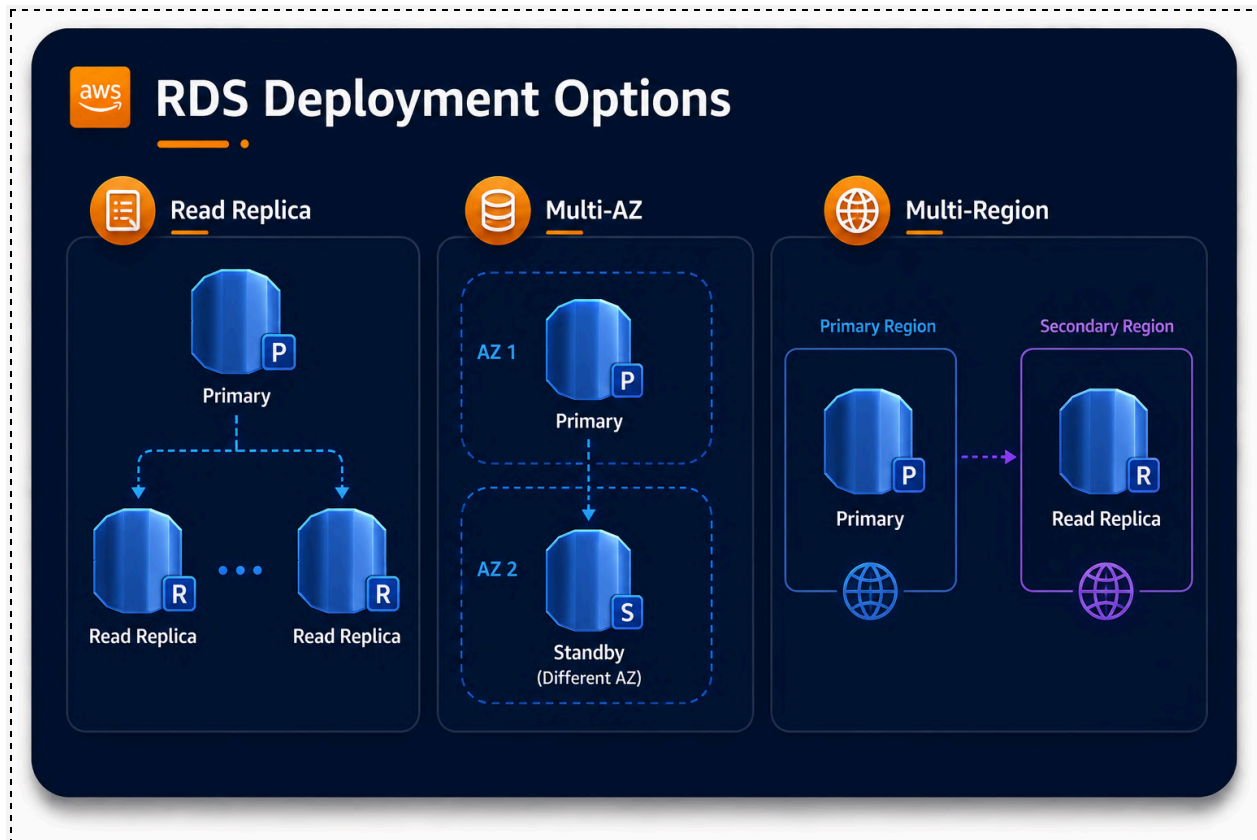
ممکن يكون عندي User جاي من الإنترنت، والـTraffic يوصل إلى Application Load Balancer.

الـLoad Balancer يوزع الـTraffic على Web Servers موجودة على EC2 Instances.

الـWeb Server يستقبل الـOrder الخاص بالـUser، وبعد كده يحتاج يخزن معلومات الـUser والـOrder.

هنا يستخدم Amazon RDS، والـ EC2 Instances تقدر تعمل Read و Write على الـ Database.

RDS Deployment Options



Read Replica

في الـ Read Replica بيكون عندي الـ Main RDS Database.

الـ Application يقدر يعمل على الـ Main Database:

- Read.
- Write.

عشان ماحملش كل عمليات القراءة على الـ Main Database، أقدر أعمل Read Replica تحتوي على نفس الـ Data الموجودة في الـ Main.

الـ Applications تقدر تعمل Read فقط من الـ Read Replica.

بالشكل ده:

- عمليات الـ Write تروح إلى الـ Main RDS.
- عمليات الـ Read ممكن تروح إلى الـ Read Replica.
- الـ Load يقل على الـ Main Database.

Multi-AZ

في Multi-AZ بيكون عندي:

- Main RDS Database.
- Standby أو Backup RDS Database.

الـ Application تعمل Read و Write على الـ Main Database.

لو الـ Main وقعت، الـ Standby Database تبقى هي الـ Main، والـ Application تبدأ تتعامل معاها وتعمل عليها Read و Write.

الـ Main RDS بتكون في Availability Zone، والـ Failover Database بتكون في Availability Zone ثانية.

بالشكل ده، لو حصلت مشكلة في الـ Availability Zone الخاصة بالـ Main Database، الـ Failover Database تحل مكانها.

Multi-Region

Multi-Region شبيهة بفكرة الـ Read Replica، لكن كل Database بتكون في Region مختلفة.

مثلاً:

- Main RDS في London.
- Database في Dubai.
- Database في North Virginia.

الـ Application الموجودة في Dubai تقدر تعمل Read من الـ Database الموجودة في Dubai.

لو محتاجة تعمل Write، تروح إلى الـ Main RDS الموجودة في London.

في Read Replica أو Multi-Region مفيش Backup Database بتحل مكان الـ Main تلقائيًا، وبالتالي لو الـ Main وقعت هتظهر مشكلة في عمليات الـ Write.

Creating an RDS Database

Creation Method

عشان أعمل Amazon RDS Database:

1. أكتب **RDS** في الـ Search Bar.
2. أفتح Amazon RDS.
3. أدخل على **Databases**.
4. أضغط على **Create database**.

AWS بتديني اختيارين:

- Standard Create.
- Easy Create.

اخترت Standard Create.

Database Engine

بعد كده أختار الـ Database Engine.

في التطبيق استخدمت:

MySQL

أقدر كمان أحدد Engine Version، لكن سيبتها على الـ Default.

Database Template

الـ Templates المتاحة بتشمل:

- Production.
- Dev/Test.
- Free Tier.

كل Template بتختلف في السعر والإعدادات.

اخترت Free Tier.

لو اخترت Production، أقدر أحدد هل الـ Deployment يكون:

- Cluster فيه Primary Database و Standby Database.
- Multi-AZ فيه Primary Database و Standby Database.
- Single Instance.

مع Free Tier، الاختيار سيكون Single Database من غير Standby.

Database Settings

حدد اسم الـ Database:

CloudKode-Database

وحدد الـ Username:

admin

بعد كده كتبت الـ Password الخاصة بالـ Database.

Instance Class

زي EC2 Instances، أقدر أختار قوة الـ Database من خلال:

- CPU.
- RAM.

اخترت Instance Class موجودة داخل Free Tier.

اختيار الـ Instance Class يعتمد على القوة المطلوبة للـ Database.

Storage Configuration

اخترت:

- General Purpose SSD **gp2**.
- Storage Size تساوي 20 GB.

فعلت خيار Storage Auto Scaling.

ده بيسمح للـ Storage إنها تبدأ من 20 GB وتعمل Scale لحد 1000 GB عند الحاجة.

Connectivity

في إعدادات الـ Connectivity حددت:

- الـ VPC اللي الـ Database هتكون موجودة فيها.
- الـ Subnet.
- هل الـ Database تكون Publicly Accessible من الإنترنت.
- الـ Security Group.
- الـ Availability Zone.

اخترت إن الـ Database ماتكونش Publicly Accessible، وبالتالي ما يكونش ليها Public IP Address.

استخدمت الـ Default Security Group الموجود عندي.

Reviewing the Database

بعد الضغط على **Create database**، بدأت AWS تعمل الـ Creation للـ Database.

من صفحة الـ Database أقدر أشوف معلومات زي:

- الـ Database Class.
- الـ CPU والـ Memory.
- الـ Availability Zone.
- الـ Connectivity.
- الـ VPC.
- الـ Security Group.
- الـ Endpoint المستخدم للاتصال بالـ Database.
- الـ Port المستخدم في الاتصال.

Database Monitoring

بما إن Amazon RDS Fully Managed Database، بيكون عندي Dashboard أقدر أعمل منها Database Monitoring.

أقدر أتابع:

- CPU.
- Memory.
- باقي المعلومات الخاصة بالـ Database.

Database Operations

حتى لو أنشأت الـ Database كـ Single Availability Zone، أقدر أعمل Modify وأحولها إلى Multi-AZ Deployment.

أقدر كمان:

- أخذ Snapshot من الـ Database.
- أستخد Snapshot في مكان ثاني.
- أعمل Restore للـ Database.
- أعمل Point-in-Time Restore للحالة اللي كانت عليها الـ Database من ساعة مثلاً.

Amazon Aurora

Amazon Aurora هي Database من AWS، ومش Open Source.

Aurora بتدعم اثنين Database Engines.

أكثر حاجة بتميز Aurora عن Amazon RDS هي الـ Performance:

- ممكن توصل إلى خمسة أضعاف Performance الخاصة بـ MySQL على RDS.
- ممكن توصل إلى ثلاثة أضعاف Performance الخاصة بـ PostgreSQL على RDS.

الـ Storage تقدر تعمل Scale لحد 128 TB و Aurora أغلى من RDS بحوالي 20%.

Aurora Serverless

في Amazon RDS كنت بحدد الـ Instance Class، والـ CPU والـ RAM.

في Aurora Serverless مش محتاج أعمل Provision للـ Capacity دي.

Aurora Serverless بتعمل Scale تلقائيًا بناءً على الـ Usage، والدفع بيكون Per Second.

هي مناسبة للـ Unpredictable Workloads.

لو مش عارف الـ Database Capacity هتكون قد إيه، أو عندي أوقات معينة بيكون فيها الـ Load عالي جدًا، Aurora Serverless تقدر تتعامل مع اختلاف الـ Load.

Amazon ElastiCache



Amazon ElastiCache هي In-Memory Database بتوفر:

- High Performance.
- Low Latency.

الـ EC2 Instance أو الـ Application كانت بتعمل Read و Write مباشرة على Amazon RDS. عشان عمليات القراءة تكون أسرع، ElastiCache بتأخذ الـ Frequently Accessed Data الموجودة في RDS وتحتفظ بيها داخل الـ Cache.

الـ Application تقدر تقرأ الـ Data من ElastiCache بدل ما تعمل Read من RDS كل مرة.

ده بيؤدي إلى:

- تسريع عمليات القراءة.
- تقليل الـ Load على Amazon RDS.

AWS بتكون مسؤولة عن:

- Patching.
- Monitoring.
- Backup.

Amazon DynamoDB

Amazon DynamoDB هي NoSQL Database، ومش Relational زي Amazon RDS و Aurora.

DynamoDB بتيجي بـ Replication على Availability Zones، وهي Serverless.

مش محتاج أعمل Provision لـ:

- CPU.
- Memory.
- Instance Size.

أدخل وأعمل Table مباشرة، وهي عندها القدرة إنها تتعامل مع ملايين الـ Requests.

الـ Latency بتاعتها قليلة، والـ Cost مش عالي.

DynamoDB Accelerator

DynamoDB Accelerator أو DAX هي Fully Managed In-Memory Cache.

بتستخدم لتسريع Performance الخاصة بـ DynamoDB حوالي عشر مرات.

فكرتها شبيهة بـ ElastiCache.

الـ Frequently Accessed Data بيتعمل لها Caching، والـ Application تقرأها من الـ Cache بدل ما تقرأها مباشرة من DynamoDB.

Creating a DynamoDB Table

عشان أعمل DynamoDB Table:

1. أكتب **DynamoDB** في الـ Search Bar.
2. أفتح Amazon DynamoDB.
3. أختار **Create table**.
4. أكتب اسم الـ Table.
5. أحدد الـ Partition Key.
6. أضغط على **Create table**.

سميت الـ Table:

CloudKode-Table

واستخدمت Partition Key باسم:

UserID

الـ Partition Key بيكون Identification Key لكل Entry داخل الـ Database.

أثناء إنشاء الـ Table ماحددتش:

- CPU.
- Memory.
- Instance Size.

وده بسبب إن DynamoDB Serverless Database.

Adding Items

بعد إنشاء الـ Table، عملت Create Item جديد.

الـ Partition Key كان UserID.

أضفت User بالبيانات دي:

- User ID: 1
- First Name: Mohamed
- Last Name: Rizk
- Favorite Color: Red

بعد كده أضفت User ثاني:

- User ID: 2
- First Name: Karim
- Favorite Color: Blue
- Age: 30

Flexible Schema

في DynamoDB مش لازم كل الـ Items يكون عندها نفس الـ Attributes.

الـ User الأول كان عنده Last Name، لكن ماكانش عنده Age.

الـ User الثاني كان عنده Age، لكن ماكانش عنده Last Name.

دي من أفكار NoSQL Databases، لأن الـ Data مش لازم تكون ماشية على Structure ثابتة لكل Entry.

في SQL أو RDS، لو عندي Columns زي Age و First Name و Last Name، كل Rows بتكون ماشية على نفس شكل الـ Table.

Additional Database Services

Amazon DocumentDB

Amazon DocumentDB شبيهة بـ Aurora، لكن خاصة بـ MongoDB.

MongoDB يستخدم لتخزين وعمل Query للـ JSON Data.

DocumentDB هي Fully Managed Document Database.

بتوفر:

- High Availability.
- Replication في ثلاث Availability Zones.
- Serverless Option.

Amazon Neptune

Amazon Neptune هي Fully Managed Graph Database.

أول ما أشوف Graph Database، أقدر أربطها بـ Neptune.

بتوفر:

- High Availability على ثلاث Availability Zones.
- لحد 15 Read Replicas.
- Replication.

هي مناسبة لتخزين العلاقات والـ Knowledge اللي محتاج Graph، زي:

- Wikipedia.
- Social Networks.

مثلاً في Facebook بيكون عندي Users وعلاقات و Comments بينهم، ومحتاج أربط كل البيانات دي داخل Database.

AWS Database Migration Service

AWS Database Migration Service أو DMS هي Service تستخدم لعمل Migration من Database إلى Database ثانية.

يكون عندي:

- Source Database.
- DMS.
- Target Database.

DMS تنتقل الـ Source Database إلى الـ Target Database الجديدة.

Amazon MemoryDB for Redis

Amazon MemoryDB for Redis هي In-Memory Database، شبيهة بـ ElastiCache و DAX.

هي Compatible مع Redis، والـ Data يتم تخزين بطريقة Key-Value Store.

بتميز بـ:

- High Availability.
- Replication على أكثر من Availability Zone.
- Serverless.
- Auto Scaling حسب الـ Usage.
- Low Latency.
- High Performance.

مش محتاج أعمل Provision لـ:

- CPU.
- Memory.
- Storage.

هي مناسبة لحالات زي:

- Real-Time Applications.
- Session Management.
- تخزين Session Data الخاصة بالـ Users، زي Cookies.

Analytics Services

Amazon Redshift

Amazon Redshift هي Online Analytical Processing Service.

أول ما أشوف Online Analytical Processing، أقدر أربطه بـ Redshift.

بتستخدم في Data Warehousing.

Redshift هي Serverless، والدفع بيكون حسب الاستخدام.

بتستخدم في:

- Reports.
- Dashboards.
- Real-Time Analytics.

Amazon EMR

Amazon EMR مرتبطة بـ Hadoop Clusters.

أول ما أشوف Hadoop Cluster، أقدر أربطه بـ Amazon EMR.

[جزء من استخدامات Amazon EMR غير واضح في الـ Transcript]

Amazon Athena

Amazon Athena هي Serverless Query Service.

بتستخدم عشان أعمل Query على S3 Objects.

أول ما أشوف Query على S3 Objects، أقدر أربطه بـ Athena.

Athena بتستخدم SQL Language عشان تعمل Query للـ Files الموجودة على S3.

Amazon QuickSight

Amazon QuickSight بتوفر Interactive Dashboards للـ Business Analytics.

من خلالها أقدر أشوف اتجاهات الـ Business باستخدام Dashboards.

أول ما أشوف Interactive Dashboard للـ Data الخاصة بـ Business Analytics، أقدر أربطه بـ QuickSight.

QuickSight تقدر تعمل Integration مع:

- Amazon RDS.
- Amazon Aurora.
- Amazon Athena.
- Amazon Redshift.

AWS Glue

AWS Glue بتستخدم لما أحتاج أعمل Transform للـ Data.

ممكن تكون الـ Data مش مجهزة بالشكل المناسب عشان أعمل عليها Training أو Processing.

Glue بتأخذ الـ Data وتحولها إلى Format أو Dataset مناسبة أقدر أستخدمها بعد كده.

Amazon Kinesis

Amazon Kinesis هي Streaming Service.

بتستخدم في Real-Time Processing.

أول ما أشوف حاجة مرتبطة بالـ Real-Time Processing، أقدر أربطها بـ Kinesis.

مثلاً ممكن يكون عندي Stock Market والـ Stocks بتطلع وتنزل بشكل لحظي.

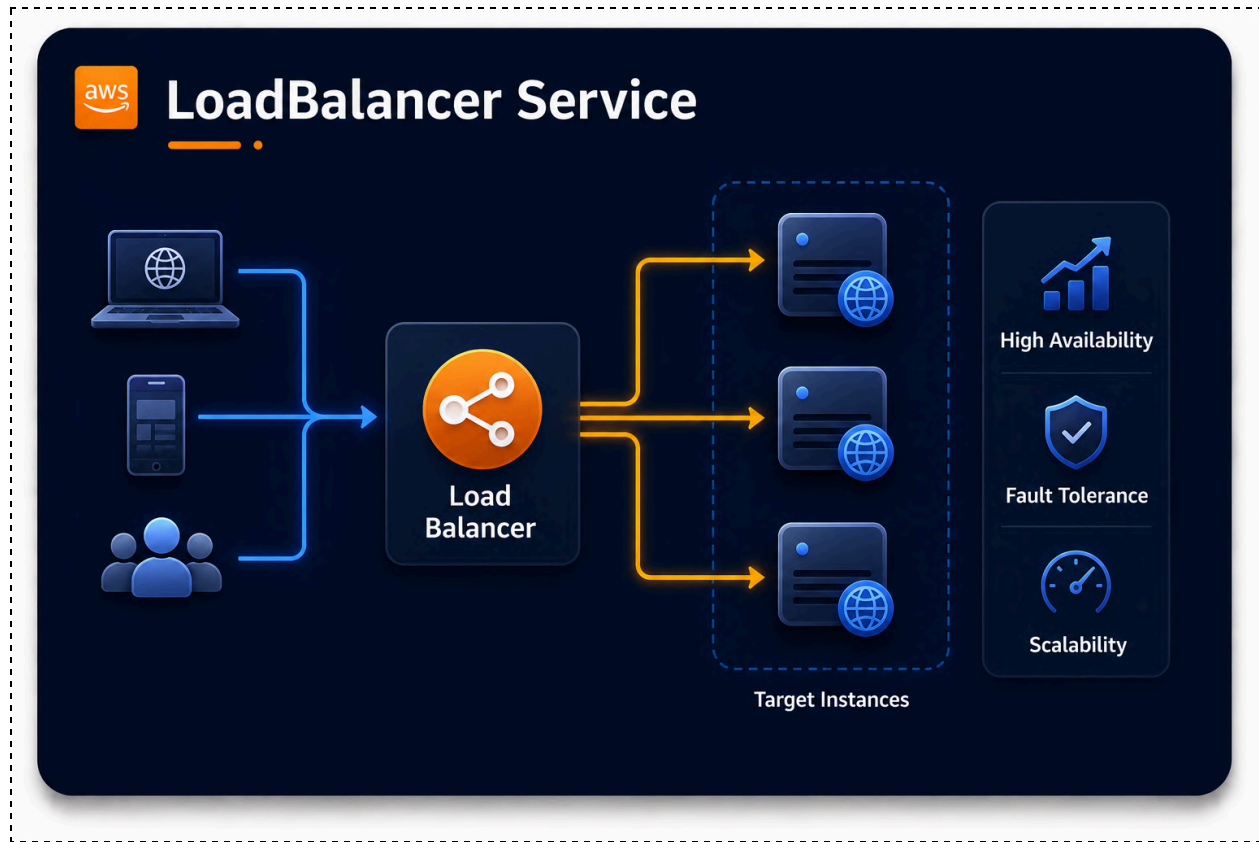
أحتاج أعمل Analytics على الـ Streaming Data عشان آخذ Action في وقت معين، زي:

- أبيع سهم.
- أشتري سهم.

Kinesis تقدر تعمل Analytics على الـ Real-Time Data وتستقبل Streaming Data.

Video 7 – LoadBalancer

Load Balancing Fundamentals



The Problem with a Single Server

لو عندي Server واحد بيستضيف Web Application، وكل الـ Users بيعملوا Access عليه، ممكن يحصل عطل في:

- RAM.
- CPU.
- Power.

لو الـ Server وقع، الـ Service كلها هتقع ومش هتكون متاحة للـ Users.

ممكن كمان الـ Server يفضل شغال، لكن الـ Load عليه يكون أكبر من قدرته، وده يسبب مشاكل في الـ Application.

Load Balancer Concept

لحل المشكلة، ممكن يكون عندي أكثر من Server، وقدامهم Load Balancer. الـ User بيتعامل مع الـ Load Balancer فقط، ويبشوف كل الـ Servers اللي وراه كأنهم Server واحد. الـ Load Balancer بيستقبل الـ Request، وبعد كده يوزعها على الـ Servers الموجودة وراه. لما الـ Server يرسل الـ Reply، الـ Load Balancer يرجعها للـ User. الـ Load Balancer بيوفر:

- Availability أعلى، لأن لو Server وقع، باقي الـ Servers تقدر تستقبل الـ Traffic.
 - توزيع الـ Load على أكثر من Server بدل ما كل الـ Traffic تروح إلى Server واحد.
 - واجهة واحدة للـ User بدل ما يتعامل مع كل Server بشكل منفصل.
-

Application Load Balancer

Application Load Balancer يشتغل على Application Layer أو Layer 7.

يستخدم مع HTTP Traffic، وبالتالي مناسب للـ Web Servers والـ Applications التي تستخدم HTTP.

لو الـ Servers الموجودة ورا الـ Load Balancer عبارة عن DNS Servers، مش تستخدم Application Load Balancer، لأنه مخصص للـ Application Traffic.

Target Groups

الـ Servers التي الـ Application Load Balancer يوزع عليها الـ Traffic بتكون موجودة داخل Target Group.

مثلاً ممكن أعمل Target Group تحتوي على:

- Server 1.
- Server 2.
- Server 3.

بعد كده أربط الـ Target Group بالـ Load Balancer.

الـ Load Balancer يستقبل الـ Requests ويوزعها على الـ Targets الموجودة داخل الـ Group.

Path-Based Routing

ممكن أعمل Routing بناءً على الـ URI أو الـ Path الموجود بعد اسم الـ Domain.

مثلاً ممكن يكون عندي Target Group أولى تحتوي على Server 1 و Server 2، و Target Group ثانية تحتوي على Server 3 و Server 4.

أقدر أعمل Rules بالشكل ده:

- لو الـ Request جاية على **users/**، تنبعت إلى Target Group الأولى.
- لو الـ Request جاية على **groups/**، تنبعت إلى Target Group الثانية.

الـ Routing هنا بيتحدد بناءً على الـ URI الموجود داخل الـ Request.

Host-Based Routing

ممکن کمان عمل Routing بناءً على الـ Hostname.

مثلاً:

- لو الـ Request جاية إلى **example1.com**، تتبعت إلى Target Group الأولى.
- لو الـ Request جاية إلى **example2.com**، تتبعت إلى Target Group الثانية.

بالشكل ده، نفس الـ Load Balancer يقدر يوجه الـ Traffic إلى Target Groups مختلفة بناءً على اسم الـ Host.

Health Checks

الـ Load Balancer بيععمل Health Checks على الـ Servers الموجودة وراه.

بيبعث Requests بشكل مستمر عشان يتأكد إن كل Server شغال.

لو Server وقع أو بقى Unhealthy، الـ Load Balancer يطلعه من الـ Target Group وما بيعتس له Traffic.

لو عندي أربع Servers وواحد منهم وقع، الـ Traffic تفضل تنوزع على الثلاثة الباقيين.

Client IP Address

لما الـ User بيعت Request إلى الـ Application Load Balancer، الـ Load Balancer يغير الـ Source IP قبل ما بيعت الـ Request إلى الـ Target.

الـ Server الموجود داخل الـ Target Group بيشف إن الـ Source IP هي الـ IP الخاصة بالـ Load Balancer، مش الـ IP الخاصة بالـ User.

الـ Destination IP كمان بتتغير من الـ IP الخاصة بالـ Load Balancer إلى الـ IP الخاصة بالـ Target Instance.

X-Forwarded-For

بما إن الـ IP الخاصة بالـ User بتختفي عند وصول الـ Request إلى الـ Server، ممكن أحاج طريقة أعرف بيها الـ Client IP الأصلية.

في الـ HTTP Field اسمه:

X-Forwarded-For

الـ Load Balancer يحيط داخله IP الخاصة بالـ User، وبالتالي الـ Application الموجودة على الـ Server تقدر تعرف الـ Client IP الأصلية.

دي معلومة موجودة على Layer 7، مش على Layer 3.

Target Types

الـ Target Group الخاصة بـ Application Load Balancer ممكن تحتوي على:

- EC2 Instances.
- IP Addresses.
- Lambda Function.

لو استخدمت EC2 Instances، لازم يكون عليها HTTP Service لأن Application Load Balancer بيستخدم HTTP Traffic.

Network Load Balancer

Network Load Balancer يشتغل على Layer 4.

يبتعامل مع:

- TCP.
- UDP.

هو مش بيحلل الـ Application Layer أو HTTP Fields.

بيستقبل Traffic على Port معينة، وبعد كده يوزعها على Targets موجودة وراه.

مثلاً يستقبل Traffic على Port 80 ويبعثها إلى Port 80 على الـ Target Instances.

Performance

Network Load Balancer أسرع من Application Load Balancer.

الـ Latency المذكورة كانت:

- Network Load Balancer: حوالي 100 Milliseconds.
- Application Load Balancer: حوالي 400 Milliseconds.

Static IP Address

Network Load Balancer بيكون عنده Static IP Address ثابتة.

ده مفيد في Whitelisting داخل الـ Firewalls.

لو الـ IP بتتغير، هحتاج أعدل Firewall Rules كل مرة، لكن مع Network Load Balancer أقدر أستخدم الـ Static IP مباشرة.

Target Types

الـ Target Group الخاصة بـ Network Load Balancer ممكن تحتوي على:

- EC2 Instances.
- Private IP Addresses.
- Application Load Balancer.

Gateway Load Balancer

Gateway Load Balancer يستخدم لتوزيع الـ Traffic على Virtual Appliances.

الـ Virtual Appliance يمكن تكون Firewall.

يمكن أحط أكثر من Firewall داخل Group، والـ Gateway Load Balancer يوزع الـ Traffic عليهم.

لما الـ Load يزيد، ممكن أضيف Virtual Appliances أكثر.

الـ Virtual Appliances بتتصل بالـ Gateway Load Balancer من خلال:

GENEVE Tunnel

Sticky Sessions

في الوضع العادي، Load Balancer يمكن يبعث كل Request إلى Server مختلفة.

مثلاً:

- أول Request تروح إلى EC2 Instance الأولى.
- ثاني Request تروح إلى EC2 Instance الثانية.
- ثالث Request ترجع إلى الأولى.

في بعض الـ Applications، محتاج كل الـ Requests الخاصة بنفس الـ Session تفضل تروح إلى نفس الـ Server.

مثلاً في Shopping Cart، ممكن تكون بيانات الـ Cart محفوظة على Server معينة.

لو Request جديدة من نفس الـ Session راحت إلى Server ثانية مش متزامنة مع الأولى، ممكن بيانات الـ Cart ماتكونش موجودة.

Session Cookies

Sticky Session بتستخدم Cookie داخل HTTP.

الـ Cookie بتكون مرتبطة بالـ Session.

كل الـ Requests الخاصة بنفس الـ Session بتتبع بنفس الـ Cookie.

الـ Load Balancer يحفظ إن الـ Cookie دي تم توجيهها إلى Server معينة، وبعد كده يفضل يبعث كل Traffic اللي جاية بنفس الـ Cookie إلى نفس الـ Server.

بالشكل ده، الـ Session تفضل مرتبطة بنفس الـ EC2 Instance من بدايتها لنهايتها.

مش كل الـ Applications بتحتاج Sticky Sessions، لكن بعض الـ Applications محتاجة كل الـ Requests الخاصة بالـ Session تروح إلى نفس الـ Server.

SSL Certificates

SSL Certificate يستخدم لتشفير Data على Application Layer أو Web Layer. Traffic بين Client والـ Load Balancer تفضل Encrypted، خصوصاً إن الجزء ده ممكن يعدي على الإنترنت.

Public and Private Keys

الـ Certificate بتعتمد على:

- Public Key.
- Private Key.

الـ Load Balancer يرسل الـ Public Key إلى الـ Client. الـ Client يستخدم الـ Public Key عشان يعمل Encryption للـ Traffic قبل ما بيعته. الـ Load Balancer بيكون عنده الـ Private Key، وبيستخدمها عشان يفك الـ Encryption. محدش يقدر يفك الـ Traffic غير الجهة اللي عندها الـ Private Key.

AWS Certificate Manager

AWS Certificate Manager أو ACM هي Service داخل AWS بتستخدم لإدارة الـ Certificates.

Server Name Indication

Server Name Indication أو SNI يستخدم لما يكون عندي أكثر من Website أو Hostname على نفس الـ Server أو الـ Load Balancer.

مثلاً ممكن يكون عندي:

- corp.com
- example.com
- Website تالته.

كل Website ليها Certificate مختلفة.

أثناء SSL Handshake، الـ Client بيحدد الـ Hostname اللي عايز يتصل بيه.

لو الـ Client طلب corp.com، الـ Load Balancer يختار Certificate الخاصة بـ corp.com.

لو طلب example.com، يختار Certificate الخاصة بـ example.com.

الـ Load Balancer بيختار أنسب Certificate بناءً على الـ Hostname الموجود أثناء SSL Handshake.

Auto Scaling Groups

Auto Scaling Group يستخدم عشان أعمل Scale In و Scale Out للـ EC2 Instances بشكل تلقائي.

لو عندي Load Balancer ووراه مجموعة EC2 Instances، ولما الـ Load يزيد، Auto Scaling Group تقدر تضيف Instances جديدة من غير تدخل يدوي.

لما الـ Load يقل، تقدر تشيل EC2 Instances مش محتاجها.

- إضافة Instances اسمها Scale Out.
- إزالة Instances اسمها Scale In.

استخدام Auto Scaling Group نفسها Free of Charge، لكن بدفع تكلفة الـ EC2 Instances اللي بتتضاف.

CloudWatch Alarms

ممكن أعمل Scaling بناءً على CloudWatch Alarm.

مثلاً لو CloudWatch أرسلت Alarm إن استخدام الـ CPU عالي، Auto Scaling Group تبدأ تضيف EC2 Instances جديدة.

الـ Load Balancer بعد كده يوزع الـ Traffic على العدد الجديد من الـ Instances.

Minimum Capacity

Minimum Capacity هي أقل عدد EC2 Instances لازم يفضل موجود داخل Auto Scaling Group.

مثلاً لو حددت Minimum يساوي 2، العدد ماينفعش يقل عن اثنين EC2 Instances.

Desired Capacity

Desired Capacity هي عدد الـ Instances اللي عايز Auto Scaling Group تحافظ عليه في الوضع العادي.

مثلاً:

- Minimum: 2 Instances.
- Desired: 4 Instances.

Auto Scaling Group تحاول تخلي العدد المعتاد أربع Instances، لكن ماتنزلش تحت اثنين.

Maximum Capacity

Maximum Capacity هي أكبر عدد EC2 Instances مسموح لـ Auto Scaling Group تضيفه.
مثلاً ممكن أحدد Maximum يساوي 10، وبالتالي حتى لو الـ Load زاد، عدد الـ Instances مايزدش عن عشرة.

Dynamic Scaling Policies

في ثلاث طرق المذكورة للتحكم في إضافة وإزالة الـ EC2 Instances:

- Target Tracking.
- Simple or Step Scaling.
- Scheduled Scaling.

Target Tracking

في Target Tracking بحد Target معين، و Auto Scaling Group تحاول تحافظ عليه.

مثلاً أحدد إن Average CPU Utilization تكون:

60%

Auto Scaling Group تضيف أو تشيل EC2 Instances عشان تحافظ على متوسط استخدام الـ CPU عند 60%.

عدد الـ Instances ممكن يكون اثنين أو أربعة أو ستة حسب الـ Load، لكن يفضل داخل حدود الـ Minimum والـ Maximum المحددة.

Simple or Step Scaling

في النوع ده، الـ Scaling بيحصل عند تشغيل Alarm معينة.

مثلاً أعمل Rule تقول:

- لو CPU زادت عن 70%، أضف EC2 Instance.

لما الـ Alarm تشتغل، Auto Scaling Group تضيف العدد المحدد من الـ Instances.

Scheduled Scaling

Scheduled Scaling بتستخدم لما أكون عارف وقت زيادة الـ Load بشكل مسبق.

مثلاً لو عارف إن يوم الجمعة من الساعة 4:00 إلى الساعة 8:00 بيكون فيه Rush Hour والـ Application عليها Traffic عالي، أقدر أعمل Schedule تضيف عشر EC2 Instances خلال الفترة دي.

Video 8 – VPC Networking

Amazon VPC

Amazon VPC هي Amazon Logical أو Virtual Container جوه AWS Account، وبحط جواها Resources الخاصة بيا.

من خلال الـ VPC بقدر أعمل Configuration لحاجات زي:

- Subnets.
- Route Tables.
- Network Components.

قبل ما أبني Network على AWS، لازم أعمل VPC.

Default and Custom VPC

قدامي اختياراتين:

- Default VPC.
- Custom VPC.

الـ Default VPC بتيجي معاها شوية Components معمولة أوتوماتيك.

لو عايز أعمل Customization وأكريت الـ Components بنفسي، بعمل VPC عادية بدل الـ Default VPC.

VPC CIDR Range

أثناء إنشاء VPC، يحدد CIDR Range خاص بيها.

مثال:

10.0.0.0/16

الـ CIDR ده هو الـ Network Range الخاص بالـ VPC.

كل الـ IP Addresses والـ Subnets الموجودة داخل الـ VPC لازم تطلع من الـ Range ده.

Subnets

الـ Subnet هي Range من الـ IP Addresses جوه الـ VPC.

ممکن أقسم الـ VPC CIDR إلى أكثر من Subnet.

مثلاً:

10.0.2.0/24

كل Subnet بتكون موجودة في Availability Zone واحدة.

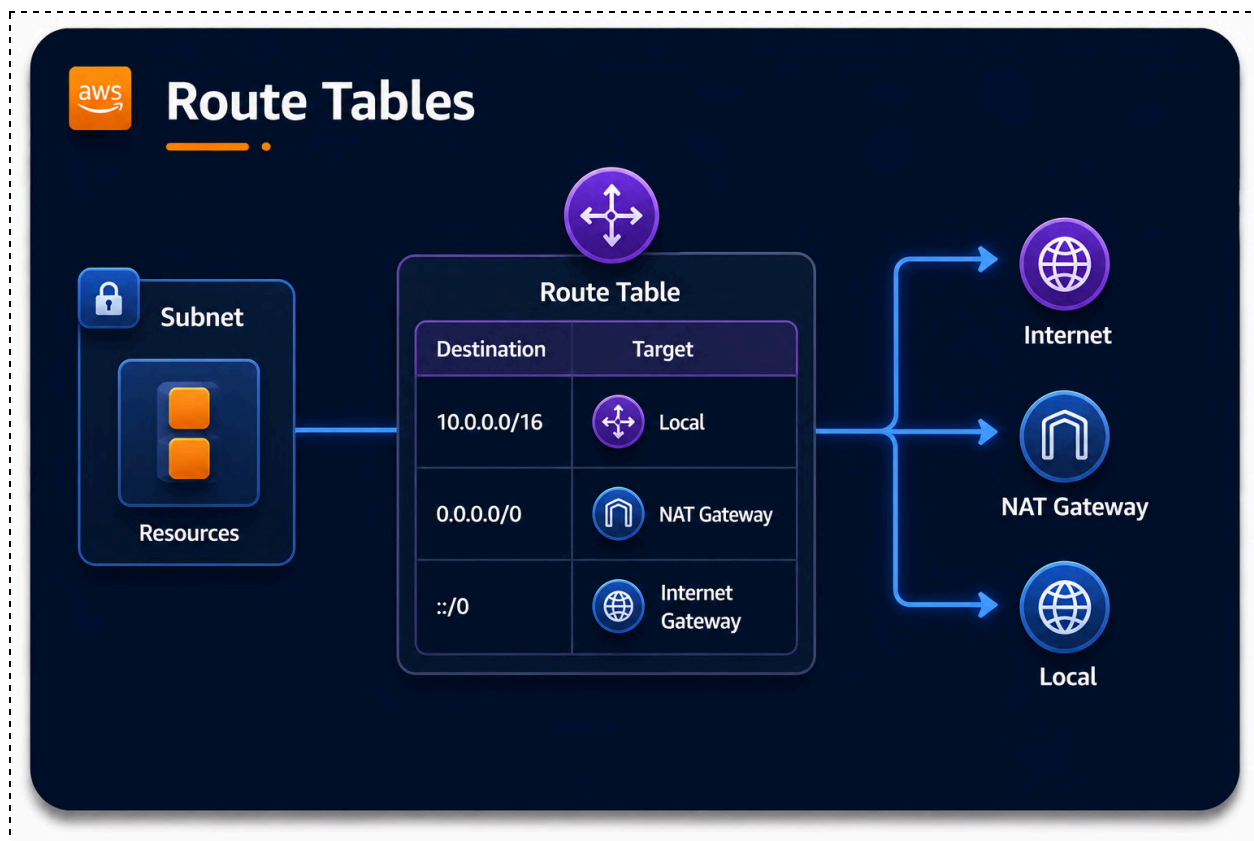
Availability Zone Distribution

ممکن أحط Subnet أولى في Availability Zone، و Subnet ثانية في Availability Zone مختلفة.

بعد كده أعمل EC2 Instance في الـ Subnet الأولى، و Backup EC2 Instance في الـ Subnet الثانية.

لو Availability Zone الأولى وقعت، الـ Resources الموجودة في Availability Zone الثانية تفضل شغالة.

Route Tables



الـ Route Table يحتوي على الـ Routes اللي بتحدد الـ Traffic هيتبع فين.

لما Packet توصل، الـ Route Table بتبص على Destination IP، وبعد كده تحدد الـ Next Hop المناسب.

مممكن يكون عندي Route للإنترنت أو Route إلى Network ثانية.

Route Table Association

كل Subnet بتكون مرتبطة بـ Route Table.

مممكن أعمل Route Table جديدة، وأسميها مثلاً X، وأحط جواها الـ Routes اللي محتاجها.

بعد كده أعمل Association بين الـ Route Table والـ Subnet.

أي Resource موجود جوه الـ Subnet بيستخدم الـ Route Table المرتبطة بيها عشان يعرف بيعت الـ Traffic فين.

Private Subnets

Private Subnet هي Subnet مالهش Direct Access إلى الإنترنت.

عشان Resources الموجودة فيها تخرج إلى الإنترنت، لازم تستخدم NAT Gateway.

مثلاً لو عندي Private Subnet بالـRange:

10.0.0.0/24

الـTraffic تروح من الـPrivate Subnet إلى NAT Gateway، وبعد كده NAT Gateway تبعها إلى الإنترنت.

Outbound Traffic

الـResources الموجودة داخل Private Subnet تقدر تبدأ Connection من جوه إلى الإنترنت.

لكن أي User من الإنترنت مايقدرش يبدأ Connection مباشرة إلى Resource موجودة داخل Private Subnet.

الـTraffic بيكون Initiated من جوه الـCloud إلى الإنترنت، مش من الإنترنت إلى الـPrivate Subnet.

Public Subnets

Public Subnet يستخدم Internet Gateway عشان تعمل Access إلى الإنترنت.

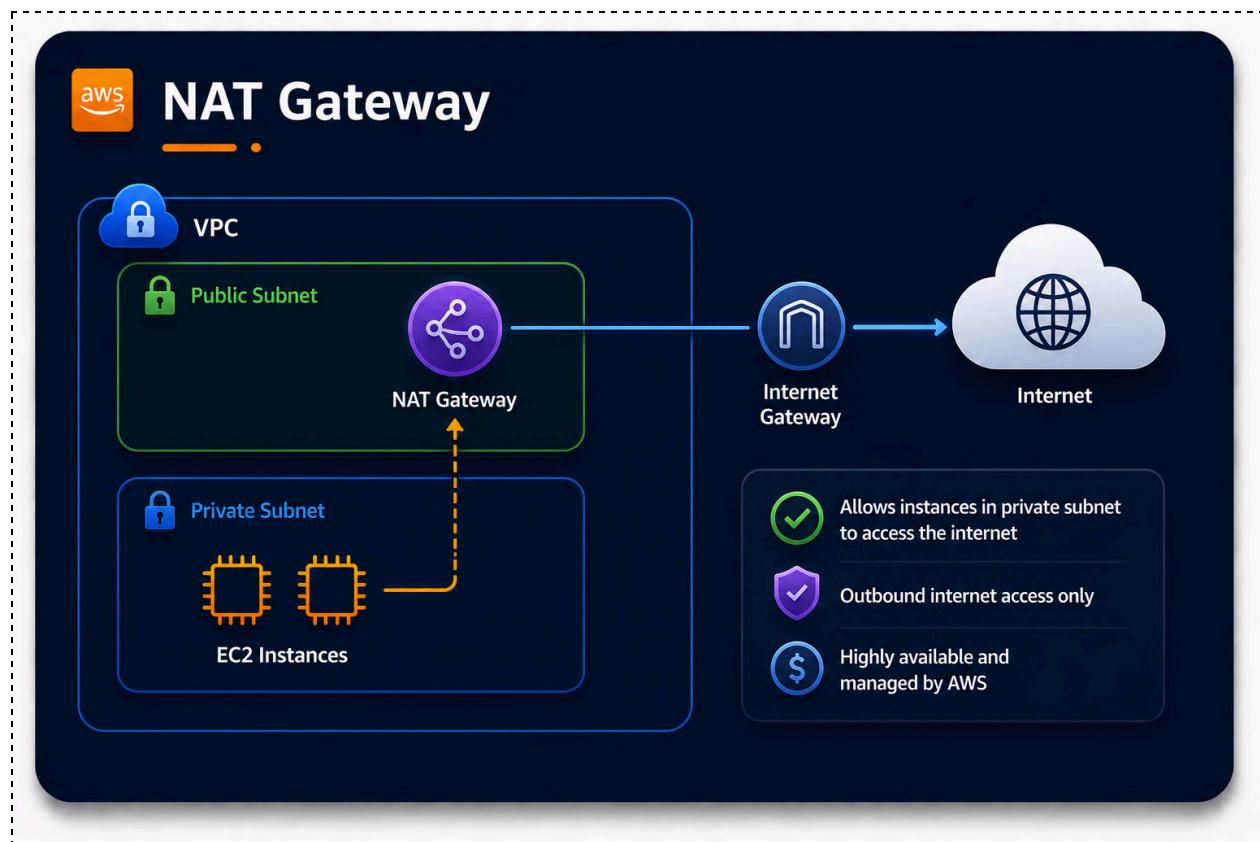
الفرق الأساسي:

- Private Subnet يستخدم NAT Gateway.
- Public Subnet يستخدم Internet Gateway.

الـ Public Subnet نفسها تستخدم Private IP Range برضه.

مصطلح Public Subnet هنا مش معناه إنها تستخدم Public IP Range، لكنه معناه إن الـ Route Table الخاصة بيها فيها Route إلى Internet Gateway.

NAT Gateway



NAT Gateway يستخدم مع الـ Private Subnets عشان تطلع إلى الإنترنت.

هي بتحول الـ Private Source IP إلى Public IP.

مثلاً لو Resource جوه Private Subnet عندها Private IP من Range زي:

10.0.0.0/24

لما تبعت Traffic إلى الإنترنت، NAT Gateway تغير الـ Source IP إلى Public IP، زي:

57.1.1.1

الإنترنت ببشوف إن الـ Source IP هي Public IP الخاصة بالـ NAT Gateway.

Private Subnet Routing

عشان الـ Private Subnet تستخدم NAT Gateway، أعمل Route Table وأضيف فيها Default Route.

0.0.0.0/0

الـ Next Hop يكون:

NAT Gateway

الـ Default Route معناها إن أي Traffic رايحة إلى Destination مش موجودة في Routes تانية، تتبعت إلى NAT Gateway.

مثلاً لو EC2 Instance بعنت Traffic إلى:

8.8.8.8

الـ Route Table تلاقي إن الـ Destination مطابقة للـ Default Route، فتبعت الـ Traffic إلى NAT Gateway.

بعد كده NAT Gateway تحول الـ Private Source IP إلى Public IP وتبعت الـ Traffic إلى الإنترنت.

Internet Gateway

Internet Gateway هي الـ Gateway التي يتوصل الـ VPC بالإنترنت.

يمكن اعتبارها زي الـ Router التي بيطلعني على الإنترنت.

أي Public Subnet محتاجة Route إلى Internet Gateway عشان الـ Resources الموجودة جواها تقدر تعمل Access إلى الإنترنت.

VPC Network Architecture

يمكن يكون عندي Region جواها VPC بالـ CIDR:

10.0.0.0/16

بعد كده أقسم الـ VPC إلى أربع Subnets:

Private Subnets

10.0.1.0/24, 10.0.2.0/24

Public Subnets

10.0.3.0/24, 10.0.4.0/24

كل الـ Subnets دي طالعة من الـ CIDR الأساسي الخاص بالـ VPC.

الـ Private Subnets تستخدم NAT Gateway للخروج إلى الإنترنت.

الـ Public Subnets تستخدم Internet Gateway.

Reserved Subnet IP Addresses

في كل Subnet، في خمس IP Addresses ماينفعش أستخدمهم.

Amazon بتحجز:

- أول أربع IP Addresses.
- آخر IP Address.

مثلاً في Subnet زي:

10.0.0.0/24

أول أربع IP Addresses وآخر IP Address مش متاحين للـ Resources.

أي IP Address بين القيم المحجوزة أقدر أستخدمها مع EC2 Instance أو Resource تانية.

Public EC2 Connectivity

لو عندي Web Server داخل Public Subnet، بياخد Private IP من Range الـ Subnet.

مثلاً:

10.0.3.5

أثناء إنشاء الـ EC2 Instance، أقدر أفعل:

Auto-assign Public IP

ساعتها الـ Instance يبقى عندها:

- Private IP.
- Public IP.

مثلاً الـ Public IP تكون:

61.1.1.1

Outbound Internet Access

لما الـ EC2 Instance تبع Traffic إلى الإنترنت، تستخدم الـ Public IP الخاصة بيها.
الـ Route Table الخاصة بالـ Public Subnet يكون فيها Default Route إلى Internet Gateway.
الـ Traffic تتبع مباشرة إلى Internet Gateway، وبعد كده إلى الإنترنت.

Inbound Internet Access

أي User على الإنترنت يقدر يعمل Access إلى الـ EC2 Instance باستخدام الـ Public IP الخاصة بيها.
الـ Traffic تدخل من Internet Gateway، وبعد كده توصل إلى الـ Instance الموجودة داخل الـ Public Subnet.

Video 9 – VPC Lab

AWS Network Lab

الترتيب الذي هشتغل بيه عشان أبني Network على AWS هو:

1. Create VPC.
2. Create Private Subnet.
3. Create Public Subnet.
4. Create Internet Gateway.
5. Create NAT Gateway.
6. Create Public Route Table.
7. Create Private Route Table.

8. Create EC2 Instance داخل الـ Public Subnet لاختبار الاتصال.

VPC Creation

أول خطوة هي إنشاء VPC وإضافة CIDR Range ليها.

في البداية استخدمت:

10.0.0.0/16

الـ CIDR ده هو الـ Range اللي هتطلع منه الـ Subnets.

أثناء التطبيق، الـ Console وضحت إن الـ CIDR لازم يكون بين / 16 و / 28، فتم استخدام:

10.0.0.0/16

بعد كده أنشأت الـ VPC باسم خاص بـ CloudKode.

Subnet Creation

بعد إنشاء الـ VPC، أنشأت Public Subnet و Private Subnet من نفس الـ VPC CIDR.

Private Subnet

استخدمت للـ Private Subnet الـ Range:

10.0.0.0/24

أثناء إنشاء الـ Subnet:

1. اخترت الـ VPC الخاصة بـ CloudKode.
2. كتبت اسم الـ Private Subnet.
3. أضفت الـ CIDR Range.
4. أنشأت الـ Subnet.

Public Subnet

استخدمت للـ Public Subnet الـ Range:

10.0.1.0/24

أثناء إنشاء الـ Subnet:

1. اخترت نفس الـ VPC.
2. كتبت اسم الـ Public Subnet.
3. أضفت الـ CIDR Range.
4. أنشأت الـ Subnet.

Internet Gateway

الـ Internet Gateway هي التي بتوصل الـ VPC بالإنترنت، ويمكن اعتبارها زي الـ Router اللي بيطلعني على الإنترنت.

عشان أعمل Internet Gateway:

1. فتحت **Internet Gateways**.
2. ضغطت على **Create internet gateway**.
3. سميتها باسم خاص بـ CloudKode.
4. أنشأت الـ Internet Gateway.
5. عملت لها Attach على الـ VPC الخاصة بـ CloudKode.

مجرد إنشاء الـ Internet Gateway مش كفاية، لازم تنربط بالـ VPC عشان تقدر تستخدمها.

NAT Gateway

الـ NAT Gateway بتستخدم عشان الـ Resources الموجودة داخل الـ Private Subnet تقدر تخرج إلى الإنترنت.

هي بتحول الـ Private IP إلى Public IP قبل ما تبعت الـ Traffic إلى الإنترنت.

الـ NAT Gateway لازم تتحت داخل الـ Public Subnet، لأن الـ Public Subnet هي اللي عندها Route إلى Internet Gateway. ومش محتاج تعمل Subnet منفصلة مخصوص للـ NAT Gateway.

عشان أعمل NAT Gateway:

1. فتحت **NAT Gateways**.
2. ضغطت على **Create NAT gateway**.
3. كتبت اسم خاص بـ CloudKode.
4. اخترت الـ Public Subnet.
5. عملت Allocate لـ Public IP.
6. أنشأت الـ NAT Gateway.

الـ Resources الموجودة داخل الـ Private Subnet تبعت الـ Traffic إلى الـ NAT Gateway، والـ NAT Gateway تعمل باستخدام الـ Public IP الخاصة بيها.

Public Route Table

بعد كده أنشأت Route Table خاصة بالـ Public Subnet.

الـ Default Route Table بتتعمل أوتوماتيك مع الـ VPC، لكن في التطبيق أنشأت Route Table جديدة بدل استخدامها.

Route Table Creation

1. فتحت **Route Tables**.
2. ضغطت على **Create route table**.
3. سميتها Public Route Table.
4. اخترت VPC الخاصة بـ CloudKode.
5. أنشأت الـ Route Table.

Subnet Association

بعد إنشاء الـ Route Table:

1. فتحت **Subnet associations**.
2. ضغطت على **Edit subnet associations**.
3. اخترت الـ Public Subnet.
4. حفظت الـ Association.

Internet Route

بعد كده أضفت Default Route:

0.0.0.0/0

وحددت الـ Target:

Internet Gateway

واخترت Internet Gateway الخاصة بـ CloudKode.

بكده أي Traffic خارجة من الـ Public Subnet إلى الإنترنت تتبع إلى Internet Gateway.

Private Route Table

بعد كده أنشأت Route Table خاصة بالPrivate Subnet.

Route Table Creation

1. فتحت **Route Tables**.
2. ضغطت على **Create route table**.
3. سميتها Private Route Table.
4. اخترت VPC الخاصة بـ CloudKode.
5. أنشأت الـ Route Table.

Subnet Association

بعد إنشاء الـ Route Table:

1. فتحت **Subnet associations**.
2. اخترت الـ Private Subnet.
3. حفظت الـ Association.

NAT Route

أضفت Default Route:

0.0.0.0/0

وحددت الـ Target:

NAT Gateway

واخترت NAT Gateway الخاصة بـ CloudKode.

بكده أي Resource موجودة داخل الـ Private Subnet وعازية تخرج إلى الإنترنت تبعت الـ Traffic الأول إلى الـ NAT Gateway.

Traffic Flow

Public Subnet Traffic

أي EC2 Instance موجودة داخل Public Subnet تقدر تخرج إلى الإنترنت عن طريق:

1. Public Route Table.
2. Internet Gateway.

لو ال Instance عندها Public IP، أي User من الإنترنت يقدر يبدأ Connection عليها.

Private Subnet Traffic

أي EC2 Instance موجودة داخل Private Subnet تخرج إلى الإنترنت عن طريق:

1. Private Route Table.
2. NAT Gateway.
3. Internet Gateway.

ال NAT Gateway بتسمح لل Private Instance إنها تبدأ Traffic إلى الإنترنت وتستقبل ال Reply.

لكن أي User من الإنترنت ما يقدرش يبدأ Connection مباشرة إلى ال Private Instance.

Public Web Server Test

بعد الانتهاء من إعدادات ال Network، أنشأت EC2 Instance لاختبار الاتصال.

Instance Configuration

أثناء إنشاء ال Instance:

1. سميتها Web Server.
2. اخترت Amazon Linux.
3. اخترت VPC الخاصة بـ CloudKode.
4. اخترت ال Public Subnet.
5. فعلت Auto-assign Public IP.
6. سمحت بال HTTP Traffic.
7. أضفت User Data لتشغيل Web Server.

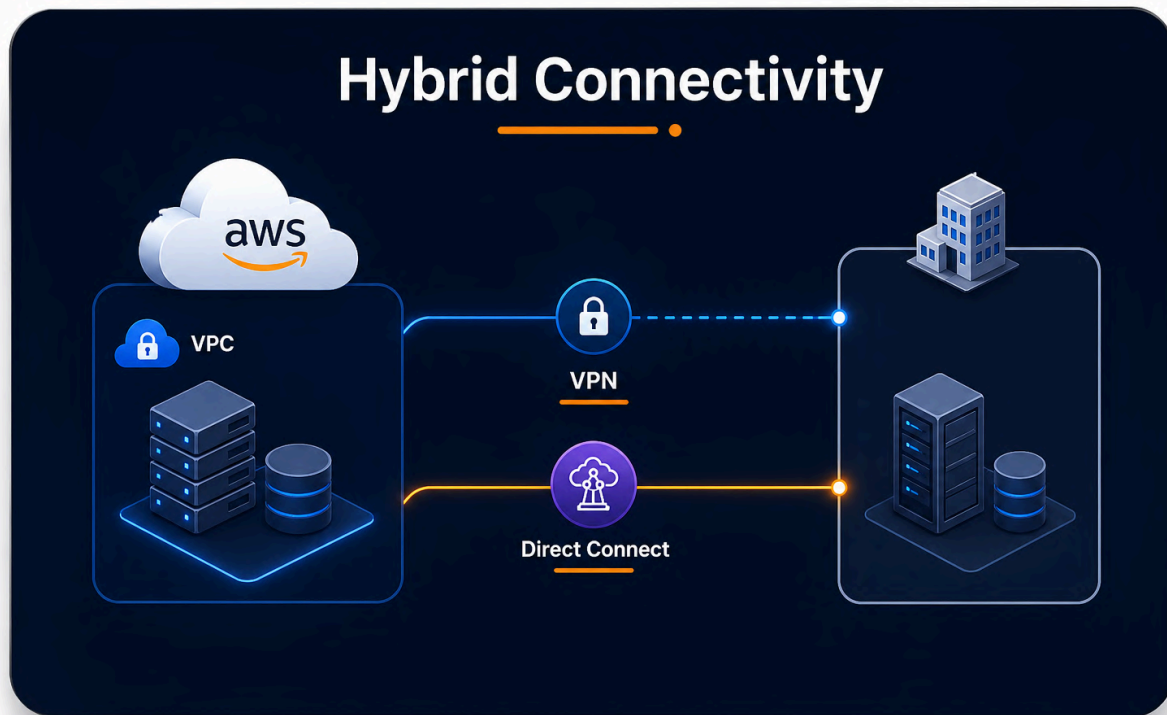
بعد تشغيل الـ Instance، استخدمت الـ Public IP الخاصة بيها من المتصفح.

بما إن الـ Instance موجودة داخل Public Subnet، وعندها Public IP، والـ Public Route Table، وتوجّه الـ Traffic إلى Internet Gateway، قدرت أوصل للـ Web Server من الإنترنت.

لو نفس الـ Instance كانت موجودة داخل Private Subnet، ماكنتش هقدر أبدأ Connection عليها من الإنترنت، لأنها مش هتكون عندها Public IP وموجودة ورا NAT Gateway.

Video 10 – Direct connect vs VPN

Hybrid Connectivity



لما تكون الـ Resources بتاعتي موجودة داخل VPC في AWS Region، وأكون محتاج أوصلها بـ On-Premises Site أو Servers موجودة عندي، عندي طريقتين:

- Site-to-Site VPN.
- AWS Direct Connect.

Site-to-Site VPN

Site-to-Site VPN يستخدم عشان أوصل الـ VPC الموجودة في AWS بالـ On-Premises Site.

Main Components

الاتصال بيعتمد على:

- Virtual Private Gateway ناحية AWS.
- Customer Gateway أو Customer Router ناحية الـ On-Premises.
- IPsec Tunnel بين الطرفين.

الـ Virtual Private Gateway هي النقطة الموجودة ناحية AWS اللي بتبني الـ Tunnel مع الـ Remote Site.

الـ Customer Gateway هي الـ Router أو الـ Gateway الموجودة عند الـ Customer في الـ On-Premises.

Traffic Flow

لو عندي EC2 Instance داخل الـ VPC وعازمة تتواصل مع Server موجود في الـ On-Premises، الـ Traffic بيتمشي بالشكل ده:

1. الـ EC2 Instance تبع الـ Traffic ناحية الـ Virtual Private Gateway.
2. الـ Virtual Private Gateway تاخد الـ Packet وتحطها داخل الـ IPsec Tunnel.
3. الـ Traffic تبقى Encrypted.
4. الـ Tunnel تعدي على الإنترنت.
5. الـ Traffic توصل إلى الـ Customer Gateway.
6. الـ Customer Gateway توصلها إلى الـ Server الموجود في الـ On-Premises.

بعد تشغيل الـ Tunnel، أقدر أوصل بين الـ Private IPs الموجودة في الـ VPC والـ Private IPs الموجودة في الـ On-Premises.

Deployment Time

من مميزات Site-to-Site VPN إنها ممكن تتعمل في دقائق.

من خلال AWS Console أقدر أعمل الـ Configuration للـ Tunnel بين الـ Virtual Private Gateway والـ Customer Gateway.

بمجرد ما الـ Tunnel تشتغل، الـ Connectivity بين الـ VPC والـ On-Premises تبقى متاحة.

Limitations

Site-to-Site VPN يستخدم الإنترنت، وده بيعمل شوية Limitations:

- ممكن يكون فيها High Latency.
- مقدرش أتحكم في Latency الموجودة على الإنترنت.
- الـ Speed محدودة إلى 1.5 Gbps

لو المطلوب في السيناريو إن الـ Connectivity تتعمل بسرعة وفي وقت قصير، Site-to-Site VPN بتكون الاختيار المناسب.

AWS Direct Connect

AWS Direct Connect بتستخدم عشان تعمل اتصال مباشر بين الـ On-Premises والـ VPC من غير ما يعتمد الاتصال الأساسي على الإنترنت.

Direct Connect Location

AWS بيكون عندها أماكن اسمها:

AWS Direct Connect Locations

داخل الـ Location دي بيكون فيه أجهزة خاصة بـ AWS تقدر توصلني بالـ VPC.

AWS بتوفرلي Port على جهازها، ولو وصلت الـ Router بتاعي بالـ Port دي، أبقي متصل بالـ VPC.

Connection Architecture

عشان أستخدم Direct Connect:

1. أجب الـ Router الخاص بيا.
2. أحطه داخل AWS Direct Connect Location.
3. أوصل Cable بين الـ Router والـ Port اللي AWS وفرتها.
4. أوصل الـ Router ده بالـ On-Premises Network.

بعد كده، الـ Servers الموجودة في الـ On-Premises تقدر توصل إلى الـ EC2 Instances الموجودة داخل الـ VPC من خلال Direct Connect.

Performance

Direct Connect مناسبة لما أحتاج:

- Low Latency
- Speeds عالية
- Connection مباشر بين On-Premises والـ VPC.

الـ Speeds المذكورة بتبدأ من 1 Gbps، ويمكن توصل إلى 10 Gbps و 100 Gbps.

لو السيناريو محتاج سرعة عالية أو Low Latency بين On-Premises والـ VPC، Direct Connect، بتكون الاختيار المناسب.

Provisioning Time

مشكلة Direct Connect إنها ممكن تاخذ من أيام لأسابيع عشان تتجهز.

لو الوقت مش عامل مهم، Direct Connect مناسبة.

لو محتاج Connectivity بسرعة، Site-to-Site VPN بتكون أسرع في التجهيز.

Direct Connect Connection Types

في نوعين من AWS Direct Connect:

- Dedicated Connection.
- Hosted Connection.

Dedicated Connection

في Dedicated Connection، أنا بجيب الـ Router الخاص بيا وأحطه داخل AWS Direct Connect Location.

بعد كده أوصله مباشرة بالـ Port اللي AWS بتوفرها.

الخطوات بتكون:

1. أجهز الـ Router الخاص بيا.
2. أحطه داخل الـ Direct Connect Location.
3. أوصله بالـ AWS Port.

4. أوصل الـ Router ده بالـ On-Premises Network.

Hosted Connection

في Hosted Connection، مش لازم أجيب الـ Router الخاص بيا وأحطه داخل الـ Direct Connect Location.

AWS Partner بيكون عنده جهاز موجود بالفعل داخل الـ Location.

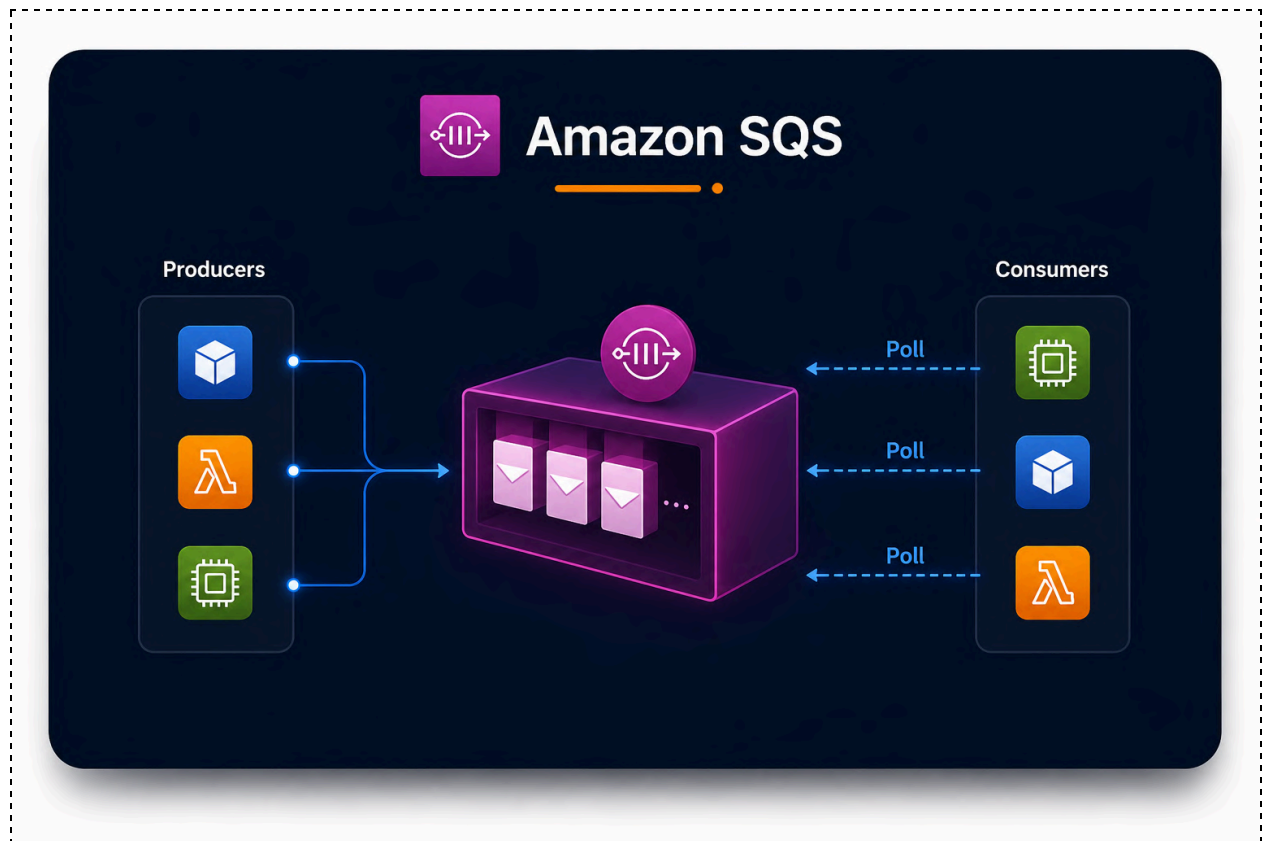
الـ Partner يقدر يوفّر لي Connection بداية من 50 Mbps، ومش لازم أبدأ من 1 Gbps.

في الحالة دي:

1. جهاز الـ Partner بيكون موجود داخل الـ Direct Connect Location.
 2. الـ Partner أو Service Provider يعمل وصلة بين جهازه والـ On-Premises Router بتاعي.
 3. أستخدم الـ Connectivity الخاصة بالـ Partner للوصول إلى AWS.
- بالشكل ده، مش محتاج أحط Hardware خاص بيا داخل الـ Direct Connect Location.
-

Video 11 – SQS,SNS, CloudWatch, CloudTrail, Eventbridge, Config

Amazon SQS



Core Concept

الـ Amazon SQS هي Simple Queue Service، وفكرتها الأساسية إنها عبارة عن Serverless Queue بتستخدمها علشان تعمل Decoupling بين الـ Applications أو الـ Components.

ببقي عندك:

- **Producers** بيعتوا الـ Messages للـ Queue.
- **SQS Queue** بتخزن الـ Messages.
- **Consumers** بتعمل Poll على الـ Queue وتسحب الـ Messages منها.

الـ SQS مش هي اللي بتتبع الـ Message للـ Consumer. الـ Consumer نفسه هو اللي ببيجي يبص على الـ Queue ويعمل Poll ويسحب الـ Message.

Application Decoupling

الفكرة إن الـ Producer والـ Consumer مايقوش معتمدين على بعض بشكل مباشر. الـ Producer ببيعت الـ Message للـ SQS Queue، والـ Consumer بعد كده يقدر يسحبها من الـ Queue. ده بي فصل الـ Application اللي ببيعت الـ Messages عن الـ Application أو الـ Component اللي بيستقبلها. بالتالي:

- الاتنين ممكن يعملوا Scale بشكل مختلف عن بعض.
 - لو حصلت مشكلة في الـ Producer، الـ Message نفسها مش بتضيع لأنها بتفضل Stored في الـ SQS.
 - الـ Consumer يقدر يسحب الـ Message بعد كده من الـ Queue.
- الـ SQS يقدر يعمل Scale لعدد كبير جدًا من الـ Messages في الثانية ويوصل لآلاف الـ Messages في الثانية.

Producer and Consumer Access

بعد ما تعمل Configure للـ SQS:

1. بتحدد مين الـ Producer اللي مسموح له بيعت الـ Messages للـ Queue.
2. بتحدد مين الـ Consumer اللي مسموح له يعمل Poll ويسحب الـ Messages من الـ Queue.

Message Retention

الـ Message اللي بتتبع للـ SQS بتفضل Stored في الـ Queue لو مفيش Consumer سحبها. الـ Default Retention Period هو 4 أيام، وممكن توصل لحد 14 يوم. يعني لو الـ Producer بعث Message ومفيش Consumer سحبها، ممكن تفضل موجودة في الـ Queue لمدة 4 أيام بالـ Default.

Visibility Timeout

لما الـ Consumer يعمل Poll ويسحب Message من الـ Queue، الـ Message بتدخل في **Hidden State**.

خلال الفترة دي مفيش Consumer ثاني يقدر يشوف أو يسحب نفس الـ Message.

الفترة دي اسمها **Visibility Timeout**.

مثلاً لو الـ Visibility Timeout هو **30 ثانية**:

- الـ Consumer يسحب الـ Message.
- الـ Message تفضل Hidden لمدة 30 ثانية.
- مفيش Consumer تاني يقدر يسحبها خلال الوقت ده.
- الـ Consumer يقرأ ويعمل الـ Message Processing.

بعد ما الـ Consumer يخلص الـ Processing، هو المسؤول إنه يعمل **Delete** للـ Message بنفسه.

الـ SQS مش بيعمل Delete للـ Message تلقائياً بمجرد ما الـ Consumer يسحبها.

Creating an SQS Queue

من الـ AWS Console:

1. اختار الـ Region.
2. ابحث عن **SQS**.
3. افتح **Simple Queue Service**.
4. اختار **Create Queue**.

Standard and FIFO Queues

فيه Standard Queue وفيه FIFO Queue.

FIFO Queue

FIFO معناها **First In, First Out**.

مثال على استخدامها لو عندك Financial Transactions وعايز تمنع وجود Duplicate Messages.

الـ FIFO بيمنع Duplicate Messages، لكن بيبقى Limited في الـ Throughput.

Standard Queue

لو محتاج Throughput أعلى وعدد Messages أكبر في الثانية، الـ Standard ممكن تكون أنسب.

في الـ Lab استخدمت **Standard Queue**.

Queue Configuration

سميت الـQueue:

CloudKode Queue

ومن الإعدادات:

- **Visibility Timeout:** 30 seconds.
- **Message Retention Period:** 4 days Default.
- **Encryption:** Disabled.

كمان تقدر تحدد:

- مين يقدر يبعث Messages، وده الـProducer.
- مين يقدر يستقبل Messages، وده الـConsumer.

بعدها تعمل **Create Queue**.

Sending a Message

عملت Test بسيط باستخدام الـAccount نفسه كـProducer.

بعث Message:

Hello From CloudKode

بعد ما عملت Send، الـMessage بقت موجودة في الـQueue.

لو فتحت الـQueue هتلاقي إن فيه Message Available.

Polling a Message

من **Send and Receive Messages** عملت:

Poll for Messages

الـ SQS رجع الـ Message:

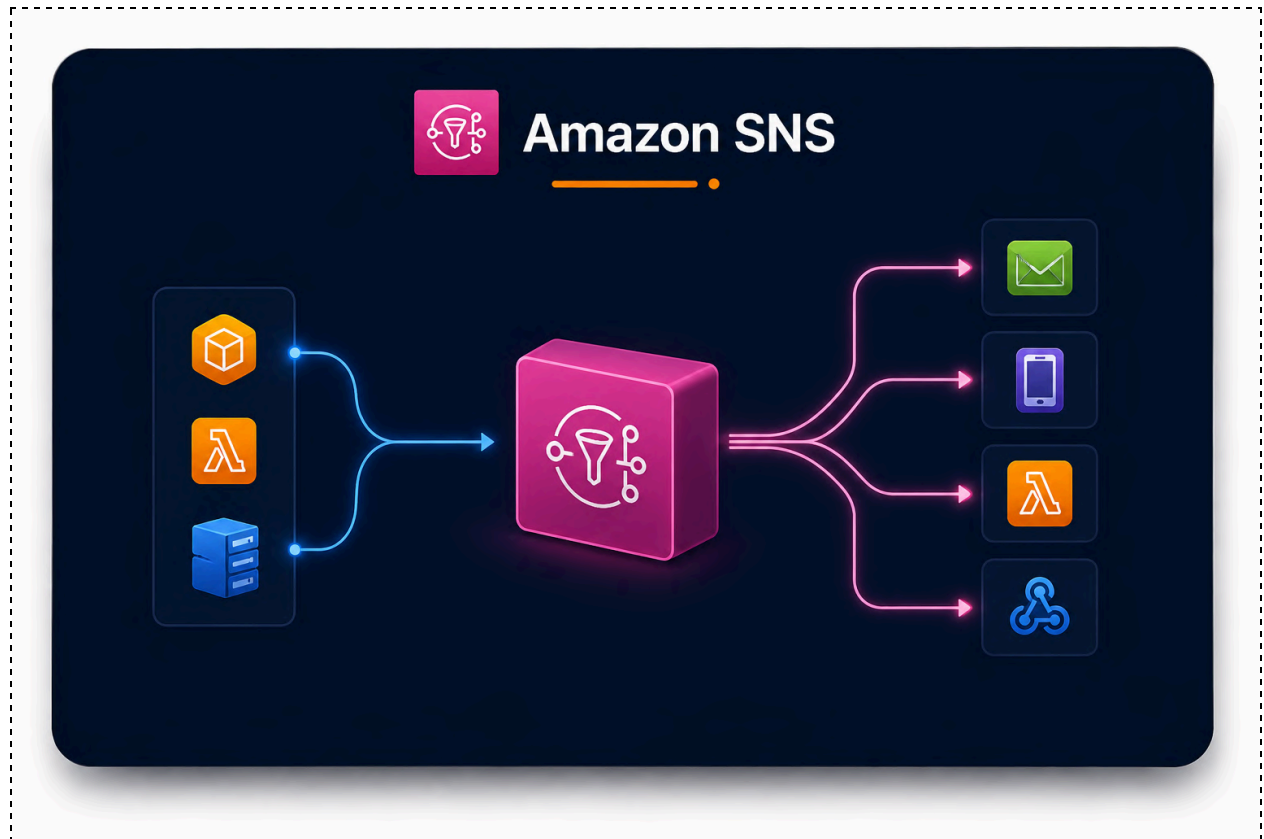
Hello From CloudKode

الـ Consumer المفروض يقرأ الـ Message ويعمل Processing ليها.

خلال الـ 30 ثانية بتوع الـ Visibility Timeout، مفيش Consumer ثاني يقدر يشوف نفس الـ Message.

بعد ما الـ Processing يخلص، لو مش عايز أي Consumer ثاني يقرأها، تعمل لها **Delete**.

Amazon SNS



Core Concept

الـ **Amazon SNS** هي **Simple Notification Service**.

الفكرة هنا مختلفة عن SQS.

في SQS الـ Consumer بيروح يعمل Poll ويشوف هل فيه Message تخصه ويسحبها.

لكن في SNS الـ Message بتتبع لكل الـ Receivers أو الـ Subscribers المرتبطين بالـ Topic.

الـ Architecture هنا اسمها:

Pub/Sub أو **Publisher/Subscriber**.

Publisher and Subscribers

ببقي عندك:

- **Publisher** يبعث الـ Message.
- **SNS Topic** الـ Message بتتبع عليه.
- **Subscribers** مرتبطين بالـ Topic.

لما الـ Publisher يبعث Message للـ Topic، الـ Message بتتبع تلقائياً لكل الـ Subscribers الموجودين على الـ Topic.

Subscription Types

الـ Subscriber ممكن يكون أكثر من نوع، زي:

- Lambda.
- Email.
- SQS.
- Mobile SMS.

SNS Architecture

الـ Flow بيكون:

1. الـ Publisher يبعث الـ Message.
2. الـ Message تدخل الـ SNS Topic.
3. الـ Topic يكون Connected بأكثر من Subscriber.
4. كل الـ Subscribers الموجودين على الـ Topic يستقبلوا الـ Message.

S3 Notification Example

ممكن يكون الـ Publisher هو **S3**.

مثلاً لو عايز أي File يترفع على S3 Bucket يوصلك عنه Email:

1. تعمل الـ S3 Configure للـ Publisher.
2. لما File يترفع على الـ Bucket، الـ S3 تبعت الـ Event للـ SNS Topic.
3. الـ Event تقول إن File معين اترفع على الـ Bucket.
4. تعمل الـ Email Address Subscriber للـ Topic.
5. الـ SNS Topic يبعث الـ Notification للإيميل.

Lambda Subscriber Example

يمكن كمان تستخدم Lambda كSubscriber.

مثلاً لو عايز تعمل Resize للصور اللي بتترفع على S3 علشان ماتاخدش مساحة كبيرة:

1. Image تترفع على S3.
2. الـ S3 تبعث Message للـ SNS Topic.
3. الـ Topic بيعتبع الـ Message للـ Lambda Subscriber.
4. الـ Lambda Function تتعمل لها Trigger.
5. الـ Function تعمل Resize للصور.

Creating an SNS Topic

من الـ AWS Console:

1. ابحث عن **SNS**.
2. افتح **Simple Notification Service**.
3. اختار **Create Topic**.
4. سميت الـ Topic:

CloudCode Topic

بعدها تعمل Create للـ Topic.

Creating an Email Subscription

بعد إنشاء الـ Topic، الخطوة اللي بعدها إنك تعمل Subscription.

اخترت إن الـ Subscriber يكون **Email**.

بعدها:

1. حطيت الـ Email Address.
2. عملت **Create Subscription**.
3. حالة الـ Subscription ظهرت **Pending Confirmation**.
4. وصل الـ Email فيه طلب تأكيد.
5. عملت **Confirm Subscription**.

بعد الـ Confirmation، الـ Email Address بقى Subscriber على الـ Topic.

Publishing a Message

عملت Publish لـ Message على الـ Topic.

الـ Message كانت:

Hello From CloudKode Topic

بمجرد ما عملت Publish، الـ Message اتبعنت لكل الـ Subscribers الموجودين على الـ Topic.

في الحالة دي كان عندي Subscriber واحد بس، وهو الـ Email Address.

بعدها وصل Email فيه الـ Message:

Hello From CloudKode Topic

Amazon CloudWatch



CloudWatch Monitoring

أول Tool في الـ Monitoring هي **CloudWatch**.

CloudWatch تقدر تجمع:

- Metrics.
- Logs.
- Alarms.

Metrics

الـ Metrics بتساعدك تتابع Performance بتاعة الـ Resources.

مثلاً مع EC2 Instance تقدر تشوف:

- CPU.
- Status.
- Network Traffic داخل وخارج الـ Instance.

وتقدر تحدد الفترة اللي عايز تشوف فيها البيانات، زي:

- آخر ساعة.
- آخر نصف ساعة.

بالـ Default الـ Metrics بتتبع من الـ Service لـ CloudWatch كل 5 دقائق.

Resource Metrics Examples

بالنسبة لـ EBS Volume تقدر تشوف:

- عدد الـ Reads.
- عدد الـ Writes على الـ Disk.

بالنسبة لـ S3 Bucket تقدر تشوف:

- Storage Size.
- عدد الـ Objects اللي اترفعت على S3.

بالنسبة لـ ALB أو الـ Load Balancer تقدر تتابع:

- عدد الـ Requests.
- الـ Response Codes اللي رجعت من الـ Backend.

مثلاً:

200
400
500

Logs

CloudWatch تقدر تجمع Logs من Services مختلفة، زي:

- ECS.
- Lambda.
- ALB.
- EC2 Instance.

لكن علشان تجمع Logs من EC2 Instance، محتاج تحط **CloudWatch Agent** على الـ Instance.

الـ Agent يقدر يجمع Logs من الـ Machine لـ CloudWatch، زي:

- Login Events.
- Failed Login.
- Logs خاصة بالـ Machine نفسها.
- Linux System Logs.

Logs بتاعة EC2 مش بتتبع لـ CloudWatch بالشكل ده بالـ Default، ومحتاج تعمل Configuration وتحط CloudWatch Agent على الـ Instance.

Alarms

بعد الـ Metrics والـ Logs، فيه **CloudWatch Alarms**.

فكرة الـ Alarm إنك تحدد Condition على Metric معينة.

لما الـ Condition تتحقق، الـ Alarm يقدر:

- يبعث Notification.
- يعمل Action.

CPU Alarm Example

مثلاً نقول:

لو CPU على EC2 Instance زاد عن **80%**، ابعتلي Email.

ممكن الـ Alarm يبعث Message لـ SNS Topic، والـ Topic يكون مربوط بـ Email Address، وبالتالي يوصلك Email إن CPU بتاعة الـ EC2 Instance زادت عن 80%.

Auto Scaling Action

يمكن بدل Notification تعمل Action.

مثلاً لو CPU زاد عن 80%، ممكن تزود EC2 Instance ثانية في Auto Scaling Group.

EC2 Reboot Action

ممكن كمان تعمل Action إنك تعمل Reboot للـ EC2 Instance لما CPU يوصل لأكثر من 80%.

الفكرة إن ممكن يكون فيه حاجة على الـ Instance مسببة إن CPU تكون عالية، ولما تعمل Reboot ممكن الـ CPU تتعدل بعد ما الـ System يقوم.

بالتالي الـ Alarm بيشتغل Based on Threshold معين على Metric معينة، وبعدها ينفذ Action.

الـ Action ممكن يكون:

- Notification.
- Stop للـ EC2 Instance.
- Reboot للـ EC2 Instance.
- Actions ثانية متاحة.

CloudWatch Metrics Lab

Viewing EC2 Metrics

كان عندي EC2 Instance علشان أعمل عليها Test للـ CloudWatch.

من CloudWatch فيه Tabs للـ:

- Alarms.
- Logs.
- Metrics.

دخلت على **All Metrics**.

بعدها اخترت:

1. EC2.
2. Per-Instance Metrics.

ظهرت الـ Metrics المرتبطة بالـ Instance، زي:

- Network In.
- Network Out.
- CPU-related Metrics.

اخترت CPU علشان أشوف الـ CPU Utilization.

CPU Utilization Graph

بعد اختيار الـ CPU Metric، ظهر Graph بيبين CPU كانت عاملة إزاي على الـ EC2 Instance.

تقدر تغير الفترة الزمنية وتعرض مثلاً:

- آخر ساعة.
- آخر 3 ساعات.
- آخر 12 ساعة.

Metrics Explorer

تقدر تستخدم الـ Explorer لو عارف بالظبط الـ Metric والـ Resource اللي عايز تدور عليهم.

مثلاً:

- تختار CPU Utilization.
- تحدد Instance ID معينة.

بعدها يظهر نفس الـ Metric على الـ Graph.

CloudWatch Dashboard

Creating a Dashboard

تقدر تضيف الـ Metrics اللي بتهمك لـ Dashboard.

الفكرة إن بدل ما كل مرة تدخل وتدور يدويًا على الـ Resource والـ Metric، تخلي الـ Critical Resources والـ Metrics المهمة ظاهرة قدامك أول ما تفتح الـ Dashboard.

عملت Dashboard باسم:

CloudKode Dashboard

وضفت فيها CPU Utilization الخاصة بالـ EC2 Instance.

بعدها لما فتحت الـ Dashboard، الـ CPU Utilization كانت ظاهرة مباشرة.

تقدر كمان تغير اسم الـ Widget، مثلاً:

CloudKode EC2 CPU

CloudWatch Logs Lab

Sending EC2 Logs to CloudWatch

بعد ما شفت إزاي أتابع Metrics، بدأت أبعث Logs من EC2 Instance إلى CloudWatch Logs.

على Linux Server الموجود على الـ EC2 Instance فيه Logs محلية خاصة بالـ System، زي:

- Login.
- Logout.
- Kernel.
- CPU.
- System-related Logs.

الـ Logs دي بتتسجل على الـ Local Machine.

تقدر تبعثها لـ CloudWatch، لكن ده مش بيتم بالـ Default.

محتاج تحط **CloudWatch Agent** على الـ EC2 Instance.

CloudWatch Agent Configuration

الهدف هنا كان توضيح الفكرة بسرعة، لأن تفاصيل الـ Configuration نفسها مش ضمن Scope الكورس.

بتعمل Configuration File للـ CloudWatch Agent.

اسم الـ File غير واضح بالكامل في الـ Transcript، لكنه JSON Configuration File.

جواه بتحدد الـ Files أو الـ Paths اللي عايز تبعت Logs منها.

الفكرة إنك تقول للـ Agent:

- خذ الـ Logs الموجودة في Path معين على الـ EC2 Instance.
- ابعثها لـ Log Group معين على CloudWatch.
- استخدم Instance ID كـ Log Stream Name.

Log Groups

عملت Log Group باسم:

System Log

الـ EC2 Instance تبعت الـ Logs الموجودة في الـ Path المحدد للـ Log Group ده على CloudWatch.

Log Streams

داخل الـ Log Group ممكن يبقى فيه أكثر من **Log Stream**.

استخدمت الـ Instance ID كـ Log Stream Name.

بالتالي لو أكثر من EC2 Instance بتبعت Logs لنفس الـ Log Group، كل Instance يبقى عندها Log Stream باسم الـ Instance ID بتاعها.

Viewing EC2 Logs

عملت Test Log على الـ EC2 Instance باسم:

Cloud Test

بعد ما الـ Agent بعث الـ Logs لـ CloudWatch، دخلت على الـ Log Group ولقيت Log Stream فيها الـ Logs الخاصة بالـ EC2 Instance.

ومن ضمنها ظهر الـ Log:

Cloud Test

بالتالي الـ Logs الموجودة على الـ EC2 Instance اتبعثت واتخزنت في الـ Log Group على CloudWatch.

CloudWatch Alarms Lab

Creating a CPU Alarm

الجزء الأخير كان إنشاء Alarm.

اخترت Metric خاصة بالـ CPU للـ EC2 Instance.

الـ Condition كانت إن CPU تكون:

Greater than 80%

يعني لو CPU Utilization تعدي 80%، الـ Alarm يتفعل.

Alarm Action

بعد تحديد الـ Condition، بتحدد الـ Action اللي يحصل لما الـ Alarm يتفعل.

ممکن مثلاً:

- تبعت Email.
- تعمل Reboot للـ EC2 Instance.
- تستخدم Actions ثانية.

في الـ Lab استخدمت SNS Topic علشان تبعت Notification.

SNS Notification Flow

الـ Flow كان:

1. CPU تزيد عن 80%.
2. CloudWatch Alarm يدخل Alarm State.
3. الـ Alarm يبعت Message للـ SNS Topic.
4. الـ SNS Topic يكون مربوط بـ Email Address.
5. يوصل Email فيه Notification إن CPU الخاصة بالـ Instance عدت 80%.

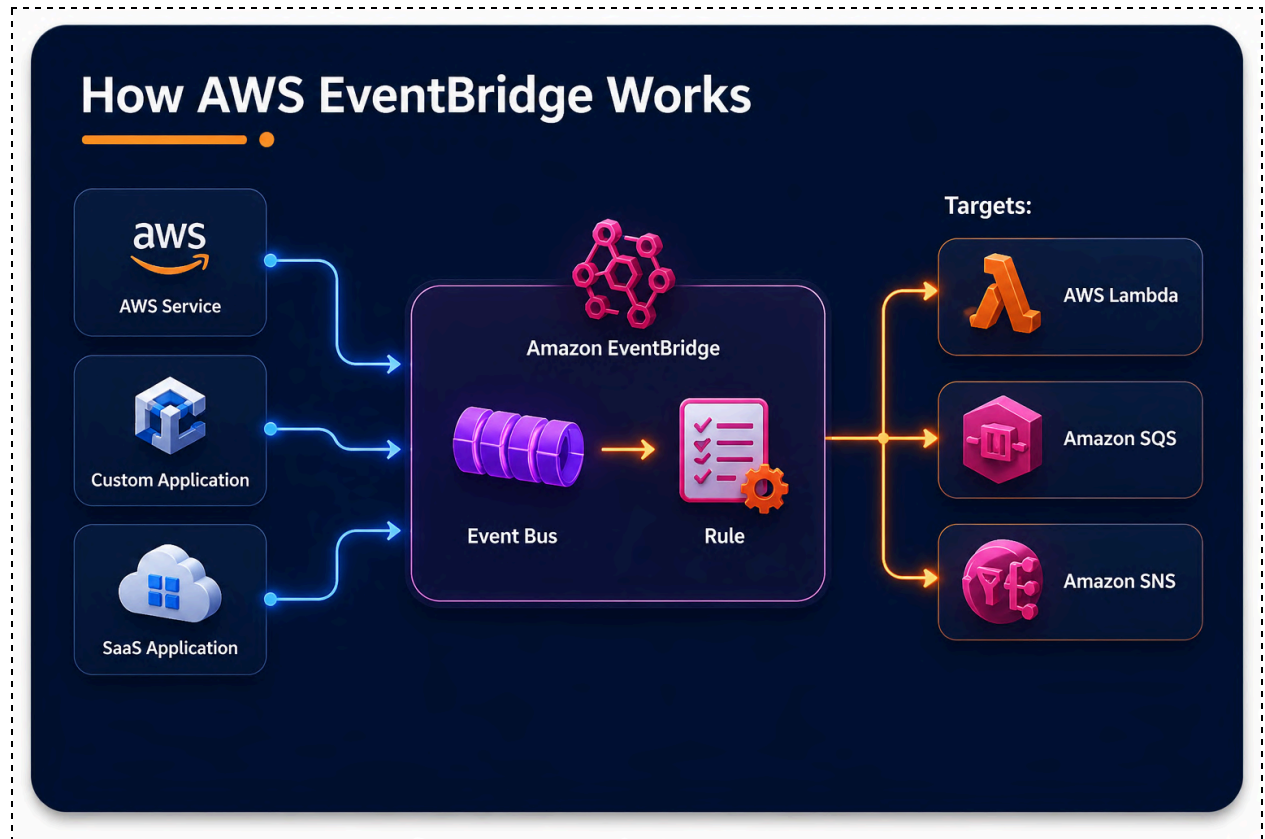
عملت Topic باسم قريب من:

Default CloudWatch Alarm

وبعدين ربطته بالـ Email Address المستخدم في التجربة.

بالتالي لو CPU تعدي 80%، الـ Alarm يبعث Message للـ Topic، والـ Topic يبعث Email
بالـ Notification.

Amazon EventBridge



Core Concept

الـ Amazon EventBridge تعتبر امتداد لفكرة كانت موجودة تحت CloudWatch، وبعد كده بقت Service مستقلة باسم **EventBridge**.

الـ EventBridge بتنشتغل بشكل أساسي مع الـ Events والـ State Transitions.

Scheduled Jobs

من الحاجات اللي تقدر تعملها باستخدام EventBridge إنك تعمل **Scheduled Job**.

مثلاً:

تقدر تقول إنك عايز كل ساعة تعمل Trigger لـ Lambda Function معينة.

في الحالة دي مش محتاج Alarm أو Event يحصل الأول علشان الـ Service تتعمل لها Trigger، لكن بتحدد Schedule معين والـ Job تشتغل بناءً عليه.

Event-Based Actions

تقدر كمان تعمل Action بناءً على Event أو State Transition.
الفرق هنا عن CloudWatch إن في CloudWatch كنا بنبص على Metric.
مثلاً:

لو CPU زادت عن 80%:

- ابعت Email.
- Trigger Function معينة.
- اعمل Reboot للـ EC2 Instance.
- أو نفذ Action معين.

لكن مع EventBridge بنركز أكثر على **State Changes** أو **Events**.
مثلاً لو EC2 Instance حصل لها Stop، تقدر تخلي EventBridge يبعث Notification.
يعني هنا بدل ما تراقب Metric زي CPU، بتراقب إن State نفسها اتغيرت.

EC2 State Change Example

ممکن تعمل Rule تقول:

لو EC2 Instance حصل لها **Stop**، ابعتلي Email.

وممكن كمان تتابع States تانية، زي:

- Stop.
- Terminate.
- Running.

S3 Event Example

ممکن كمان تستخدم EventBridge لما يحصل Event على Service تانية.
مثلاً:

لو File اترفع على S3، اعمل Trigger لـ Lambda Function معينة.

Root User Sign-In Example

ممکن تعمل Event تقول:

لو Root User عمل Sign In على الـ AWS Account، ابعثلي Email.

الفكرة هنا إنك عايز تعرف لو أي حد دخل باستخدام الـ Root Account.

EventBridge and CloudWatch Difference

CloudWatch بتركز أكثر على **Metrics**.

مثلاً:

CPU أعلى من 80%.

أما EventBridge فبتركز أكثر على:

- Events.
- State Transitions.
- Actions حصلت على الـ Resources.

EventBridge Lab

Creating a Rule

من الـAWS Console:

1. ابحث عن **EventBridge**.
2. افتح EventBridge.
3. من الشمال اختار **Rules**.
4. اعمل Create Rule.

الـRule هي المكان الذي بتحدد فيه الـCondition أو الـEvent الذي عايز تتابعه.

Scheduled Rule

ممكن تعمل Rule على Schedule معين.

مثلاً:

كل ساعة Trigger الـLambda Function معينة.

EC2 Stop Event

في المثال استخدمت Event متعلق بـEC2 Instance.

حددت إن EventBridge يراقب الـEC2 Instance ولما يحصل لها **Stop** ينفذ Action.

ممكن تختار State معينة زي:

- Stopped.
- Terminated.
- Running.

Selecting the Target

بعد ما تحدد الـ Event، بتحدد الـ Target اللي هيستقبل الحدث.

ممکن يكون:

- Lambda Function
- SNS
- Targets ثانية حسب الـ Action اللي عايز تنفذه.

في المثال استخدمت **SNS** علشان يتبع Email.

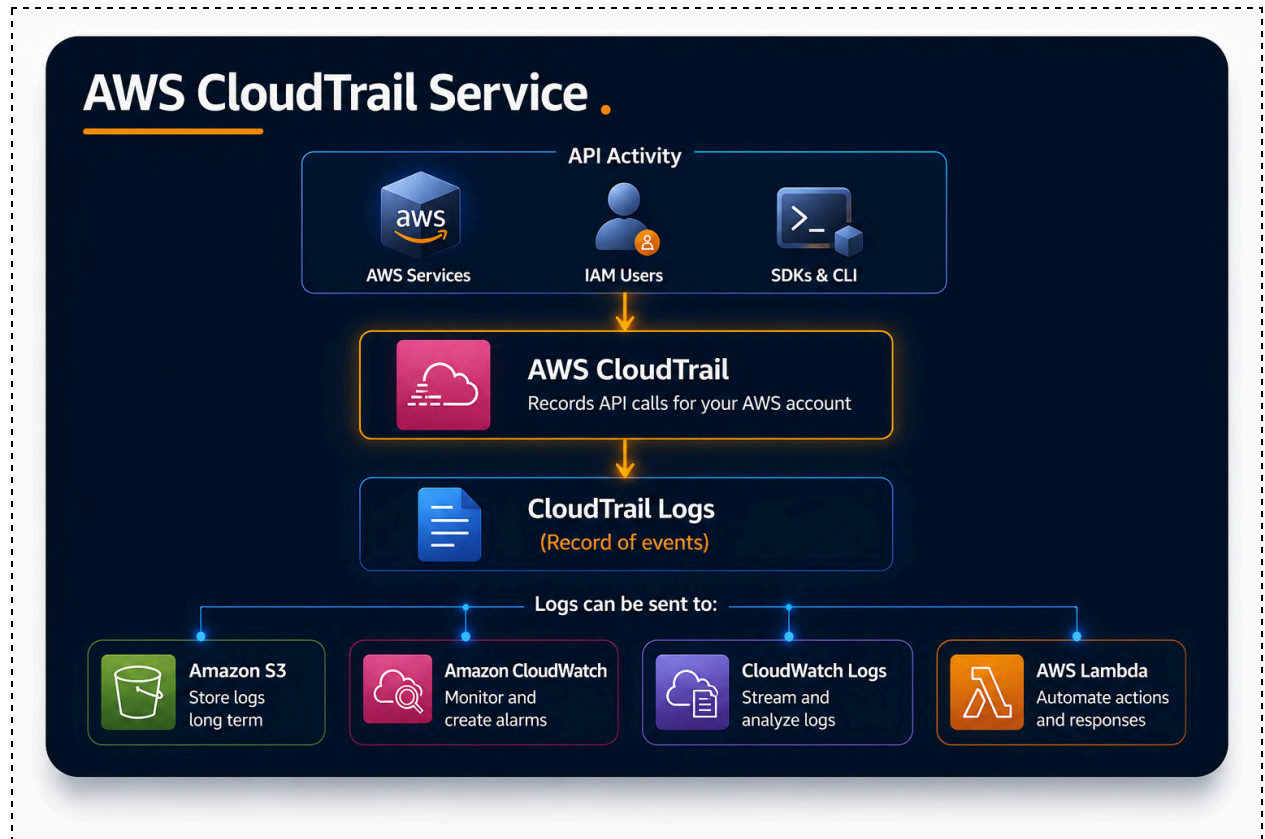
المفروض يكون عندك SNS Topic معمول ومتربط بالـ Email Address بتاعك.

بعدها تختار الـ Topic كـ Target.

بالتالي الـ Flow يبقى:

1. EC2 Instance تقف.
 2. EventBridge Rule تلاحظ الـ State Change.
 3. الـ Rule تبع الـ Event للـ SNS Topic.
 4. الـ SNS Topic بيعت Email.
 5. يوصل Notification إن الـ EC2 Instance وقفت.
-

AWS CloudTrail



Core Concept

الـ AWS CloudTrail بتستخدم في الـ Audit.

وظيفتها إنها بتسجل الـ Management Activity والـ Actions اللي بتحصل على الـ AWS Account.

تقدر من خلالها تعرف:

- مين عمل Action.
- عمل إيه.
- إمتى.
- وفين.

Account Activity Tracking

CloudTrail بتسجل الـ Actions اللي بتحصل على الـ Account سواء حصلت من خلال:

- AWS Console.
- SDK.
- CLI.

مثلاً لو حد عمل:

- EC2 Instance لـ Stop.
- EC2 Instance لـ Delete.
- Lambda لـ Invoke.
- S3 على Upload.

تقدر ترجع لـ CloudTrail وتشوف مين نفذ الـ Action وإمتى حصل.

Event History

الـ Logs الخاصة بالـ Activity بتكون متخزنة في CloudTrail لمدة **90 يوم**.

في الـ Console فيه **Event History** تقدر من خلالها تشوف الـ Events اللي حصلت على الـ Account خلال آخر 90 يوم.

Long-Term Log Storage

لو عايز تحتفظ بالـ Logs لمدة أكثر من 90 يوم، تقدر تعمل **CloudTrail Trail**.

الـ Trail يخليك ترمي الـ Logs في مكان للـ Long-Term Storage زي:

- S3.
- CloudWatch.

Audit Example

لو حد عمل Delete لـ Resource عندك، مثلاً EC2 Instance، تقدر تدخل CloudTrail وتشوف:

- مين الشخص اللي عمل الـ Delete.
- إمتى عمله.
- الـ Action اللي حصل.

CloudTrail Lab

Viewing Event History

من الـAWS Console:

1. ابحث عن **CloudTrail**.
2. افتح CloudTrail.
3. من الشمال اختار **Event History**.

الـEvent History بتعرض الـActions اللي حصلت على الـAccount.

في المثال ظهر Events زي:

- Stop Instance.
- Create Log Group.
- Actions تانية حصلت على الـAccount.

تقدر تستخدم المكان ده علشان تعرف أي حد عمل أي Action على الـAccount.

Creating a Trail

لو عايز تخزن الـLogs لأكثر من 90 يوم، تعمل Trail.

مثلاً سميت الـTrail:

Management Trail

بعدها بتحدد:

- هل هتستخدم S3 جديدة.
- أو S3 موجودة بالفعل.
- هل عايز Encryption ولا لأ.

Selecting Events

بعد كده بتحدد نوع الـ Events اللي عايز الـ Trail يسجلها.

ممکن تختار:

- Management Events.
 - Data Events.
 - Network Activity.
-

AWS Config



Core Concept

الـ AWS Config يستخدم علشان نتابع الـ Configuration بتاعة الـ AWS Resources.

تقدر تعمل Rules وتحدد Configuration معينة، وبعدها AWS Config تراقب الـ Resources وتشوف هل هي Compliant مع الـ Rule ولا لا.

Configuration Monitoring

مثلاً ممكن نقول:

لو فيه أي S3 Bucket عندها **Public Access**، طلع Alert.

ممكن كمان تبعث Notification عن طريق SNS للـ Email بتاعك.

مثال تاني:

لو فيه Security Group فاتح SSH من أي IP، طلع Alert.

Continuous Monitoring

AWS Config تقدر ترأقب الـ **Over Time** Resources.

يعني مش مجرد Check مرة واحدة.

تقدر تسببها شغالة، وأي وقت Resource Configuration تتغير وتبقى عكس الـ Rule اللي محددتها، AWS Config تطلع Alert وتقول إن الـ Resource بقت **Non-Compliant**.

Compliance Rules

فيه Rules جاهزة تقدر تستخدمها مع AWS Config.

الفكرة إنك تحدد Rule للـ Configuration اللي عايزها، وبعدھا AWS Config ترأقب الـ Resources وتقارنها بالـ Rule.

لو Resource مش ماشية مع الـ Configuration المطلوبة، تظهر إنها **Non-Compliant**.

Pricing

AWS Config Service مش Free.

AWS Config Lab

Initial Configuration

من الـ AWS Console:

1. ابحث عن **AWS Config**.
2. اختار **Get Started**.
3. حدد الـ Resources اللي عايز AWS Config تراقبها.

ممکن تختار:

- كل الـ Resources.
- Resource Type معين.

في المثال اخترت **S3**.

وتمكن تصنيف Resources ثانية زي Security Groups.

Recording Frequency

بعدها بتحدد الـ Frequency.

ممکن تكون:

- Continuous.
- Daily.

في المثال اخترت **Continuous** علشان يفضل يتابع الـ Configuration باستمرار.

IAM Role

AWS Config محتاجة IAM Role علشان تقدر تعمل Access للـ Resources.

ممکن:

- تخلي AWS Config تعمل Create للـ IAM Role.
- أو تختار IAM Role موجودة بالفعل.

Log Storage

بعدها بتحدد الـ Logs الناتجة من AWS Config هتتخزن فين.

ممکن:

- تعمل S3 Bucket جديدة.
- أو تستخدم S3 Bucket موجودة بالفعل.

SNS Notifications

ممکن کمان تخلي AWS Config تبعت Notification لما يحصل Change أو Resource تبقى Non-Compliant.

الـ Notification ممکن تتبعت من خلال SNS Topic مربوط بالـ Email Address بتاعك.

S3 Public Access Rule

في المثال كان الهدف إن كل S3 Buckets الموجودة في الـ Account مايقاش عندها Public Access.

ولو أي S3 Bucket بقت Public:

- AWS Config تكتشف إنها Non-Compliant.
- يظهر Alert.
- و ممکن يتبعت Notification عن طريق SNS.

بعد الإعدادات الأساسية، تختار Rule خاصة بـ S3 Public Access.

لو S3 Bucket خالفت الـ AWS Config Rule، تظهر إن الـ S3 دي **Not Compliant**.

Video 12 – Serverless, LAMBDA & API Gateway

AWS Lambda



Lambda Overview

AWS Lambda من أشهر الـ Services الموجودة على الـ Cloud، ويمكن تكون Service مهمة جداً لشركات كثير بتعتمد عليها، ويمكن أصلاً تنتقل للـ Cloud بسبب AWS Lambda.

الـ Lambda عبارة عن Function بنشتغل مع أكثر من Programming Language، ومن أشهرهم:

- Python
- Java
- Node.js

- ++C
- وفي Languages ثانية متاحة كمان.

Serverless Execution

بدل ما علشان تشغل Python تحتاج تجيب Server أو تأجر EC2 VM، وتحدد:

- Operating System.
- CPU.
- Memory.

مع AWS Lambda مش محتاج تعمل الكلام ده. كل اللي محتاجه إنك تجهز الـ Code بتاعك وتشغله على Lambda.

AWS Lambda تعتبر Serverless، فمش محتاج يبقى عندك Servers تديرها.

بتحط الـ Code داخل Lambda، وتختار الـ Language اللي عايز تستخدمها، والـ Function بتشغل لما يحصل لها Trigger.

الـ Code يقدر يعمل Execute لمدة توصل لحد 15 دقيقة.

Automatic Scaling

AWS Lambda بتعمل Scale بشكل Automatic.

لو جه عدد كبير من الـ Requests، Lambda تقدر تعمل Scale تلقائيًا وتستحمل عدد كبير من الـ Requests.

Lambda Pricing

الـ Charging في Lambda بيكون:

- Per Request.
- Per Compute Time.

وده مختلف عن EC2، لأن مع EC2 بتحدد Resources زي:

- CPU.
- RAM.
- Storage.

وبالتالي بتكون عارف الـ Pricing حسب نوع الـ Instance وطريقة الدفع.

لكن في Lambda ممكن تعمل Function وتحط عليها الـ Code من غير ما بيدأ عليك Charge لمجرد وجودها.

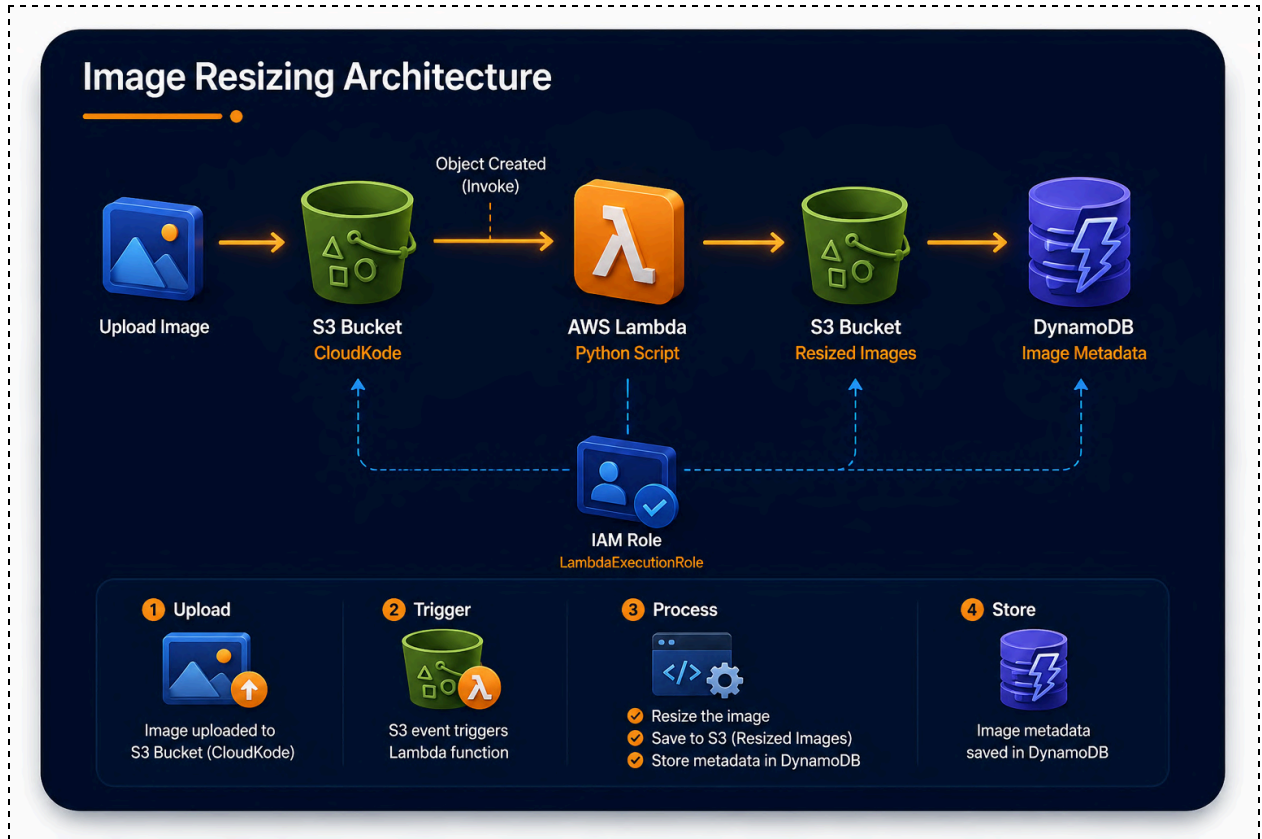
الـ Charge بيدأ لما يحصل Requests تعمل الـ Invoke لـ Lambda، وبيحدد كمان حسب الـ Compute Time اللي بتأخذه كل مرة يحصل Trigger لـ Function.

Service Integrations

AWS Lambda تقدر تعمل Integration مع Services مختلفة، منها:

- API Gateway.
- DynamoDB.
- SNS.

Image Resizing Architecture



Lab Components

في Lab هنعمل السيناريو بالشكل ده:

- CloudCode باسم S3 Bucket
- S3 Bucket ثانية للـ Resized Images
- AWS Lambda Function
- DynamoDB
- IAM Role

هنرفع Image على أول S3 Bucket

لما الصورة تترفع، الـ S3 هتعمل Invoke للـ Lambda

الـ Lambda هيبقى عليها Python Script يقدر:

1. يقلل حجم الصورة.
2. يخزن الصورة بعد التعديل في S3 Bucket ثانية.
3. يحط شوية معلومات عن الصورة في DynamoDB.

Image Compression Scenario

الفكرة شبيهة باللي ممكن يحصل مع صور الـ Profile على Services زي WhatsApp أو Facebook.

الصورة اللي بتظهر مش لازم تكون الـ Original Image بنفس الحجم الكامل، لأن تخزين الـ Full Pictures والـ Full Videos لكل الناس هيجتاج Storage كبير.

علشان كده ممكن يحصل Compression لتقليل الحجم.

في السيناريو بتاعنا، لو عندنا صورة حجمها مثلاً 5 MB وارتفاعها على S3:

1. الـ S3 تبعت Event للـ Lambda وتقول إن الصورة اترفعت.
2. الـ Lambda تعالج الصورة وتصغر حجمها.
3. الصورة الجديدة تتخزن في S3 Bucket ثانية.
4. بعد كده ممكن الصور الأصلية تنتشال من الـ S3 الأولى.

بدل ما الصورة تفضل واحدة 5 MB، ممكن مثلاً تتخزن بحجم 50 KB.

في الـ Script هنخلي أي صورة تترفع تتحول لصورة حجمها 50 KB.

وممكن الحجم يتحدد بشكل مختلف زي:

- 50 KB.
- 100 KB.
- 500 KB.

كمان بيانات الصورة زي اسمها وحجمها بتتحت في DynamoDB.

Lab Preparation

S3 Buckets

عندنا S3 Bucket الصورة الأصلية هتترفع عليها.

وعندنا S3 Bucket ثانية للـ Resized Images.

لما الصورة تترفع على أول Bucket:

1. الـ S3 تعمل Trigger للـ AWS Lambda.
2. Lambda تغيّر حجم الصورة.
3. Lambda ترمي الصورة الجديدة في الـ S3 Bucket الثانية.

DynamoDB Table

عندنا DynamoDB Table باسم CloudCode.

الـ Lambda هتاخذ بيانات ليها علاقة بالصورة وتحطها داخل الـ Table.

Lambda IAM Role

عملنا Role لازم تتعمل Attach للـ Lambda علشان الـ Function تقدر تتعامل مع الـ Services المطلوبة.

الـ Role باسم CloudCode Lambda Role، وعليه:

- Full Access على DynamoDB.
- Full Access على S3.

الـ Role ده هيتربط بالـ Lambda وقت إنشائها، علشان تقدر تكلم S3 و DynamoDB.

Creating the Lambda Function

Function Configuration

علشان نعمل الـ Lambda:

1. ندخل على AWS Lambda.
2. نختار Create Function.
3. نسمي الـ Function:
 - CloudKode Lambda.
4. نختار الـ Runtime.

من الـ Runtimes المتاحة ظهر:

- .Node.js
- .Python
- .Ruby
- وغيرهم.

بعد كده نختار الـ Role اللي عملناه قبل كده، وهو CloudKode Lambda Role، واللي بيدي Access على S3 وDynamoDB.

بعدها نعمل Create للـ Function.

Adding the Code

داخل الـ Lambda فيه Tab باسم Code.

هنا بنحط الـ Code اللي عايزين نشغله.

الـ Code المستخدم في التجربة بيعمل Resize للصورة، بحيث لو الصورة مثلاً 5 MB تتحول إلى 50 KB.

الهدف من التجربة هو توضيح الفكرة نفسها، ومش شرط الصورة بعد التصغير تطلع بنفس الجودة.

الـ Script بيتعامل مع الـ Buckets والـ DynamoDB المطلوبة في السيناريو.

بعد إضافة الـ Code، بنضغط Deploy علشان الـ Code يتعمل له Update داخل الـ Lambda.

Configuring the S3 Trigger

Trigger Behavior

محتاجين نضيف Trigger للـ Lambda.

الـ Trigger المطلوب هو إن لما صورة تترفع على S3، يحصل Invoke للـ Lambda.

بعد تشغيل الـ Lambda:

1. تأخذ الصورة.
2. تغيّر حجمها إلى 50 KB.
3. تحط الصورة الجديدة في الـ S3 Bucket الثانية.
4. تحط بيانات الصورة في DynamoDB.

Trigger Setup

من Add Trigger:

1. نختار S3.
2. نحدد الـ Bucket:

CloudCode S3 Bucket.

3. نحدد الـ Event:

All Object Create Events.

بكده الـ S3 بقت Trigger للـ CloudCode Lambda.

وفي Configuration بنلاقي الـ Role المرتبط بالـ Lambda، وهو نفس الـ Role اللي عملناه قبل كده.

Testing the Lambda Workflow



Uploading the Image

هنروح للـCloudCode S3 Bucket ونرفع صورة.

الصورة المستخدمة في التجربة كانت صورة من استاد Giuseppe Meazza أو San Siro في Milan.

بعد الـUpload، ظهر إن حجم الصورة الأصلية حوالي 2.8 MB.

المطلوب إن Lambda تغيّر الحجم ده وتخليه 50 KB.

Checking the Resized Bucket

بعد ما الصورة اترفعت، دخلنا على الـS3 Bucket الخاصة بالـResize.

ظهر فيها ملف جديد للصورة بعد المعالجة.

الصورة اتعمل لها Resize، والحجم بقى 50 KB.

Checking DynamoDB

بعد كده دخلنا على DynamoDB نشوف الـ Update اللي حصل.

ظهر Record فيه Metadata عن الصورة، منها:

- اسم الصورة.
- .Original Size
- الحجم الجديد: 50.
- .Target Bucket
- معلومات تانية عن الصورة.

API Gateway



API Gateway Overview

API Gateway Service بتقدر من خلالها تعمل Create و Publish للـ APIs.

وكمان بتعمل Scale Automatically، بنفس فكرة Lambda، ومش محتاج تعمل Provision لـ CPU أو Servers علشان تشغلها.

API Gateway تعتبر Serverless.

بتدعم أنواع مختلفة، منها:

- REST.
- WebSocket.
- HTTP API.

وتقدر تعمل Integration مع Services زي:

- Lambda.
- DynamoDB.
- S3.

API Gateway Use Cases

Reading Data from DynamoDB

تخيل إن عندك Data موجودة في DynamoDB، وعاليز User يقدر يشوفها.

مش منطقي تدي User Access على AWS Console علشان يدخل يشوف الـ Data.

بدل كده، ممكن تبديله Link أو URL من خلال API Gateway.

من خلال الـ Link ده، الـ User يقدر يعمل Request ويشوف شوية Data من DynamoDB.

والـ Lambda ممكن تدخل في الـ Architecture دي وتساعد في عملية جلب الـ Data.

Uploading Images to S3

في الـ Example السابق كنا بنرفع الصورة على S3 من خلال AWS Console.

لكن مش لازم كل User يدخل على الـ Console ويروح لـ S3 بنفسه.

ممكن تدي الـ Users Link من خلال API Gateway، ومن خلاله يقدروا يرفعوا الصورة، والصورة في الآخر تترفع على S3.

Products API Example

Product Data

في المثال الجديد هنفترض إن عندنا شوية Products.

بدل ما نعمل DynamoDB ونربطها بالـ Lambda في التجربة، هنعط الـ Data جوه Lambda.

عندنا Product أول:

- Laptop.
- Price: 1200\$.

وعندنا Product ثاني:

- Smartphone.
- Price: 800\$.

المطلوب إن User أو Application من بره يبقى معاه URL يقدر من خلاله يشوف الـ Products والأسعار.

Creating the API Gateway

API Types

لما ندخل على API Gateway، بنختار نوع الـ API.

ظهر أكثر من اختيار، منها:

- HTTP API.
- WebSocket API.
- REST API.
- REST API Private.

الـ HTTP API بيشتغل مع Lambda و HTTP Backends.

الـ WebSocket API بيستخدم Persistent Connections، وبيشتغل مع Lambda و HTTP و AWS Services.

الـ REST API بيشتغل برضه مع Lambda و HTTP و AWS Services.

وفي REST API Private لو عايز الـ API Gateway تبقى Accessible بس من الـ VPC.
في التجربة هنستخدم REST API.

Creating the API

هنعمل:

1. Create REST API
2. نختار New API
3. نسميها مثلاً CloudCode API
4. نعمل Create API

API Resources

Resource Paths

بعد إنشاء الـ API، بنعمل Resource.
الـ Resource بيمثل Path داخل الـ URL.
ممكن نفس الـ API يبقى جواها أكثر من Path.
مثلاً ممكن يبقى عندك Paths مختلفة لـ Services أو Data مختلفة.
في التجربة هنخلي الـ Path:

```
/products
```

بالتالي الـ Link الأساسي هيبقى فيه **products/** علشان يرجع الـ Products.
بعد كده نعمل Create Resource

API Methods

GET Method

بعد إنشاء `products/`، بنحدد الـ `Method`.

يمكن نفس الـ `Resource` أو `Resources` مختلفة تستخدم `Methods` مختلفة حسب المطلوب.

مثلاً:

- `GET` لجلب `Information`.
- `PUT` لرفع أو إرسال `Information`.

في التجربة نستخدم `GET`.

نعمل `Integration` مع `Lambda Function`.

ونختار الـ `Lambda` التي فيها الـ `Product Data`.

بكد `products/` هيعمل `GET` من الـ `Lambda Function`.

Multiple Resources

نفس الـ `API Gateway` ممكن يكون فيها أكثر من `Resource`.

مثلاً:

```
/products  
/users
```

وكل `Resource` ممكن يعمل `Integration` مع `Service` مختلفة، وكمان يستخدم `Method` مختلفة.

Deploying the API

Deployment Stage

بعد ما خلصنا الـ `Resource` والـ `Method` `GET`، بنعمل `Deploy` للـ `API`.

وقت الـ `Deployment` بنحدد `Stage`.

ممكن يكون عندنا `Stages` مختلفة زي:

- `Dev`.

- Test.
- Production.

في التجربة سمينا الـ Stage:

dev

بعد الـ Deployment ظهر الـ API وتحتها:

- Stage: dev.
- Resource: products.
- Method: GET.

Invoke URL

API Gateway بتوفر Invoke URL نقدر نستخدمه علشان نعمل الـ Service.

الـ URL في التجربة بيحتوي على:

/dev/products

وده هو الـ Link اللي هنستخدمه للوصول للـ Products.

Testing the Products API

حطينا الـ Invoke URL وجربناه.

الـ Response رجّع الـ Data الموجودة في الـ Lambda:

- الـ Laptop وسعره.
- الـ Smartphone وسعره.

بكدّه بدل ما نحتاج ندخل على الـ Database أو AWS Console علشان نشوف المعلومات، قدرنا نستخدم API Gateway URL علشان نعمل الـ Retrieve للـ Information.

Video 13 – Containers, ECS, ECR & EKS

Container Services Overview

الجزء ده ليه علاقة بـ Docker و Kubernetes.

بالنسبة للامتحان، مش مطلوب تذاكر كل تفاصيل Docker و Kubernetes، لكن مطلوب تعرف المصطلحات دي:

- ECR.
- ECS.
- Fargate.
- EKS.

ولو هتشتغل عملياً بالـ Services دي على AWS، هتحتاج يكون عندك معرفة بـ Docker و Kubernetes.

Docker Basics

Traditional Application Setup

لو عايز تعمل Web Server بالطريقة التقليدية، ممكن يكون عندك Server وتحط عليه:

- .Ubuntu
- .Apache
- HTML File فيه محتوى الـ Web Server.

وممكن يبقى عندك Application ثاني محتاج:

- .Ubuntu
- .Python
- شوية Files خاصة بالـ Application.

كل Application بيحتاج مجموعة Components مع بعض علشان يشتغل.

المشكلة بتظهر لو الـ Server حصل فيه مشكلة، أو محتاج تنقل الـ Application من Server لـ Server ثاني، أو تنقل الـ Infrastructure من Physical Infrastructure للـ Cloud.

في الحالة دي محتاج تنقل نفس الـ Combination من الـ Components مع بعض.

Docker Image

من هنا بتيجي فكرة Docker.

ممکن تاخذ:

- Ubuntu Operating System.
- Apache.
- HTML File.

وتعمل منهم Docker Image.

الـ Docker Image بتحتوي على الـ Components اللي محتاجها الـ Application.

Docker Container

من الـ Docker Image تقدر تعمل Container على Infrastructure مختلفة.

مثلاً ممكن تستخدم الـ Image على:

- VM على Azure.
- EC2 Instance على AWS.

وتعمل منها Docker Container.

الـ Container هيشغل نفس الـ Components الموجودة في الـ Image، وبالتالي تقدر تشغل نفس الـ Web Server.

Docker Registry

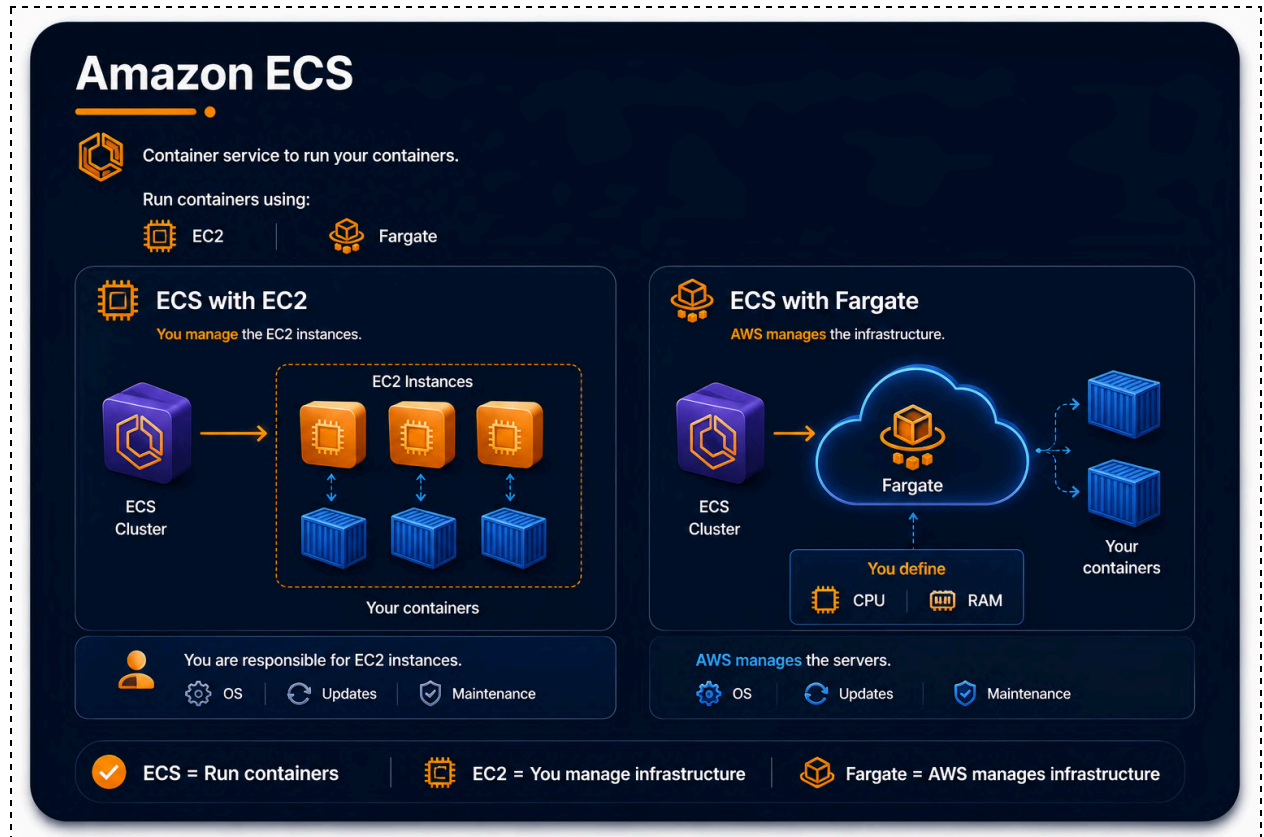
الـ Docker Image محتاجة تتخزن في مكان علشان:

- تشاركها مع زميلك.
- أو تخليها Public بحيث ناس تانية تقدر تستخدمها.

ممکن تستخدم Docker Hub Registry علشان ترفع عليه الـ Docker Images.

وفي AWS عندنا Service بتعمل نفس الفكرة وهي ECR.

Amazon ECS



Amazon ECS هي Container Service بتقدر من خلالها تشغل Containers.

الـ Containers ممكن تشغل باستخدام:

- EC2.

- Fargate.

ECS with EC2

يمكن اختيار مجموعة EC2 Instances وتستخدمهم كـ ECS Cluster. بعد كده تشغل Containers على Instances الموجودة داخل الـ Cluster. يعني إنت المسؤول عن الـ EC2 Instances اللي الـ Containers هتشغل عليها.

ECS with Fargate

الاختيار الثاني هو Fargate.

مع Fargate مش محتاج تدير الـ Servers بنفسك.

مش محتاج تهتم بحاجات زي:

- Operating System.
- Updates الخاصة بالـ Server.

إنت بتحدد الـ Resources اللي محتاجها، زي:

- CPU.
- RAM.

وبعد كده تشغل الـ Container.

Amazon EKS

Amazon EKS هي Service خاصة بـ Kubernetes Clusters.

وتقدر تستخدم معها EC2 أو Fargate.

Managed Nodes

في AWS، Managed Nodes بتعمل Create و Manage للـ Nodes بالنسبة لك.

Self-Managed Nodes

في Self-Managed Nodes، إنت اللي بتعمل Create للـ Nodes وبتربطها بالـ EKS Cluster.

Fargate with EKS

تقدر تستخدم AWS Fargate مع EKS.

ميزة Fargate إنها مناسبة لو إنت مش عارف الـ Workload عندك هيتحتاج Infrastructure عامله إزاي.

مثلاً لو مش عارف:

- هتحتاج 8 GB RAM ولا أكثر.
- هتحتاج CPU قد إيه.

ممكن تستخدم Fargate علشان تتعامل مع الـ Workload.

أما لو إنت عارف الـ Load بتاعك وعارف الـ Infrastructure المطلوبة، ممكن تستخدم Managed Nodes أو Self-Managed Nodes.

Amazon ECR



Amazon ECR هي Container Registry على AWS.

بتستخدمها علشان ترفع عليها الـ Docker Images، وبعد كده تستخدم الـ Images دي لتشغيل Containers داخل AWS.

Creating a Docker Image

في التجربة كان عندي Docker Image معمولة بالفعل على الـ Machine اللي بسجل منها.
اسم الـ Image:

```
cloudkode-fan
```

والـ Tag:

```
latest
```

الـ Image اتعملت باستخدام الـ Dockerfile.

الـ Dockerfile بيحدد إن الـ Image يكون فيها:

- NGINX Latest.
- `index.html`.

ملف `index.html` بيتحط في Path محدد لما الـ Container يتعمل من الـ Image.

محتوى الـ HTML File كان بيعرض:

```
I'm a Big Fan of CloudKode
```

أي حد يستخدم الـ Image ويعمل منها Container هيقدر يشغل Static Website باستخدام NGINX.

Pushing the Image to ECR

Authenticating with ECR

أول خطوة إننا نعمل Authentication مع ECR.

استخدمنا Command خاص بالـ Authentication فيه:

- Region.
- Account Details.

وقبلها كان معمول AWS Configure باستخدام:

- User Name.
- Secret.

بعد تنفيذ الـ Authentication ظهر:

```
Login Succeeded
```

Tagging the Image

بعد كده عملنا Tag للـ Docker Image.

استخدمنا **docker tag** علشان نعمل Re-tag للـ Image بالاسم المطلوب من ECR.

الهدف إن اسم الـ Image يبقى بالشكل اللي ECR محتاجه.

بعد تنفيذ **docker images** ظهر الاسم الجديد للـ Image.

Pushing to ECR

بعد كده استخدمنا:

```
docker push
```

وعملنا Push للـ Image باستخدام الـ Tag الجديد.

بعد انتهاء الـ Push وعمل Refresh داخل ECR، ظهرت الـ Image داخل الـ Repository.

ده معناه إن الـImage اترفعت بنجاح على Amazon ECR.

Creating an ECS Fargate Cluster

بعد رفع الـImage على ECR، بدأنا نعمل Cluster على Amazon ECS.
الخطوات كانت:

1. الدخول على Elastic Container Service.
2. اختيار Clusters.
3. اختيار Create Cluster.
4. تسمية الـCluster:

CloudKode Cluster

5. اختيار الـInfrastructure.

كان عندنا اختيارين:

- EC2 Instances.
- Fargate.

في التجربة اخترنا Fargate.

Creating a Task Definition

بعد إنشاء الـCluster، عملنا Task Definition.

الـCluster ممكن يشتغل باستخدام EC2 أو Fargate، وإحنا اخترنا Fargate.

Compute Resources

داخل الـTask Definition حددنا:

- CPU.

- Memory.

Task Execution Role

اخترنا Task Execution Role معمول مسبقاً.

الـ Role عليه Permission تسمح للـ Task إنها تتعامل مع ECR.

السبب إن الـ Task Definition محتاجة تعمل Pull للـ Image من ECR.

Container Configuration

داخل الـ Task Definition حددنا اسم الـ Container:

CloudKode_Web

بعد كده حددنا الـ Image URI الخاصة بالـ Image اللي رفعناها على ECR.

وحددنا إن الـ Container يشتغل على:

Port 80

وبعد كده عملنا Create للـ Task Definition.

Running a Task on the Cluster

بعد إنشاء الـ Task Definition، رجعنا للـ Cluster علشان نشغل الـ Task.

كان فيه Task قديمة حصل فيها مشكلة بسبب IAM Permission، وكانت الـ Task مش قادرة تكلم ECR.

علشان كده لازم الـ IAM Role المرتبط بالـ Task يسمح لها تتعامل مع ECR.

Task Configuration

وقت إنشاء الـ Task:

1. اخترنا Fargate.
2. حددنا عدد الـ Tasks:
 - Task واحدة.

3. اخترنا الـ Task Definition:
CloudKode Task Definition ○

يمكن تشغيل أكثر من Task، لكن في التجربة استخدمنا Task واحدة.

Networking

في Networking حددنا:

- VPC.
- Security Group.
- Public IP.

وبعد كده عملنا Create للـ Task.

CPU Architecture Issue

أول محاولة للـ Task فشلت بسبب CPU Architecture.

الـ Image الأصلية كانت معمولة على Mac باستخدام Architecture:

ARM64

وده ماكانش Supported في الـ Setup المستخدم على AWS Fargate.

علشان كده عملنا Build للـ Image مرة ثانية باستخدام Architecture:

x86

وبعد كده:

1. رفعنا الـ Image الجديدة على ECR.
 2. استخدمنا الـ Image الجديدة في الـ Task Definition.
-

Subnet Configuration

في الـ Networking لازم تختار الـ Subnet المناسبة حسب الـ Use Case.

لو عايز الـ Container يبقى Accessible من الإنترنت، محتاج تستخدم Public Subnet.
أما لو الـ Container المفروض يبقى داخل Private Network، تستخدم Private Subnet.
في التجربة تم اختيار Public Subnet علشان الـ Container يبقى Accessible من الإنترنت.

Running Container

بعد تشغيل الـ Task، ظهر إن:

- Launch Type هو Fargate.
- الـ CPU والـ Memory بالقيم اللي تم اختيارها.
- الـ Container شغال.
- الـ Image المستخدمة هي الـ Image الموجودة على ECR.

الـ Static Website كانت معمولة باستخدام NGINX.

Security Group Issue

الـ Container كان Running، لكن الـ Static Website في البداية ما كانتش بتفتح.

المشكلة كانت في الـ Security Group، لأن:

Port 80

ماكانش مفتوح.

تم إنشاء Task جديدة والتأكد إن الـ Security Group تسمح بالوصول على Port 80.

بعد كده الـ Container اشتغل، ولما فتحنا الـ Public IP ظهر الـ Static Website ومحتواها:

Fan of CloudKode

وبكده اتأكدنا إن الـ Container اشتغل والـ Static Website شغالة عليه.

Video 14 – AI artificial intelligence

AWS AI Services

Amazon Rekognition



Amazon Rekognition يتعمل Analysis على Videos والـ Images.

أول ما تسمع Amazon Rekognition، اربطها على طول بصورة أو فيديو بيتعمل عليهم Analysis.

Object Detection

تقدر تستخدم Rekognition علشان تدور على Objects جوه الصورة أو الفيديو.
مثلاً:

- تدور على قلم.
- تعرف فيه كام قطعة في الصورة.
- تدور على أي Object موجود جوه الصورة.

People and Face Detection

ممكّن تستخدمها علشان تدور على أشخاص.
مثلاً تدي Rekognition صورة لشخص معين وتقول لها تدور عليه جوه Video، وتحدد:

- ظهر كام مرة.
- ظهر في أنهي ثانية في الفيديو.

Text Detection

ممكّن تديها Images وتطلب منها تطلع الـ Text الموجود جواها.

Face Analysis

Rekognition تقدر تعمل Analysis للوجه الموجود في الصورة وتطلع Information زي:

- Age Range.
- Gender.
- Smiling.
- Happy.
- Wearing Glasses.

مثلاً ممكّن تطلع Results بالشكل ده:

- Female بنسبة 100%.
- Happy بنسبة 98%.
- Smiling بنسبة 100%.

Sports Game Analysis

ممكّن تستخدم Rekognition في Sports Game Analysis.

مثلاً في Match تقدر تستخدمها علشان تحلل حاجات زي:

- نسبة الاستحواذ لكل فريق.
- اللاعب أخذ الكرة كام مرة.
- عمل كام Pass صح.
- عمل كام Shot على الجون.

Unsafe Content Detection

من الاستخدامات المعروفة لـ Rekognition، ويمكن تظهر في الامتحان، إنها تعمل لـ Unsafe Detection Content.

مثلاً لو في Video فيه أجزاء:

- 18+.
- فيها عنف شديد.
- أو Content غير مناسب.

Rekognition تقدر تعمل لـ Unsafe Detection ده.

Rekognition Practical Example

داخل Rekognition فيه Options مختلفة للـ Analysis.

في Face Analysis، ممكن ترفع صورة وهي تطلع Results زي:

- Age Range.
- Smiling بنسبة معينة.
- Wearing Glasses بنسبة معينة.

في تجربة بصورة ليا في استاد Inter Milan، ظهر إن:

- Appears to be Male بنسبة 99.9%.
- Age Range من 37 لـ 45 سنة.
- Appears to be Smiling بنسبة 94.2%.
- Appears to be Happy بنسبة 99.3%.
- Wearing Glasses مش.

Amazon Transcribe



Amazon Transcribe بتحوّل Audio إلى Text.

مثلاً لو قلت:

I love CloudKode

تقدر تحولها إلى Text:

I love CloudKode

أول ما تلاقي في الامتحان حاجة بتحوّل Audio إلى Text، فكر في Amazon Transcribe.

Meeting Transcription

يمكن يكون عندك Meeting مع Customer والـ Meeting مسجل.

تقدر تستخدم Transcribe علشان تطلع:

- كل الكلام اللي اتقال في الـ Meeting على شكل Text.
- أو Text تقدر تستخدمه بعد كده.

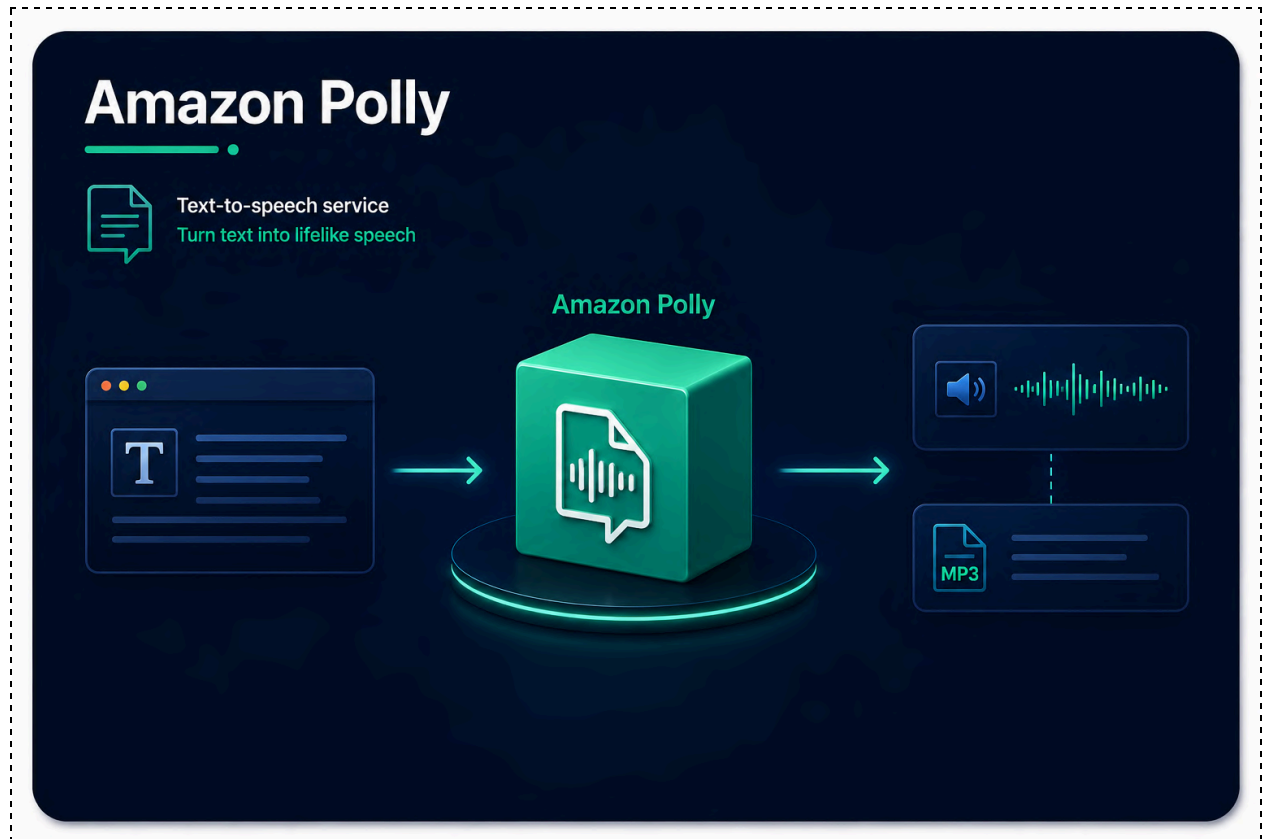
وفي نفس الوقت ممكن يكون في بيانات حساسة اتقال أثناء الـ Meeting.

Amazon Transcribe تقدر تعمل Removal للـ PII Information الحساسة الموجودة في الـ Text.

من أمثلة البيانات دي:

- أسماء الأشخاص.
 - العناوين.
 - أرقام التليفونات.
 - أي Information ممكن تحدد هوية الشخص.
-

Amazon Polly



Amazon Polly عكس Amazon Transcribe.

بدل ما تحول Audio إلى Text، إنت بتديها Text وهي تحوله إلى Audio.

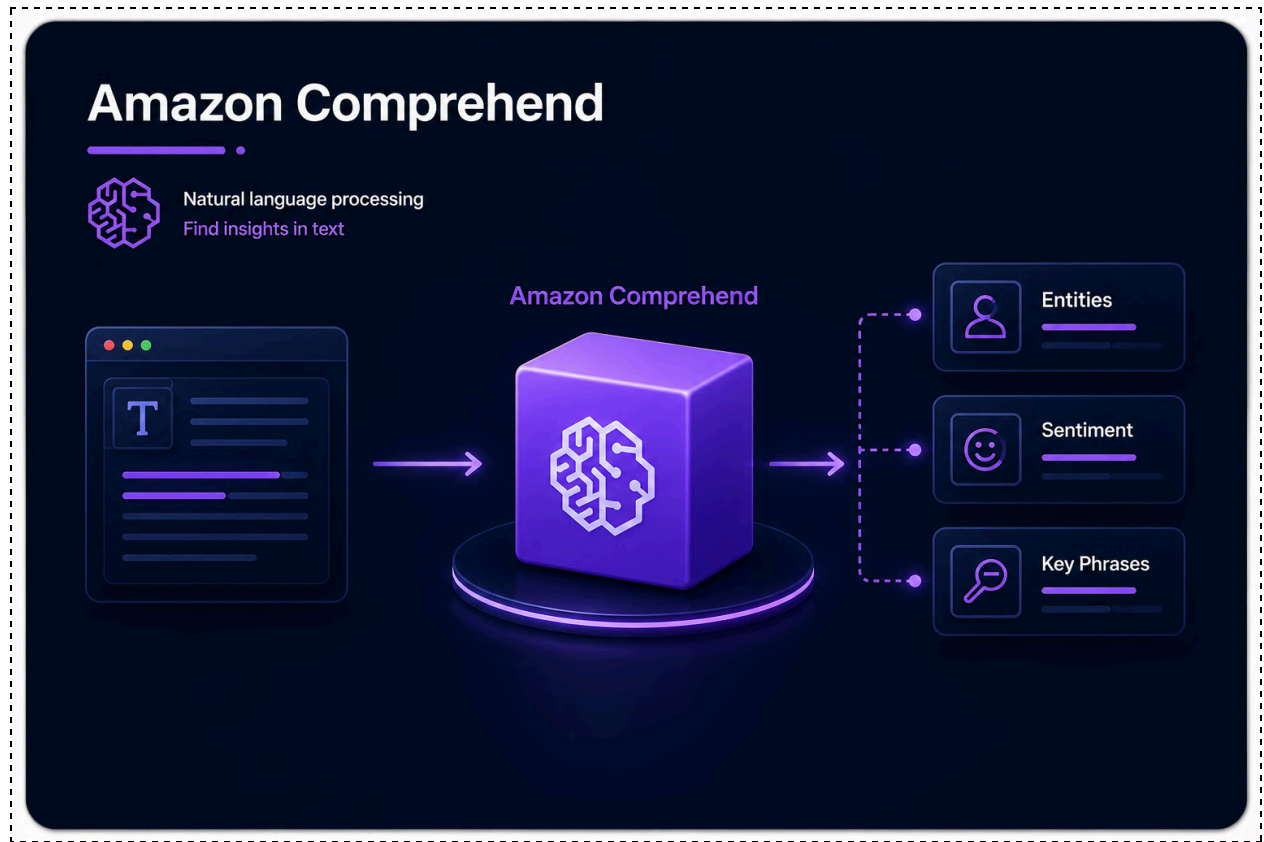
مثلاً تكتب:

I love CloudCode

وهي تطلع Audio بيقول:

I love CloudCode

Amazon Comprehend



لما تلاقي Natural Language Processing أو NLP، فكر في Amazon Comprehend.

إنت بتدي Comprehend Text أو Email، وهي تعمل Analysis على الـText.

Sentiment Analysis

ممکن تحدد إذا كان الـText:

- Positive.
- Negative.

مثلاً لو شركة بييجيلها Complaints أو Feedback من Customers من خلال Emails، تقدر تدخل محتوى الـEmail إلى Comprehend وتعرف الـTone بناعه Positive ولا Negative.

Language Detection

Comprehend تقدر تحدد الـLanguage المستخدمة في الـText أو Email.

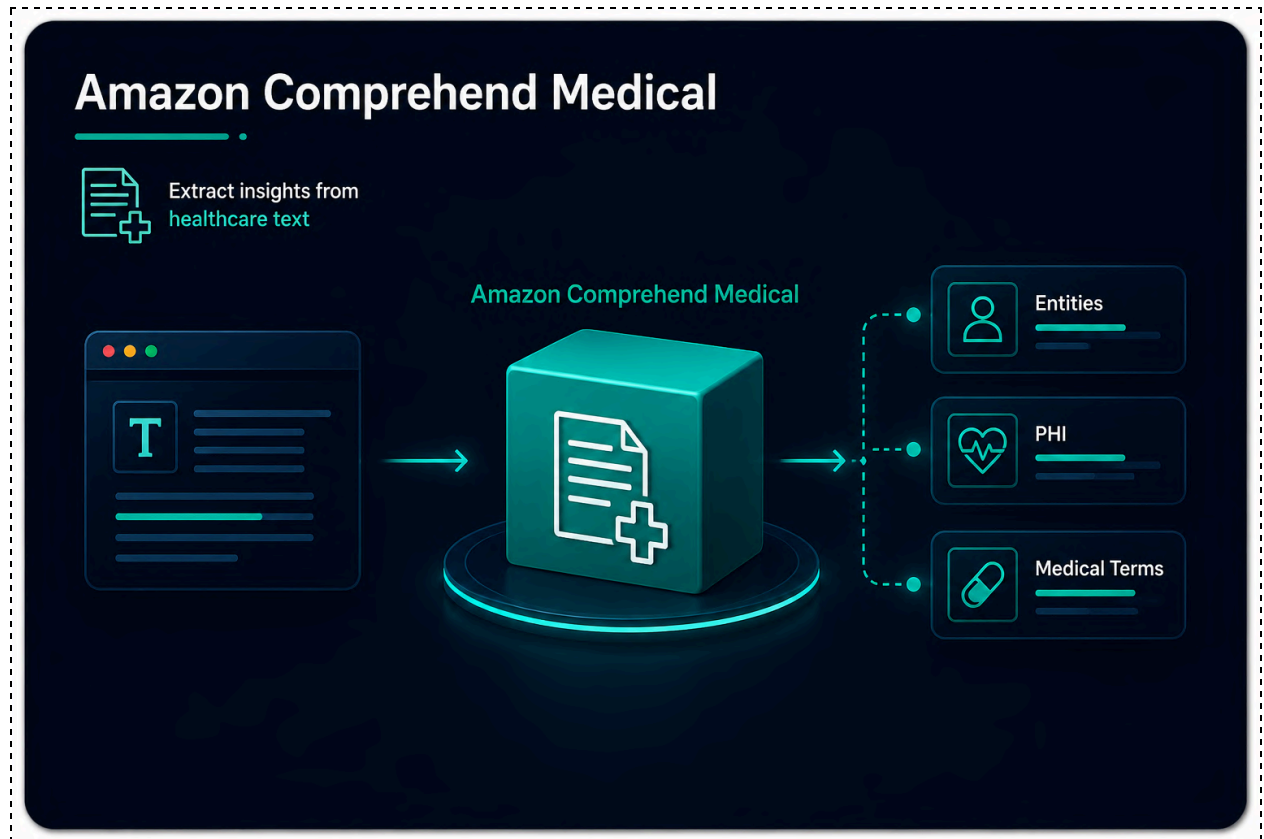
PII Detection

ممکن کمان تطلع الـ PII الموجودة في الـ Text.

مثلاً:

- اسم الشخص.
 - العنوان.
 - رقم التليفون.
-

Amazon Comprehend Medical



Amazon Comprehend Medical مخصصة للـ Medical Data.

تقدر تدخل لها Data مرتبطة بالمجال الطبي، وهي تعمل Analysis على البيانات دي.

لو في الامتحان Scenario له علاقة بالمستشفيات وظهر Amazon Comprehend Medical ضمن الاختيارات، فده الـ Option المرتبط بالـ Medical Data.

Amazon Polly Practical Example

داخل Amazon Polly كتبنا:

I love CloudCode

وبعدھا استخدمنا Listen، فتحول الـ Text إلى Audio.

Amazon Comprehend Practical Example

دخلنا Text فيه بيانات عن شخص اسمه Mohamed Rizk ورقم تليفونه، وإنه اشترى جهاز وكان فيه عيب، وهو محتاج Refund لأن الـ Product كان سيئ.

لو شركة استقبلت Email بالشكل ده، تقدر تستخدم Comprehend علشان تعمل Analysis عليه.

ظهر في الـ Analysis:

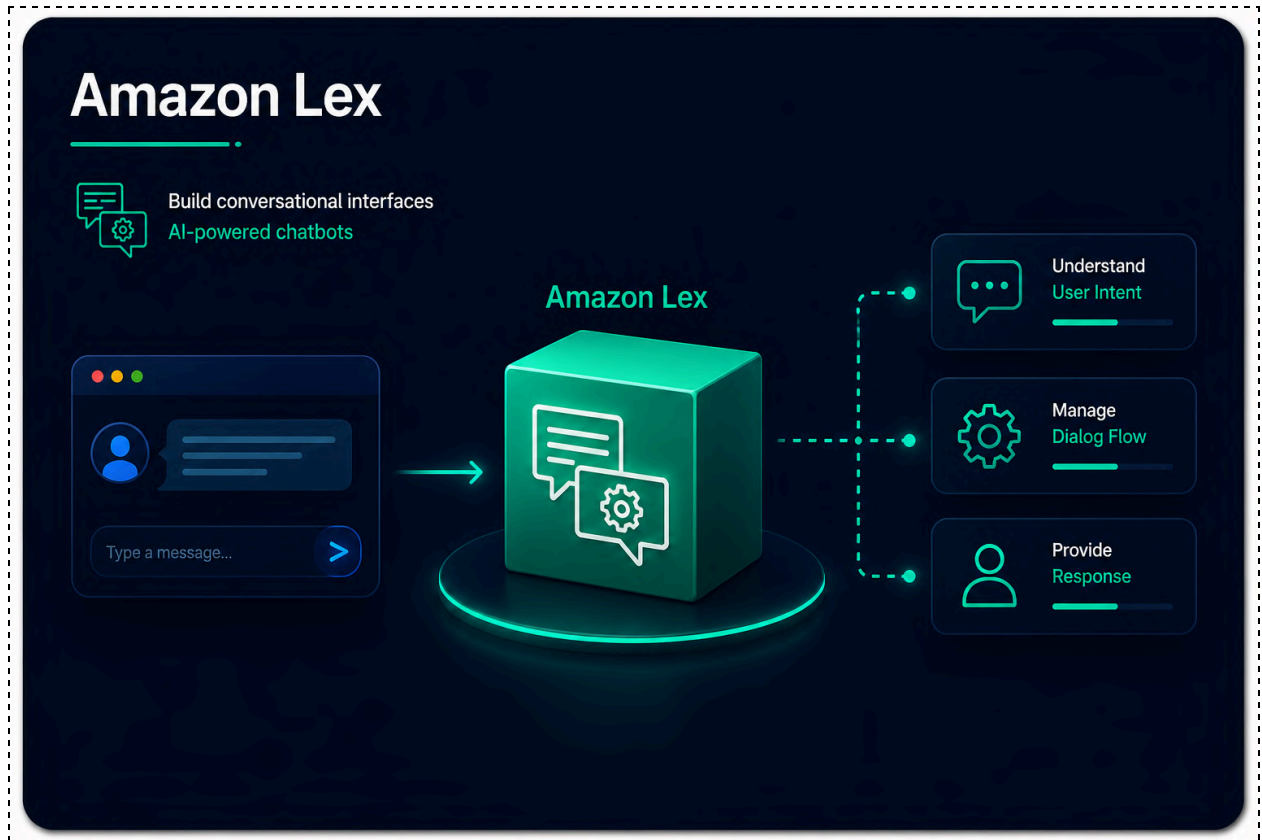
- Language: Arabic
- Confidence: 100%

وفي الـ Sentiment:

- Positive بنسبة 0%.
- Negative بنسبة 95%.

وكان ممكن Comprehend تطلع الـ PII الموجودة في الـ Text لو فيه بيانات زي Address أو بيانات شخصية تانية.

Amazon Lex

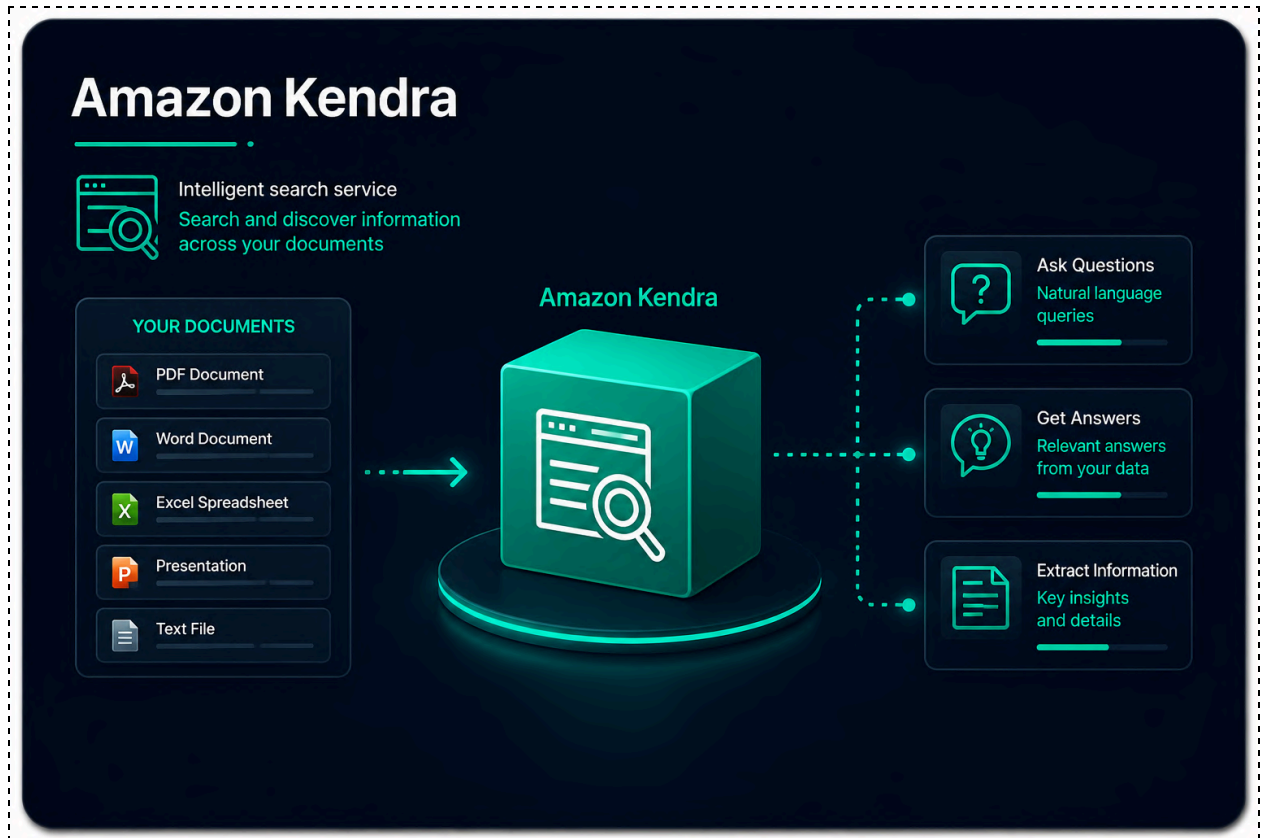


لما تلاقى كلمة Chatbot أو Conversational Interface، فكر في Amazon Lex.

Amazon Lex بتساعدك تبني AI Chatbot.

مثلاً لما تدخل Website وتلاقى Customer Service عبارة عن Chatbot بيتكلم معاك، Amazon Lex تقدر تساعدك تبني الـ Model ده.

Amazon Kendra



Amazon Kendra بتستخدم علشان تعمل Search على Information موجودة داخل Documents.

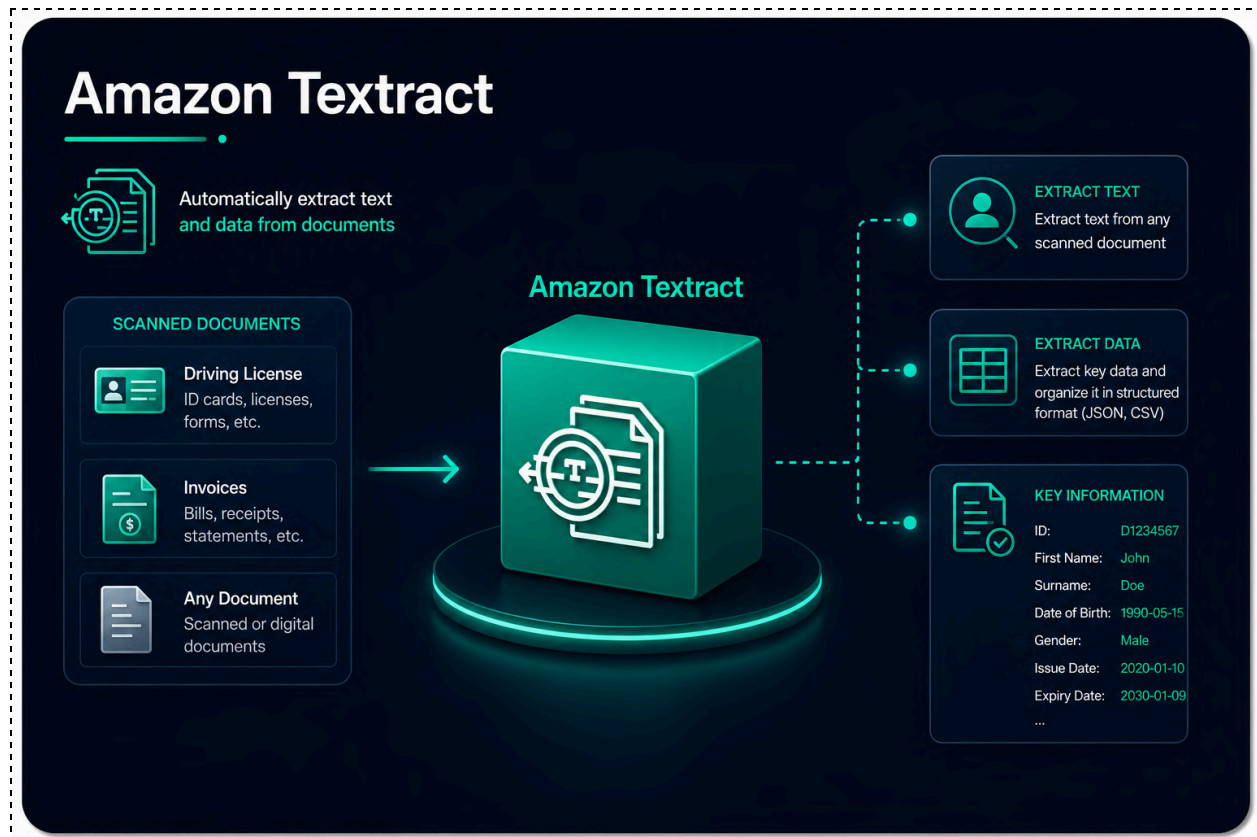
أول ما تلاقى Document Search، فكر في Amazon Kendra.

هي عبارة عن Document Search Engine.

تقدر ترفع لها Documents، وبعدها تقدر:

- تسأل أسئلة عن الـ Documents.
- تاخذ Answers.
- تطلع Information منها.

Amazon Textract



Amazon Textract من اسمها بتستخدم لاستخراج الـ Text من Documents.

لو عندك Scanned Document، تقدر تديه لـ Textract وهي تطلع الـ Text والبيانات الموجودة جواه.

Driving License Example

ممکن تعمل Scan لـ رخصة قيادة وتبعثها لـ Textract.

تقدر تطلع منها بيانات زي:

- ID.
- First Name.
- Surname.
- Date of Birth.
- Male or Female.
- Issue Date.
- Expiry Date.

Invoice Example

لو عندك شركة Financial Services وعندك Invoices كثير، تقدر ترفع الـ Invoices لـ Textract.

Textract تقدر تحول المعلومات الموجودة في الـ Invoice إلى Data منظمة في Table.

الفكرة الأساسية إن Textract بتقدر Extract الـ Text من أي Scanned Document.

Amazon SageMaker

Amazon SageMaker بتساعد Developers و Data Scientists إنهم:

- Build Machine Learning Models.
- Train Machine Learning Models.

لو لقيت في الامتحان Scenario بيتكلم عن Build أو Train لـ Machine Learning Models، فكر في Amazon SageMaker.

Video 15 – Security, WAF, Shield, Secret Manager, KMS, GuardDuty, Cognito

AWS WAF



Web Application Firewall

الـ AWS WAF هي Web Application Firewall، وبتستخدم علشان تمنع Attacks في الـ HTTP Layer، زي:

- SQL Injection.
- XSS Attacks.

Supported Services

يمكن نعمل Activate للـ WAF على أكثر من Service في AWS، ومن أشهرهم:

- CloudFront.
- Application Load Balancer.

الـ Application Load Balancer يشتغل على Layer 7 مع HTTP و HTTPS، ولما يستقبل Request ببيعه على أكثر من EC2 Instance في الـ Backend.

ده بيساعد إن الـ Load يتوزع على أكثر من Instance، وفي نفس الوقت لو Instance وقعت، باقي الـ Instances تفضل شغالة.

Web ACL

علشان نستخدم الـ WAF بنعمل حاجة اسمها Web Access List أو Web ACL.

من خلالها بنحدد إيه اللي عايزين نعمل له Inspection في الـ HTTP Layer.

الـ Web ACL بتحتوي على Rules، والـ Rules دي ممكن تكون جاهزة أو نعملها بنفسنا.

Managed Rules

AWS عندها Rules معمولة Pre-configured.

يعني لو عايز تحمي نفسك من Attack معين زي SQL Injection، ممكن تختار الـ Set of Rules المناسبة من الـ Managed Rules.

موجود أكثر من Set of Rules، وبنختار حسب نوع الـ Attack اللي عايزين نحمي نفسنا منه.

الـ Managed Rules بيكون عليها Fees.

Custom Rules

ممكن بدل الـ Managed Rules نعمل الـ Rules بتاعتنا بنفسنا لو عارفين بالظبط عايزين نبص على إيه في الـ HTTP Request وإمتى نعمل Block.

مثال: ممكن أعمل Rule تقول إن لو الـ HTTP Request فيه كلمة **admin** في الـ Body، يحصل Drop.

وممكن أعمل Rule تانية إن لو الـ Username هو **CloudCode**، يحصل Block للـ Request.

AWS Shield



Shield Standard

AWS Shield فيه نوعين، أول نوع هو Shield Standard.

Shield Standard يعمل Protection ضد DDoS Attacks على:

- Layer 3.
- Layer 4.

والنوع ده بيكون بدون Cost.

DoS and DDoS Attacks

فكرة DoS إن User واحد يفتح عدد كبير من Sessions على الServer.

الServer عنده Capacity معينة لعدد Sessions اللي يقدر يستحملها، ولما عدد كبير جداً من Sessions يتفتح عليه، ممكن الServer يقع.

الDDoS بيعمل نفس الفكرة ولكن باستخدام عدد كبير من Bots.

مثلاً ممكن المهاجم يستخدم Bot 100، وكل Bot يفتح حوالي 550 ألف Session مع الServer، وبالتالي الServer ممكن يقع.

علشان كده بنحتاج نحمي الServer من DDoS Attacks، لأنها حاجة صعب نعمل لها Control بنفسنا.

Shield Advanced

النوع الثاني هو Shield Advanced، وده عليه Fees.

Shield Advanced بيوفر Protection كمان على Layer 7.

كمان بيكون عندنا Access لفريق اسمه Shield Response Team.

الفريق ده فيه Engineers متخصصين في DDoS Attacks، ولو حصلت مشكلة نقدر نفتح Ticket معاهم علشان يساعدونا في الResponse.

WAF Practical Lab

Application Load Balancer Setup

عندي Application Load Balancer معمول بالفعل.

الApplication Load Balancer مربوط بـ Target Group، والTarget Group بيرمي على EC2 Instance عليها Web Server.

الDNS بتاع الApplication Load Balancer بيروح للـ Load Balancer، والLoad Balancer بعد كده بيرمي الRequest على الEC2 Instances الموجودة وراه.

الWeb Page شغالة بشكل بسيط، والمطلوب إن لما يتبعث HTTP Request والUsername يكون الRequest يحصل له Block. **CloudCode**

في البداية الـ **CloudCode** Username كان شغال عادي.

Creating a Web ACL

من AWS Console:

1. ندخل على WAF & Shield.
 2. ندخل على Web ACLs.
 3. نختار Create Web ACL.
 4. نسمي الـ Web ACL مثلاً **CloudCode WAF**.
 5. نعمل Association بينها وبين الـ Application Load Balancer.
- ممکن کمان نعمل Association مع CloudFront.

Using Managed Rules

أثناء إنشاء الـ Web ACL نقدر نختار Managed Rules.

هتلاقي أكثر من Rule معموله Pre-configured بالفعل، فنختار الـ Rules اللي محتاجينها ونكمل إنشاء الـ Web ACL.

بالنسبة للـ Pricing، نقدر نفتح WAF Pricing ونشوف التكلفة.

Creating a Custom Rule

لو عايزين نعمل Rule بنفسنا، نبدأ إنشاء Web ACL جديدة، ونربطها بالـ Application Load Balancer.

بدل ما نختار Managed Rules، نختار My Own Rule.

عملت Rule باسم مرتبط بـ CloudCode User-Block.

الـ Rule بتعمل Match على الـ Query String.

الشرط إن الـ Query String تحتوي على الـ String:

CloudCode

لو الـ Request فيه **CloudCode**، الـ Action يكون Block.

ولو مفيش Match، أي Request ثاني يكون Allow.

بعد كده بنكمل إنشاء الـ Web ACL.

Testing the Custom Rule

بعد إنشاء الـ Web ACL جربت أبعت نفس الـ Request مرة ثانية بالـ Username:

CloudKode

الـ Request رجع Forbidden، لأن الـ WAF وقف الـ Request.

Data Encryption



الـ Data Encryption معناها تشفير الـ Data.

في AWS عندنا نوعين أساسيين من الـ Encryption.

Encryption in Transit

Encryption in Transit سيكون للـ Data وهي بتتنقل من مكان لمكان.

مثلاً عندي User يبيع Traffic إلى Server أو Application.

الـ Traffic وهو بيتحرك بين الـ User والـ Application محتاج يبقى Encrypted.

ده اسمه Encryption in Transit.

Encryption at Rest

Encryption at Rest بيكون للـ Data وهي Stored بالفعل.

مثلاً عندي EBS Volume عليه File، وعايز الـ File وهو Stored على الـ Volume يكون Encrypted.

أو عندي صورة موجودة على S3، وعايز الصورة وهي موجودة على S3 تكون Encrypted.

لو حد دخل على S3، ما يقدرش يشوف الـ Data إلا باستخدام الـ Key المطلوبة.

AWS Certificate Manager

AWS Certificate Manager أو ACM بتساعدنا في Encryption in Transit.

من خلالها نقدر نعمل Issue للـ Certificates.

مثلاً عندنا Application Load Balancer، والـ Application Load Balancer ممكن يكون عليه Certificate.

الـ Certificate بتستخدم في الـ Encryption عن طريق SSL أو TLS.

بدل ما الـ Connection تكون HTTP وتبقى Clear، تبقى HTTPS والـ Traffic يبقى Encrypted.

لما الـ User يفتح الـ Web Application من خلال الـ Application Load Balancer، الـ Load Balancer بيعت له Certificate.

نقدر نعتبر الـ Certificate زي الـ Encryption Key.

لما الـ User بيعت Data للـ Application Load Balancer، الـ Data تبقى Encrypted، والـ Application Load Balancer هو اللي يقدر يفك التشفير.

AWS Certificate Manager هي الـ Service اللي نقدر من خلالها نعمل Issue للـ Certificate ونستخدمها مع Services زي الـ Application Load Balancer.

AWS KMS

Manage و Create Service هي AWS Key Management Service KMS بتخلينا نعمل Encryption Keys.

لو عندي File على EBS أو صورة على S3 وعمايز أشفرها، بيكون فيه Encryption Key مستخدمة علشان تعمل Encryption للـ File أو الصورة.

الـ KMS هي الـ Service اللي بتدير الـ Encryption Keys دي، وبتستخدم مع Services زي:

- S3.
- EBS.
- RDS.

KMS Key Types

فيه نوعين اتكلمنا عنهم:

- Customer Managed Key.
- AWS Managed Key.

Customer Managed Key

في الـ Customer Managed Key إحنا بنقدر نحدد الـ Policy الخاصة بالـ Key.

نقدر نحدد:

- مين يتحكم في الـ Key.
- إمتى الـ Key تبقى Enabled أو Disabled.
- Key Rotation.
- حذف الـ Key.

مثلاً ممكن نحدد Rotation كل 30 يوم، أو حذف الـ Key بعد 20 يوم.

AWS Managed Key

في الـ AWS Managed Key إحنا مش بنحدد الـ Policy الخاصة بالـ Key بنفسنا.

AWS هي اللي بتدير حاجات زي:

- مين يستخدم الـ Key.
- Enable و Disable.
- Delete.
- Key Rotation.

S3 Encryption with KMS

لو User عنده Document وعايز يرفعها على S3، بعد رفعها بيكون فيه Encryption Key نقدر نخلي S3 تستخدمها علشان تشفر الـ Data وهي Stored على S3.

فيه Mechanism بين KMS و S3 علشان الـ Encryption Key تستخدم بطريقة Secure، وفي النهاية الـ Document تكون Encrypted.

Object and Key Access

لما الـ File يترفع على S3 ويبقى Encrypted بالـ Encryption Key، مش أي حد يقدر يشوفه.

الشخص محتاج حاجتين:

- Access على الـ S3 Object نفسه.
- Access لاستخدام الـ Key علشان يعمل Decrypt.

الـ Document لما تترفع على S3 بتبقى Object.

علشان User يقدر يشوف الـ Object المشفر، لازم يبقى عنده Access على الـ Object نفسه، وفي نفس الوقت Access على الـ Key اللي مستخدمة في الـ Encryption.

KMS Practical Lab

Users

عندي في IAM اثنين Users:

CloudCode.
Test.

الـ CloudCode User سيكون Administrator على الـ Key.

الـ Test User في البداية مش هيكون موجود ضمن الناس اللي مسموح لهم يستخدموا الـ KMS Key.

الـ Test User عنده S3 Read Only Access، يعني يقدر يعمل Read للـ S3 Resources، لكن معندوش Access على الـ Key.

Creating the KMS Key

دخلت على Key Management Service وعملت Create للـ KMS Key جديدة.

اخترت Symmetric Key.

سميت الـ Key مثلاً:

CloudCode Key 1

بعد كده اخترت الـ CloudCode User كـ Administrator على الـ Key.

الـ Administrator يقدر يعمل حاجات زي:

- Delete الـ Key.
- Rotation.
- Assign Users.

بعد كده بنحدد مين يبقى الـ User للـ Key.

الـ Key User هو الشخص اللي يقدر يستخدم الـ Key علشان يعمل Decrypt للحاجات المشفرة بيها.

اخترت **CloudCode** كـ Key User، وما اخترتش **Test**.

بعد كده كملت إنشاء الـ Key.

Creating the S3 Bucket

عملت S3 Bucket جديدة.

في إعدادات الـ Encryption اخترت Server-side Encryption باستخدام KMS.

بعد كده اخترت الـ KMS Key:

CloudCode Key 1

بكده أي File يترفع على الـ Bucket يحصل له Encryption باستخدام الـ Key دي.

Uploading an Object

رفعت File على الـ S3 Bucket.

وأنا داخل بالـ CloudCode User، قدرت أفتح الـ Object بشكل عادي لأنه عنده Access لاستخدام الـ Key.

Testing the Test User

بعد كده دخلت بالـ Test User وحاولت أفتح الـ Object.

الـ Test User عنده Access على S3، لكن مش موجود ضمن الـ Users بتوع الـ KMS Key، وبالتالي ما قدرش يستخدم الـ Key علشان يعمل Decrypt للـ File.

Granting Key Access

رجعت للـ CloudCode User، ودخلت على الـ KMS Key، وضفت الـ Test User ضمن الـ Users اللي يقدرُوا يستخدموا الـ Key.

بعد كده رجعت تاني للـ Test User وفتحت نفس الـ S3 Object.

المرّة دي الـ Object فتح، لأن الـ Test User بقى عنده Access لاستخدام الـ Key.

AWS Secrets Manager



AWS Secrets Manager Service مهمة لأنها بتقدر تحتفظ بمعلومات زي:

- API Keys.
- Tokens.
- Passwords.

مثلاً عندي Application شغالة على EC2 Instance، والـ Application محتاجة تتصل بـ RDS.

بدل ما أحط Username والـ Password بتوع الـ RDS على الـ EC2 Instance نفسها، أقدر أحط الـ Credentials في Secrets Manager.

لو الـ Username والـ Password موجودين مباشرة على الـ Instance، أي حد يدخل على الـ Instance ممكن يعرفهم ويستخدمهم علشان يوصل للـ RDS.

الأفضل إن الـ Application تعمل Reference للـ Credentials من Secrets Manager.

Amazon Macie



Amazon Macie يستخدم Machine Learning علشان تعمل Discover وتحمي Sensitive Data زي PII الموجودة في S3.

مثلاً لو عندي S3 Bucket عليها Data فيها PII Information.

الـ PII ممكن تكون معلومات زي:

- الاسم.
- العنوان.
- معلومات تقدر تعرف منها الشخص.

الـ Keyword المهمة هنا إن Macie يستخدم Machine Learning مع PII في S3.

Amazon Cognito



Amazon Cognito Service بتساعد الـ Users في الوصول للـ Web و Mobile Applications.

لو عندي Web Application أو Mobile Application وعندي Users عايزين يعملوا Access، Cognito تقدر توفر Sign-in وتتعامل مع Authentication و Authorization.

كمان ممكن نستخدم Social Identity Provider.

مثلاً لو الـ User عنده Facebook Account، يقدر يستخدمه ونربطه بالـ Profile الخاص بيه في الـ Web أو Mobile Application.

AWS Artifact

AWS Artifact بنستخدمها لما نحتاج نتأكد من Compliance الخاصة بـAWS.

مثلاً ممكن نحتاج نعرف هل AWS متوافقة مع:

- PCI.
- GDPR.

لو عندي Requirement أو محتاج Certification معينة، ومستخدم Data Center بتاع AWS، ممكن أحتاج أتأكد إن AWS نفسها متوافقة مع الـCompliance المطلوبة.

AWS Artifact بتساعدنا نشوف المعلومات والتقارير الخاصة بالـCompliance.

Amazon GuardDuty



Amazon GuardDuty بتجمع Logs من مصادر مختلفة، زي:

- VPC
- DNS
- Workloads الموجودة في Account.

الهدف إنها تعمل Detection للـ Threats والـ Suspicious Activity.

GuardDuty مش Protection Service بشكل مباشر، لكنها Detection Service.

بتقدر تقول إن فيه Activity مش طبيعية في Workload أو Account معين.

بتجمع Logs من الـ Infrastructure والـ Applications علشان تساعد في اكتشاف إن فيه Activity غير طبيعية أو مشكلة أمنية.

Amazon Inspector



Amazon Inspector عبارة عن Scan Service بتعمل Scan على الـ Workloads.

مثلاً لو عندي EC2 Instances، أقدر أعمل Scan على Instances معينة، ومش لازم أعمل Scan عليهم كلهم.

الهدف إن Inspector تشوف لو فيه Vulnerability أو Network Exposure.

Network Exposure Example

يمكن Inspector تقول إن EC2 Instance معينة عندها TCP Port 23 مفتوح من الإنترنت.

Port 23 خاص بـ Telnet، وده ممكن يعرض الـ EC2 Instance لمشكلة، خصوصاً إن Telnet مش Encrypted.

حتى لو شخص داخل الشركة بيعمل Access للـ Instance باستخدام Telnet، الـ Session نفسها بتكون Clear Text ومش Encrypted زي SSH.

Inspector ممكن تطلع Report وتقول إن فيه Unintended Network Exposure.

وفي مثال تاني، ممكن تقول إن TCP Port 80 الخاص بـ HTTP Reachable من الإنترنت.

HTTP مش Secure، والـ Traffic بتاعه مش Encrypted.

Amazon Detective

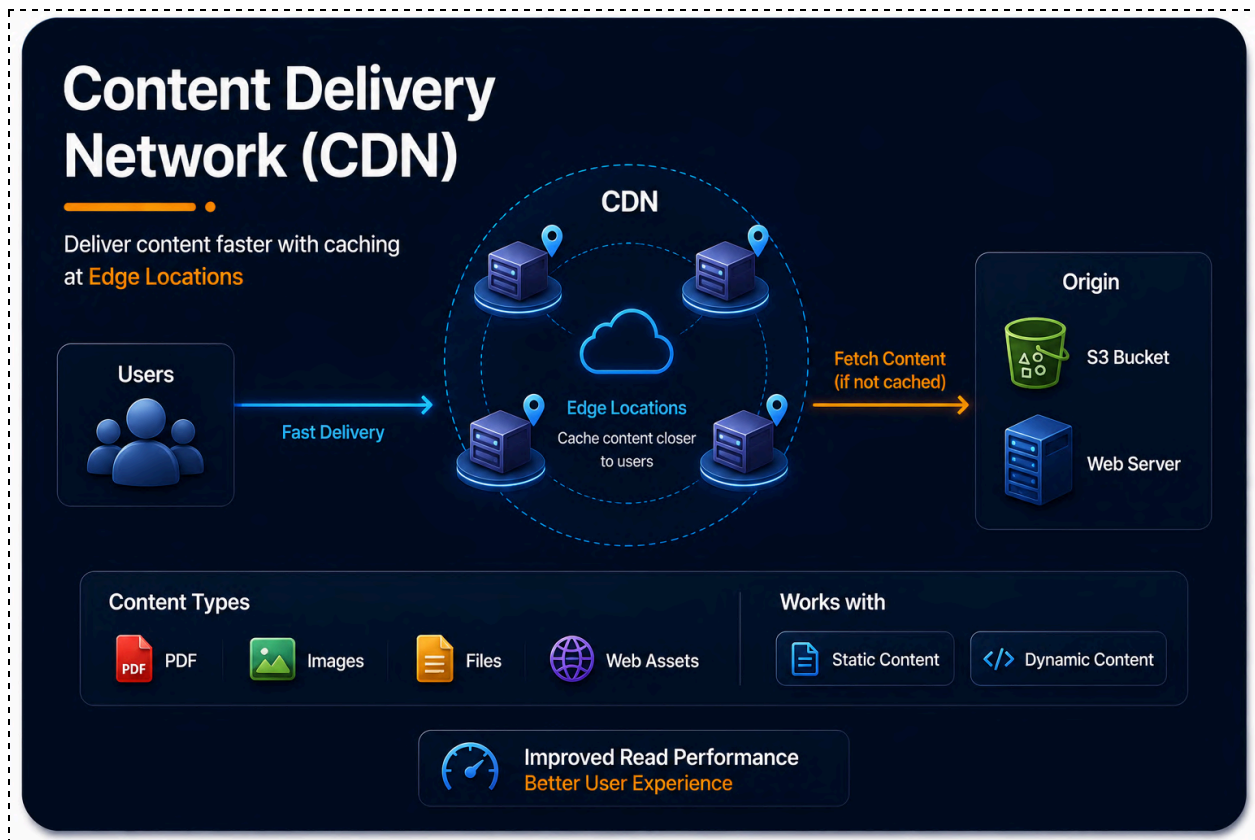
Amazon Detective الـ Keyword المهمة معاها هي Root Cause.

لما يحصل Amazon Detective Security Issue، بتساعد في توفير Root Cause للمشكلة.

بتحلل الـ Security Data الموجودة علشان تساعدنا نعرف سبب الـ Security Issue.

Video 16 – CloudFront, Route53, Outpost, Wavelength, local zone

Content Delivery Network



CDN Purpose

الـ CDN هي Content Delivery Network، وتستخدم علشان تحسن الـ Read Performance للـ Content.

الـ Content ممكن يكون زي:

- .PDF
- .Image
- File موجود على Server.

- File موجود على S3.
- Website فيها صور وملفات.

الهدف إن الـ User يقدر يعمل Read للـ Content ده بشكل أسرع.

الـ CDN بتعمل ده عن طريق إنها تعمل Cache للـ Content وتحطه في مكان اسمه Edge Location.

ده بيحسن الـ User Experience.

الـ CDN ممكن تستخدم مع:

- Static Content.
- Dynamic Content.

Regions and Availability Zones

Region

الـ Region هي Geographic Location موجود فيها AWS Data Centers.

مثلاً:

- North Virginia Region.
- London Region.

لما بنشتغل على AWS بنختار الأول الـ Region اللي عايزين نحط فيها الـ Resources بتاعتنا.

اتذكر إن AWS عندها تقريباً 33 Region.

Availability Zones

كل Region بيتكون من Availability Zones.

ممكن يبقى فيها أكثر من Availability Zone، ومش شرط يكونوا 3 بس.

كل Availability Zone بتبقى معزولة عن الـ Availability Zones التانية جوه نفس الـ Region.

الفكرة إن لو Availability Zone وقعت، يفضل فيه Availability Zones تانية شغالة.

مثلاً ممكن نخط EC2 Instance في Availability Zone، و EC2 Instance ثانية في Availability Zone مختلفة، والـ Application يستخدم الاثنين كـ Primary و Backup أو Active-Active. لو Availability Zone وقعت، يفضل فيه Availability Zone ثانية شغالة جوه نفس الـ Region.

Edge Locations

Edge Location Concept

تخيل إن عندي S3 أو EC2 Instance موجودة في London Region، وعليها Web Server أو صور وملفات.

لو عندي User في القاهرة وعاليز يحمل صورة موجودة في London Region، هيبقى فيه Latency بسبب المسافة بين القاهرة ولندن.

بدل ما الـ User يروح كل مرة للـ Region، نقدر نستخدم CDN ونخلي الـ Files تتعمل لها Cache في Edge Location أقرب للـ User.

اتذكر إن AWS عندها أكثر من 450 Edge Location موزعين في أماكن مختلفة في العالم.

الـ Edge Location تعتبر Data Center أقرب للـ Users، وبتستخدم مع الـ CDN علشان تعمل Cache للـ Content.

لو الـ User موجود في القاهرة، بدل ما يجيب الـ File من London Region، يقدر يجيبه من Edge Location أقرب إليه.

Edge Location Distribution

الـ Edge Locations بتبقى موزعة في مناطق مختلفة في العالم.

لما الـ User يطلب Content، يروح لأقرب Edge Location إليه، والـ Content يتقدم له منها بدل ما يروح للـ Origin البعيد.

Amazon CloudFront

Amazon CloudFront هي Service خاصة بالـCDN.

لو عندي S3 موجودة في Region بعيدة، والـUser موجود في القاهرة، بدل ما الـUser يروح مباشرة للـS3 في الـRegion، يقدر CloudFront يعمل Cache للـContent في أقرب Edge Location.

مثلاً لو عندي Request على:

cloudkode.net

بدل ما الـRequest يوصل للـServer أو الـS3 الموجودين في US Region، الـContent ممكن يتجاب مباشرة من أقرب Edge Location.

Origin Access Control

OAC Concept

Origin Access Control أو OAC بتستخدم لما CloudFront يحتاج Access على S3 Bucket مش Public.

لو الـS3 Bucket Public، أي حد يقدر يعمل Fetch للـInformation منها.

لكن لو الـS3 Bucket مش CloudFront، Public محتاج Permission علشان يقدر يعمل Fetch للـFiles والـImages.

الـOAC هي الـPermission أو الـPolicy اللي بنستخدمها علشان ندي CloudFront Access على الـS3 Bucket.

أثناء Configuration بتاع CloudFront:

1. بندي CloudFront Access على الـS3 Bucket.
 2. بنأخد Policy ونحطها على الـS3 Bucket.
 3. الـS3 Bucket تسمح لـCloudFront إنها تعمل Fetch للـFiles والـImages.
-

CloudFront Origins

Origin Concept

لو عندي S3 Bucket فيها Image، فالـ S3 في الحالة دي اسمها Origin.

الـ Origin هو المكان الأصلي اللي موجود فيه الـ Content، زي:

- PDF.
- Image.
- File.

CloudFront بياخد الـ Content من الـ Origin ويحطه في Edge Locations.

CloudFront Scenario

مثلاً:

- عندي S3 في Paris فيها Image.
- عندي User في القاهرة.

بدون CloudFront، الـ User بيروح من خلال الإنترنت للـ S3 الموجودة في Paris علشان يشوف الصورة.

مع CloudFront، الصورة تتعمل لها Cache في Edge Location أقرب للقاهرة، والـ User ياخد الصورة من هناك بدل ما يروح لـ Paris.

CloudFront Practical Lab

S3 Origin

عملت S3 Bucket ورفعت فيها Image.

الـ Image دي هي الـ Object اللي عايز أعمل لها Cache من خلال CloudFront في الـ Edge Locations.

خليت الـ Image Public علشان ما نحتاجش نعمل OAC في الـ Lab ده.

قدرت أفتح الـ Image مباشرة من الـ Public Link الخاص بـ S3.

Creating a CloudFront Distribution

من AWS Console دخلت على CloudFront.

أثناء إنشاء الـ Distribution:

1. اخترت الـ Origin.
2. الـ Origin كان الـ S3 Bucket اللي اتعملت قبل كده.
3. حددت إن الـ S3 Public.
4. ما استخدمتش Origin Access Control.
5. ما استخدمتش WAF في الـ Configuration دي.
6. اخترت Create Distribution.

بعد ما الـ Distribution تتعمل، CloudFront بتطلع Domain خاص بالـ Distribution.

Accessing the Cached Object

بعد ما CloudFront يخلص Deployment، نستخدم الـ Domain Name الخاص بالـ Distribution.

بعد الـ Domain Name بنضيف Path بتاع الـ Object.

مثلاً لو اسم الصورة:

CloudKodeText.png

بنستخدم الـ CloudFront Domain ومعاه اسم الصورة.

الصورة في الحالة دي بتكون Served من CloudFront عن طريق الـ Edge Location.

أي PDF أو Image أو File أو Object يترفع على الـ S3 Origin نقدر نوصل له من خلال CloudFront باستخدام نفس الـ Distribution.

CDN Performance Test

استخدمت CDN Performance Tool علشان أقارن الوصول لنفس الصورة مباشرة من S3 والوصول لها من خلال الـ CDN.

الفكرة إن الـ Tool تعمل Test من أماكن مختلفة في العالم.

S3 Performance

الصورة موجودة في S3 في US.

لو Request جاي من New York، الوقت كان حوالي:

63 ms

لأن الصورة موجودة في أمريكا.

لو Request جاي من Amsterdam، الوقت كان حوالي:

281 ms

ومن London كان حوالي:

305 ms

CDN Performance

لما نفس الصورة بقت موجودة على الـ CDN، لو Request جاي من Amsterdam، الوقت كان حوالي:

29 ms

ده بيوضح الفرق في الـ Performance.

نفس الصورة لو جايها مباشرة من مكانها في أمريكا ممكن توصل لحوالي 280 ms من Amsterdam، لكن باستخدام الـ CDN وصلت لحوالي 29 ms.

Amazon Route 53



Amazon Route 53 هي DNS Service في AWS، وهي Managed DNS Service مقدمة من AWS.

DNS Concept

لو عندي EC2 Instance عليها Public IP، ممكن أوصل للـ Web Server عن طريق كتابة:

```
http://<public-ip>
```

لكن بدل ما الـ Users يحفظوا الـ IP، الأفضل يستخدموا Domain Name زي:

```
www.cloudcode.net
```

الـ DNS Server بيكون عليه Records بتربط الـ Domain Names بالـ IPs.

مثلاً:

- Google → IP معين.
- CloudKode.net → IP معين.
- Facebook → IP معين.

الـ User يكتب الـ URL، والـ DNS Server يرجع له الـ IP المرتبط بالـ Domain.

DNS Record Types

A Record

الـ A Record يترابط الـ Domain Name بالـ IP Address.

مثال:

cloudkode.net → IP

CNAME Record

الـ CNAME يترابط الـ Name بالـ Name ثاني.

ممكّن نحتاجها مثلاً مع الـ Application Load Balancer.

الـ Application Load Balancer يبطلع الـ DNS Name طويل، وإحنا مش عايزين الـ User يستخدم الاسم ده.

نقدر نعمل الـ CNAME بحيث:

www.cloudkode.net

يبقى الـ Mapped للـ DNS Name الخاص بالـ Application Load Balancer.

الـ User يكتب الـ Domain العادي، والـ DNS في الـ Background يوصل للـ Name الخاص بالـ Load Balancer.

Alias Record

الـ Alias Record تخص الـ AWS.

نقدر نستخدمها علشان نعمل الـ Map للـ DNS Record على الـ AWS Service.

مثلاً نقدر نعمل Map على:

- CloudFront Distribution
- S3 Static Website
- Services تانية داخل AWS

لو CloudFront طلعت Domain Name طويل، نقدر نعمل DNS Record باستخدام Alias بدل ما الـ Users يستخدموا الـ Domain الطويل.

Route 53 Routing Policies

Simple Routing

Simple Routing هي أبسط Routing Policy.

الـ Client يسأل Route 53 مثلاً:

cloudcode.net

Route 53 يرد عليه بالـ IP المرتبط بالـ Record.

Weighted Routing

Weighted Routing بتسمح إننا نوزع الـ DNS Responses بنسب مختلفة.

مثلاً:

- 70% من الـ Requests تترد بـ IP معين.
- 30% تترد بـ IP تاني.

بالتالي بعض الـ Clients هيتبع لهم IP الأول، والباقي هيتبع لهم IP التاني.

فكرة Routing Policy إن الـ DNS مش لازم يرد كل مرة بنفس الـ Record.

ممكن الـ Response تختلف حسب حاجات زي:

- Percentage.
- Geographic Location.
- Latency.

مثلاً:

- لو الـ User في أمريكا يتردد عليه بـ IP معين.
- لو الـ User في هولندا يتردد عليه بـ IP ثاني.
- لو Latency بشكل معين يتردد عليه بـ IP مناسب.

Route 53 Hosted Zones

Public Hosted Zone

Public Hosted Zone يستخدم مع Public Domain يقدر الناس يوصلوا له من الإنترنت.

مثلاً لو اشتريت:

cloudcode.net

تقدر جوه الـ Hosted Zone تعمل Records زي:

- www.cloudcode.net → IP.
- cdn.cloudcode.net → Alias.
- Subdomains → Load Balancer أو Services أو تانية.

Domain Registration

من Route 53 نقدر نعمل Register لـ Domain جديد.

قدامنا اختيارات:

- نشترى Domain جديد.
- ننقل Existing Domain إلى Route 53.

بعد كده نقدر نـ Manage الـ Domain من Route 53 وننشئ الـ DNS Records المطلوبة.

Private Hosted Zone

Private Hosted Zone يستخدم لدومين داخلي جوه الـ VPC.

ممکن نستخدمه بين EC2 Instances أو Resources جوه الـ VPC.

لو محتاجين Domain حقيقي الناس توصله من الإنترنت، بنستخدم Public Hosted Zone.

Route 53 Practical Lab

Web Server

عملت EC2 Instance عليها Web Server.

لما فتحت الـ Public IP باستخدام HTTP ظهرت صفحة:

Welcome to CloudCode

Creating an A Record

دخلت على Route 53، وبعد كده على الـ Public Hosted Zone الخاصة بـ:

cloudkode.net

عملت Create Record.

أنشأت Subdomain:

www.cloudkode.net

واخترت الـ Record Type يكون A Record.

ربطت الـ Record بالـ Public IP الخاص بالـ EC2 Instance.

بعد ما الـ DNS Record اتعمل واستنتيت حوالي 60 ثانية، قدرت أفتح:

www.cloudkode.net

وظهرت نفس الـ Web Page الموجودة على الـ EC2 Instance.

ده معناه إن الـ DNS قدر يعمل Resolve للـ Domain إلى IP الخاص بالـ EC2 Instance.

Hosted Zone Structure

الـ Hosted Zone ممكن تكون:

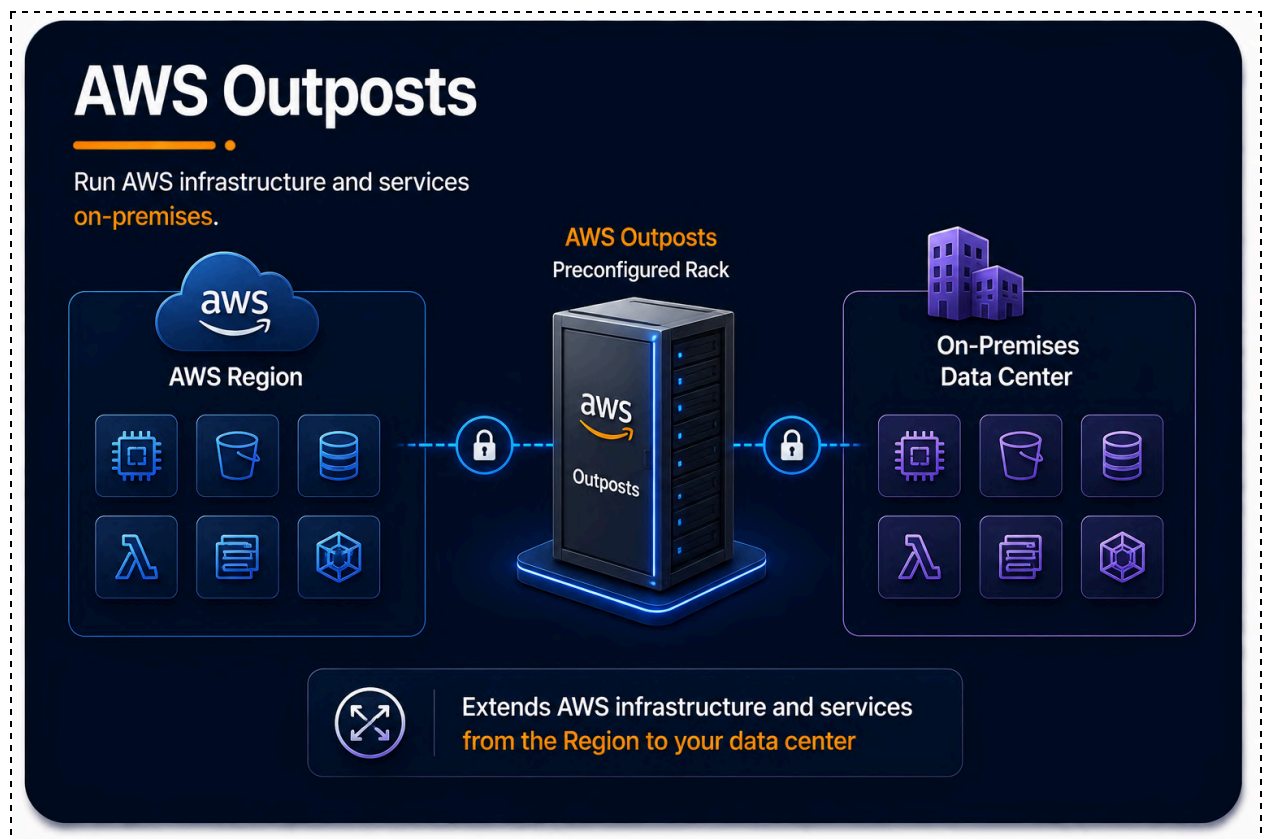
- Public.
- Private.

جوه الـ Hosted Zone نقدر نعمل Create للـ Records، ونحدد نوع الـ Record، ونختار Routing Policy.

من الـ Routing Policies اللي اتذكرت:

- Simple.
- Weighted.
- Latency.
- Failover.

AWS Outposts



AWS Outposts بتستخدم لما نكون مش عايزين نحط كل الـ Workloads في AWS Region، وعايزين نحط بعض الـ AWS Infrastructure والـ Services عندنا On-Premises.

AWS بتوفر Rack جاهز Preconfigured ويتخط داخل الـ On-Premises Data Center.

نقدر نشغل عليه مجموعة من AWS Services بدل ما تكون موجودة في الـ Region.

كأننا عملنا Extension للـ AWS Infrastructure والـ Services من الـ Region إلى الـ On-Premises Data Center.

Outposts Use Cases

اتذكر سببين مهمين لاستخدام Outposts.

Latency

لو الـ Users موجودين في القاهرة والـ Region موجودة مثلاً في دبي، ممكن الـ Latency تكون عالية.

ممكن نحط EC2 أو RDS أو Services ثانية محلياً في القاهرة على Outposts، وبالتالي الـ Interaction بين الـ Users والـ Services يبقى أسرع.

Data Residency

لو عندي مثلاً Bank ومش عايز الـ Data تخرج بره مصر، ممكن نستخدم Outposts ونحط الـ Database في الـ On-Premises Data Center داخل مصر بدل ما نحط الـ Data في Region بره البلد.

Supported Services

من الـ Services اللي اتذكرت إنها تشتغل مع Outposts:

- Load Balancer.
- S3.
- ECS.
- EKS.
- RDS.

Region Connectivity

مش كل Regions بتدعم Outposts، ول لازم نتأكد إن الـ Region المطلوبة بتدعم الموضوع ده.

الـ Outposts بتكون Connected بالـ Region.

الـ VPC الموجودة في الـ Region ممكن تمتد للـ Outposts، والـ Outposts يبقى فيها Subnets تعتبر جزء من نفس الـ VPC.

بيكون فيه Connectivity ما بين Outposts والـ Region عن طريق Anchor Point.

انتشبهت الفكرة بأن فيه Tunnel بين Outposts والAnchor Point.

لازم الس-On-Premises اللي فيه Outposts يقدر يوصل للRegion عن طريق:

- Direct Connect.
- Internet Connectivity.

AWS Wavelength

AWS Wavelength فكرتها قريبة من Outposts، لكن بتستخدم مع 5G Networks.

ممكن 5G Operator يحط AWS Wavelength داخل شبكة الـ 5G الخاصة بيه، ويقدر يعمل Connection مع الـ VPC الموجودة في الـ Region.

الهدف إننا نعمل Extension للـ AWS Infrastructure أقرب للـ 5G Users.

Wavelength بتدعم Ultra-Low-Latency Applications.

بدل ما الس-Mobile Users يروحوا للـ Application الموجودة في Region بعيدة، نقدر نحط الـ Application أقرب ليهم داخل Wavelength، وبالتالي نقلل الـ Delay بشكل كبير.

AWS Local Zones

Local Zone Concept

Local Zones فكرتها شبه إننا نقرب الس-AWS Services من الس-Users، لكن بدل ما نحط Rack Outposts داخل الس-On-Premises عند العميل، الس-AWS بتوفر Local Zones قريبة من مناطق معينة.

مثلاً لو الـ Region هي North Virginia وعندي User في Miami أو Boston، الوصول للـ Application في North Virginia ممكن ياخذ مثلاً:

100 ms

لو عايزين نقلل الـ Latency إلى حوالي:

10 ms

نقدر نخط الـApplication في Local Zone أقرب للـUser.

الـLocal Zone بتقرب:

- Compute.
- Storage.
- Database.

من الـUsers وLow-Latency Applications.

Supported Services

من الـServices اللي اتذكرت مع Local Zones:

- EC2.
- RDS.
- ECS.
- EBS.

Managing Local Zones

من AWS Console نقدر ندخل على Manage Local Zones ونشوف الـLocal Zones المتاحة في الـRegion الحالية.

في مثال North Virginia ظهرت Local Zones في أماكن زي:

- Atlanta.
- Boston.
- Buenos Aires.
- Chicago.
- Lima.
- Miami.

علشان نستخدم Local Zone معينة، بنعمل لها Enable من Manage Zone، وبعد كده نقدر نخط الـResources بتاعتنا فيها.

Video 17 – Organization, Cloudformation, Snowfamily, Billing&Costs

AWS Organizations

Organization Purpose

AWS Organizations يستخدم لما يكون عندي شركة كبيرة ومش مناسب إن كل الـ Departments والـ Environments تشتغل جوه AWS Account واحدة.

مثلاً ممكن يبقى عندي:

- Production.
- Development.
- Testing.
- HR Department.
- Finance Department.
- IT Department.

بدل ما كل الناس والـ Resources تشتغل جوه Account واحدة، أقدر أفصل الـ Accounts عن بعض وأManage كل الـ Accounts من مكان واحد.

Management Account

الـ Account الرئيسية في AWS Organizations اسمها Management Account.

زمان كانت بتسمى Master Account.

الـ Management Account بتستخدم علشان:

- Create Organization.
- Create Accounts تحت الـ Organization.
- Manage الـ Accounts الموجودة جوه الـ Organization.

Consolidated Billing

من أهم مميزات AWS Organizations إنك تقدر تستخدم Consolidated Billing. بدل ما يكون عندك Bill منفصلة لكل Account، بيكون عندك Bill واحدة لكل Accounts الموجودة تحت الـ Organization. كمان كل الـ Usage والـ Volume والـ Traffic الخاص بالـ Accounts بيتحسب مع بعض. ده ممكن يساعد إنك تاخد Discount أكثر، خصوصاً مع حاجات زي Savings Plans، لأن الاستخدام بيتجمع بدل ما يتحسب كل Account لوحدها.

Organizational Units

Organizational Unit أو OU هي Logical Group بنجمع جواها Accounts مرتبطة ببعض. مثلاً ممكن أعمل:

- IT OU.
- HR OU.
- Finance OU.

وبعد كده أحط Accounts تحت كل OU. مثلاً:

- IT OU تحتها Account 1 و Account 2.
- HR OU تحتها Account 3 و Account 4.
- Finance OU تحتها Account 5 و Account 6.

الـ Accounts ممكن تنتظم تحت الـ OU المناسبة حسب الـ Department.

Service Control Policies

SCP Purpose

Service Control Policy أو SCP بتستخدم علشان تعمل Restrictions على الـ Accounts الموجودة جوه الـ AWS Organizations.

بالـDefault أي Account جديدة سيكون عليها Full Access، يعني مفيش Restriction من ناحية الـSCP.

لو عايز Restrict Services معينة، أقدر أعمل SCP.

مثلاً ممكن أعمل Policy تمنع Access على:

- EC2.
- S3.

الـSCP ممكن تتطبق على:

- OU Level.
- Account Level.

OU Level Policies

لو طبقت SCP على OU معينة، كل الـAccounts الموجودة تحت الـOU دي بتتأثر بالـPolicy.

مثلاً لو عندي OU وعملت عليها Deny لـS3، كل الـAccounts الموجودة تحتها هتورث الـRestriction دي.

Account Level Policies

ممكن كمان أطبق SCP على Account معينة بشكل مباشر.

مثلاً لو Account موجودة تحت OU عليها Deny لـS3، وبعد كده طبقت على الـAccount نفسها Deny لـEC2، فالـAccount دي هيبقى عليها:

- Deny S3.
- Deny EC2.

Policy Impact

الـSCP بتأثر على:

- Users.
- Roles.
- Root User.

يعني لو Service معموله لها Deny من خلال SCP، حتى الـRoot User جوه الـAccount مش هيقدر يستخدم الـService دي.

يمكن كمان نستخدم SCP علشان نحدد Region معينة ونخلي إنشاء الـ Resources مسموح بس فيها.

AWS Organizations Structure

في أعلى الـ Hierarchy بيكون عندنا Root الخاصة بالـ Organization.

تحتها بيكون الـ Management Account.

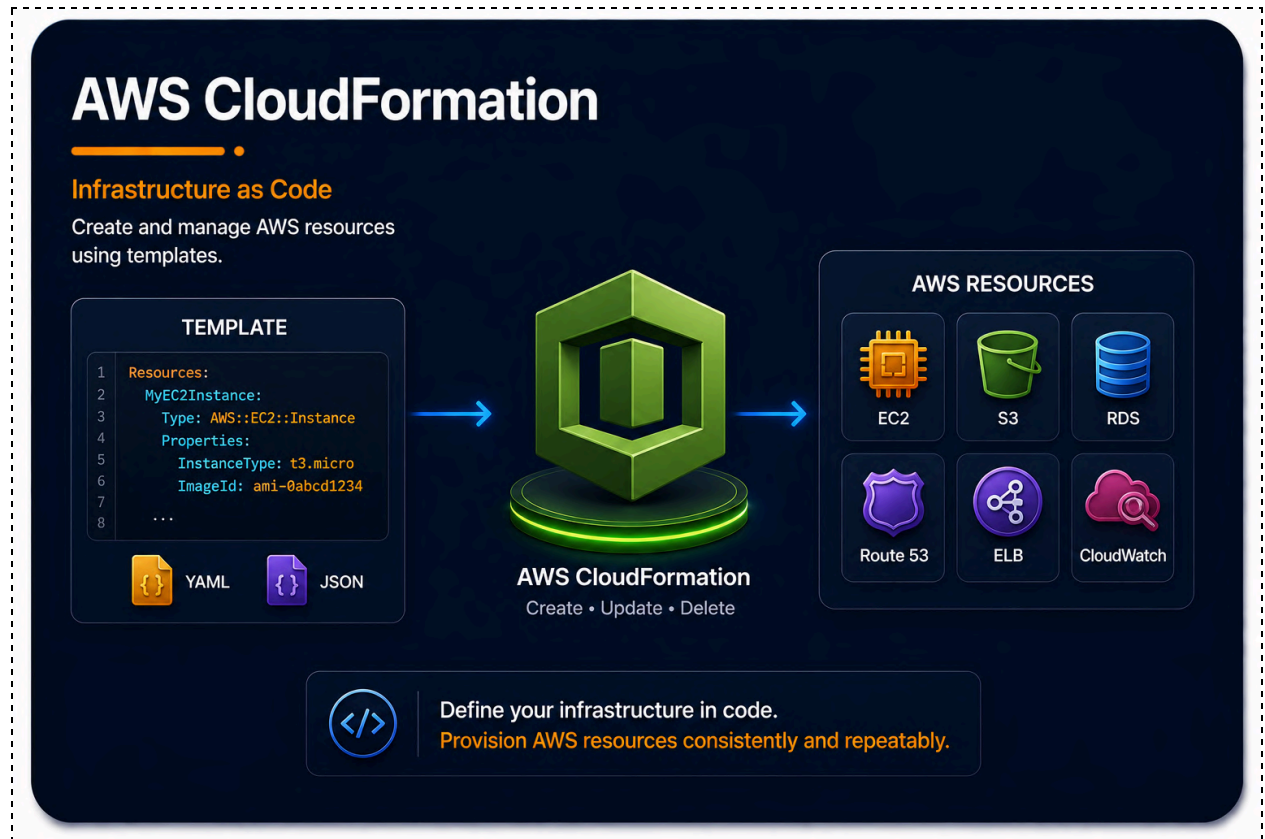
بعد كده نقدر نعمل Organizational Units حسب الـ Departments، زي:

- IT.
- HR.
- Finance.

وتحت كل OU نخط الـ Accounts الخاصة بيها.

يمكن كمان نعمل Invitation لـ Existing AWS Account علشان تنضم للـ Organization بدل ما نـ Create Account جديدة.

CloudFormation



Infrastructure as Code

AWS CloudFormation تعتبر Infrastructure as Code.

بتستخدم علشان نقدر ن Create AWS Resources من خلال CloudFormation Template.

الTemplate بتكون عبارة عن Code مكتوب بصيغة:

- YAML.
- JSON.

Resource Definition

جوه الـTemplate نقدر نحدد الـResources اللي عايزين نـCreateها.

مثلاً:

- Security Group.
- EC2 Instance.
- S3 Storage.
- Load Balancer.

الـCloudFormation بتتعامل مع ترتيب إنشاء الـResources والـDependencies المطلوبة بينهم.

مثلاً لو محتاج Security Group الأول وبعدها EC2 Instance نستخدم الـSecurity Group، وبعد كده Load Balancer وCloudFormation بتتعامل مع ترتيب إنشاء الـResources.

CloudFormation Template Sources

أثناء إنشاء Stack من CloudFormation، بنحدد الـTemplate موجودة فين.

ممکن الـTemplate تكون:

- موجودة على S3.
- موجودة على Git.
- موجودة على الكمبيوتر ونرفعها.

أهم حاجة إن الـFormat يكون:

- JSON.
- YAML.

بعد ما نرفع الـTemplate ونكمل الخطوات، CloudFormation بتـCreate الـResources المحددة في الملف.

AWS Snow Family

Snowball Purpose

AWS Snowball بتستخدم لما يكون عندي Site معزول أو مفيهوش Internet، وعايز أنقل Data بين المكان ده وAWS.

AWS بتبعت Device Portable للمكان.

بنحمل الـData على الـDevice، وبعد كده نعمل Shipping للجهاز لـAWS علشان الـData تنتقل.

من أشهر الأماكن اللي ممكن الـData تنتقل ليها داخل AWS هي S3.

Snowball Device Types

اتذكر نوعين من الأجهزة:

- Storage Optimized.
- Compute Optimized.

الـStorage Optimized بيركز أكثر على الـStorage.

الـCompute Optimized بيكون فيه Compute أكبر من ناحية CPU وMemory مع Storage.

Data Migration Scenario

لو عندي Data في Site مفيهوش Internet وعايز أرفعها على S3:

1. AWS تبعت Snowball Device للموقع.
2. أحمل الـData على الجهاز.
3. أرجع أشحن الـDevice لـAWS.
4. AWS ترفع الـData على S3.

Snow Family Job Types

أثناء طلب Snow Family Device، ممكن أختار أكثر من نوع Job.

Import Data

AWS تبعت Empty Device.

أستخدمه للـStorage والـCompute في الموقع.

بعد ما أحمل عليه الـData، أرجعه لـAWS علشان الـData تنتقل إلى S3.

Export Data

ممكن يكون عندي Data موجودة بالفعل على S3 وعايز أخرجها من AWS للموقع.

في الحالة دي AWS تبعت الـDevice وعليه الـData المطلوبة.

Local Compute and Storage

يمكن استخدام الـ Device محليًا للـ Compute والـ Storage من غير ما يبقى فيه Data Transfer بعد كده لـ AWS.

EC2 Pricing Review

من أنواع Pricing الخاصة بـ EC2 اللي اتذكرت قبل كده:

- On-Demand.
- Reserved.

On-Demand

الـ On-Demand أو Pay As You Go معناه إنك بتدفع حسب الوقت اللي الـ Instance بتشتغل فيه.

Reserved

في الـ Reserved بتحدد إنك عايز الـ Instance لمدة معينة، زي سنة كاملة.

AWS Marketplace

AWS Marketplace مكان داخل AWS تقدر تشتري منه Software من Vendors مختلفين.

مثلاً لو عايز Firewall من Fortinet، بدل ما تـ Create EC2 Instance وبعدين تدور على الـ License وتعمل كل الـ Setup بنفسك، تقدر تدخل AWS Marketplace وتختار الـ Product وتعمل Deploy بسرعة.

يمكن تلاقي Products من Vendors زي:

- Fortinet.
 - Cisco.
-

AWS Pricing Calculator

AWS Pricing Calculator بتستخدم علشان تعمل Estimate للـ Cost قبل ما تستخدم AWS Services.

تحدد الـ Resources التي عايز تستخدمها، وبعد كده تدخل الـ Parameters الخاصة بيها.

مثلاً مع EC2 ممكن تحدد:

- Region.
- Operating System.
- Number of Instances.
- Instance Family.

بعد كده الـ Calculator تطلع الـ Estimated Cost للفترة الزمنية المطلوبة.

AWS Budgets

AWS Budgets بتستخدم علشان ما تتفاجئش بـ Bill أكبر من المتوقع.

ممكن تحدد Budget وتعمل Alert لما الـ Spending يوصل لقيمة معينة.

مثلاً ممكن أعمل Budget بقيمة:

\$10

وأحدد إن لو الـ Monthly Cost عدت الـ 10 دولار، يتبعلي Email Alert.

في مثال تاني، شركة حاطة Budget بقيمة:

\$500

وعايزة أول ما الـ Spending يوصل:

\$400

يتبع Notification أو Email.

AWS Budgets موجودة تحت Billing and Cost Management.

AWS Cost Explorer

AWS Cost Explorer بتستخدم علشان Track الـ Spending مع الوقت.

بتعرض الـCost في Graphs، ونقدر نعمل Filtering على أجزاء معينة.

مثلاً لو شركة دفعت الشهر اللي فات Cost أعلى بنسبة 20%، نقدر نستخدم Cost Explorer علشان نعرف الفلوس اتصرفت فين وإيه سبب الزيادة.

AWS Trusted Advisor

AWS Trusted Advisor بتعمل Analysis للـAWS Account وبتطلع Recommendations.

الـRecommendations ممكن تساعد في:

- تقليل Cost.
- تحسين Security.
- تحسين Performance.

Performance Example

ممكن Trusted Advisor تقول إن EC2 Instance معينة عندها CPU Utilization أعلى من:

90%

وده Finding مرتبط بالـPerformance.

Security Example

ممكن تقول إن Security Group معينة فاتحة Port خاص بـRDP على الإنترنت.

Trusted Advisor بتوضح النوع ده من الـFindings علشان نقدر نحسن إعدادات الـAccount.



Thanks.